Projeto e Análise de Algoritmos

Listas de Exercícios 1 a 5 — enunciados e resoluções

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

Sumário

Lista	Tema	Situação	Página
Lista 1	Notação assintótica (O, Ω, Θ), complexidade de laços	estudo — versão ajustada em 13/09/2026 (Q1(f) retirado)	2
Lista 2	Notação assintótica, algoritmos iterativos	estudo	10
Lista 3	Indução e relações de recorrência	estudo — revisada em 09/09/2026	17
Lista 4	Indução, Teorema Mestre, crítica de prova por indução	estudo — revisada em 09/09/2026	24
Lista 5	Divisão e conquista, busca num vetor sem fim, Teorema Mestre	avaliada (5 pontos), entrega em 24/09/2026	37

> Em cada lista, cada questão traz primeiro o enunciado do professor (transcrito literalmente, com eventuais erros de digitação marcados com [sic]) e em seguida a resolução. Este volume é a junção das cinco resoluções individuais, sem alteração de texto; a numeração de página de cada lista recomeça no próprio documento dela. Desde 06/10/2026 cada lista traz também o cabeçalho e as instruções do professor, os códigos das questões de laços transcritos linha a linha com a numeração original, e o anexo “Informações que podem ser úteis”.

Projeto e Análise de Algoritmos — Lista de Exercícios I

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Convenção de custo.** Salvo menção em contrário, o custo é medido em *operações primitivas* no modelo RAM, atribuindo 1 unidade a cada atribuição, comparação e incremento, conforme o guia de análise fornecido no enunciado. Escreve-se $T(x)$ para o custo de uma entrada específica x e $W(n) = \max\{T(x) : |x| = n\}$ para o custo de pior caso. Declarar a unidade importa: uma testemunha obtida contando comparações não vale automaticamente para operações primitivas.

Cabeçalho e instruções da lista (transcrição literal)

Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios I · Formiga,MG, 11 de agosto de 2026

Instruções:

1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1

Enunciado

Responda V para os itens que são verdadeiros e F para os itens que são falsos e justifique suas respostas. Respostas sem justificativas não serão consideradas.

a. Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em *algumas* entradas?
b. Se eu provar que um algoritmo leva $O(n^2)$ no pior caso, é possível que leve $O(n)$ em *todas* as entradas?
c. Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em *algumas* entradas?
d. Se eu provar que um algoritmo leva $\Theta(n^2)$ no pior caso, é possível que leve $O(n)$ em *todas* as entradas?
e. Se eu provar que um algoritmo leva $\theta(n^2)$ no pior caso, é possível existir alguma entrada suficientemente grande para a qual o algoritmo realiza pelo menos $100 * n$ operações primitivas?

> **Lista revisada pelo professor** (o cabeçalho da lista manteve a data original). Em 09/09/2026 o item (e) trocou $O(n^2)$ por $\theta(n^2)$ e entrou um item (f); em 13/09/2026 o professor retirou o item (f). Esta resolução segue a versão de 13/09. As versões anteriores estão arquivadas em `listas-de-exercicios/versoes-anteriores/`.

Os cinco itens giram em torno de uma única distinção: **O é teto e Θ é teto com piso**, e ambos falam do *máximo* $W(n)$, não de cada entrada individual. Por isso custo $O(n)$ em *algumas* entradas é possível — existe algoritmo em que isso ocorre —, e (a) e (c) são V com ou sem piso: o piso, quando existe, só obriga a entrada que realiza o máximo a ser cara; nada impede outra entrada de custar $O(n)$, e o insertion sort com a entrada já ordenada é uma testemunha (itens (a) e (c) abaixo). Já custo $O(n)$ em *todas* as entradas é incompatível com um piso de ordem n^2: (b) é V porque O não impõe piso nenhum, e (d) é F porque o piso $c_1 n^2$ do Θ obriga alguma entrada a custar pelo menos $c_1 n^2$, o que nenhuma cota $c \cdot n$ acompanha para n grande. O item (e) usa a mesma distinção: é o piso $c_1 n^2$ do θ que obriga alguma entrada grande a passar de 100n.

(a) V

Provar $W(n) = O(n^2)$ limita por cima o custo *máximo*. Nada é dito sobre entradas individuais, que podem custar muito menos que o teto.

Tome o *insertion sort*, contando comparações de chave. Em toda entrada de tamanho n o algoritmo executa no máximo $n(n-1)/2$ comparações, logo $W(n) \le \frac{1}{2}n^2$ para todo $n \ge 1$: temos $W(n) = O(n^2)$ com $c = \frac{1}{2}$ e $n_0 = 1$. Já numa entrada previamente ordenada em ordem crescente, o teste do `while` falha na primeira avaliação de cada iteração do `for`, e o custo é exatamente $n-1$ comparações, isto é, $O(n)$ com $c = 1$ e $n_0 = 1$.

As duas coisas convivem sem contradição: $\frac{1}{2}n^2$ é teto de todas as entradas, e as entradas ordenadas ficam bem abaixo dele. **Logo, é possível.**

(b) V

O ponto aqui é que **um teto pode ser folgado**. Provar $W(n) = O(n^2)$ não afirma que exista entrada custando da ordem de n^2; a notação O não impõe piso nenhum.

De fato, custar $O(n)$ em todas as entradas já *implica* $O(n^2)$ no pior caso: se $T(x) \le c{\cdot}n$ para toda entrada x de tamanho $n \ge n_0$, então $W(n) \le c{\cdot}n \le c{\cdot}n^2$ para todo $n \ge \max(n_0, 1)$, pois $n \le n^2$ quando $n \ge 1$.

Exemplo concreto: a busca linear executa no máximo n comparações de chave (com o elemento buscado) em qualquer entrada, logo é $O(n)$ em *todas* elas ($c = 1$, $n_0 = 1$) — e também é $O(n^2)$ no pior caso. A prova de $O(n^2)$ simplesmente não era justa. **Logo, é possível.**

(c) V

$W(n) = \Theta(n^2)$ significa que existem $c_1, c_2 > 0$ e n_0 com $c_1 n^2 \le W(n) \le c_2 n^2$ para todo $n \ge n_0$. O piso $c_1 n^2$ incide sobre o **máximo**: ele obriga a existir, para cada n grande, *alguma* entrada cara. Não obriga que *toda* entrada seja cara.

Insertion sort de novo, em comparações. No pior caso (vetor estritamente decrescente) a iteração j executa exatamente $j-1$ comparações *de chave*: são j avaliações do teste do `while`, mas a última sai por esgotamento do índice ($i = 0$) e nem chega a comparar chaves. Somando, $\Sigma_{j=2..n}\,(j-1) = n(n-1)/2$. Para $n \ge 2$ vale $n-1 \ge n/2$, logo $W(n) \ge \frac{1}{4}n^2$; e sempre $W(n) \le \frac{1}{2}n^2$. Testemunhas: $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, $n_0 = 2$.

Ainda assim, a entrada já ordenada custa $n-1$ comparações, que é $O(n)$. **Logo, é possível.**

(d) F

Aqui o quantificador é universal, e ele colide frontalmente com o piso do Θ.

Suponha as duas hipóteses simultaneamente. De $W(n) = \Theta(n^2)$ vêm $c_1 > 0$ e n_1 com $W(n) \ge c_1 n^2$ para $n \ge n_1$. De "$O(n)$ em *todas* as entradas" vêm $c > 0$ e n_2 com $T(x) \le c{\cdot}n$ para toda entrada de tamanho $n \ge n_2$; como $W(n)$ é o máximo de um conjunto de valores todos limitados por $c{\cdot}n$ — conjunto finito e não vazio, logo o máximo existe —, segue $W(n) \le c{\cdot}n$.

$$\text{Para todo } n \ge n_0 = \max(n_1, n_2):\ \ c_1 n^2 \le W(n) \le c{\cdot}n \implies c_1 n^2 \le c{\cdot}n \implies n \le c/c_1.$$

Isto é, *todo* $n \ge n_0$ teria de satisfazer $n \le c/c_1$, o que é falso: basta tomar $n^* = \max(n_0, \lfloor c/c_1 \rfloor + 1)$, que cumpre $n^* \ge n_0$ mas $n^* > c/c_1$. Contradição.

> O passo decisivo é $W(n) \le c{\cdot}n$, e ele só existe porque o limite $O(n)$ foi suposto para **todas** as entradas. Em (c), onde apenas *algumas* entradas eram $O(n)$, esse passo não pode ser dado — é exatamente aí que os dois itens se separam.

Logo, é impossível.

(e) V

Duas leituras do enunciado. "Alguma entrada suficientemente grande para a qual realiza pelo menos 100n operações" pode querer dizer (L1) que existe *alguma* entrada x, de tamanho n qualquer, com $T(x) \ge 100{\cdot}n$; ou (L2) que, para *todo* n a partir de algum ponto, existe entrada de tamanho n com $T(x) \ge 100{\cdot}n$. (L2) implica (L1). Aqui se prova (L2); por isso a resposta vale em qualquer das leituras.

Suponha $W(n) \in \theta(n^2)$, com testemunhas $c_1, c_2 > 0$ e $n_0 \ge 1$: $c_1 n^2 \le W(n) \le c_2 n^2$ para todo $n \ge n_0$. Defina

$$N = \max(n_0, \lfloor 100/c_1 \rfloor + 1).$$

Para todo $n \ge N$: de $n \ge \lfloor 100/c_1 \rfloor + 1 > 100/c_1$ vem $c_1{\cdot}n > 100$ e, multiplicando por $n > 0$, $c_1{\cdot}n^2 > 100{\cdot}n$; e de $n \ge n_0$ vem $W(n) \ge c_1{\cdot}n^2$. Encadeando, $W(n) > 100{\cdot}n$. Como $W(n)$ é o máximo de $T(x)$ sobre as entradas de tamanho n, ele é atingido por alguma entrada x_n, e $T(x_n) = W(n) > 100{\cdot}n$. (Se se preferir definir $W(n)$ como supremo, o argumento continua: como o supremo passa de $100{\cdot}n$, pela definição de supremo existe x com $T(x) > 100{\cdot}n$.)

Portanto, para *todo* $n \ge N$ existe uma entrada de tamanho n com mais de $100{\cdot}n$ operações — não só "é possível": é obrigatório. Quem força isso é o piso $c_1 n^2$ do θ. **Logo, é possível (e até obrigatório).**

> **Exemplo numérico.** Insertion sort, contando só as comparações de chave: no pior caso (vetor em ordem decrescente) são $C(n) = n(n-1)/2$ comparações, e $C(n) \in \theta(n^2)$ com $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, $n_0 = 2$ (para $n \ge 2$, $n - 1 \ge n/2$, logo $n(n-1)/2 \ge n^2/4$; e $n(n-1)/2 \le n^2/2$ para todo $n \ge 1$). Então $N = \max(2, \lfloor 100/(1/4) \rfloor + 1) = \max(2, 401) = 401$ e, para $n \ge 401$, $C(n) \ge n^2/4 > 100n$. Como cada comparação é pelo menos uma operação primitiva, o número de operações primitivas da entrada de pior caso também passa de $100n$. O limiar N da demonstração é suficiente, não o menor: as comparações da entrada de pior caso já atingem $100n$ em $n = 201$, onde $201{\cdot}200/2 = 20\,100 = 100{\cdot}201$.
> *Fencepost:* com $N = 400$, o elo $c_1 n^2 > 100n$ da demonstração falharia, pois $(1/4){\cdot}400^2 = 40\,000 = 100{\cdot}400$ — embora neste exemplo $C(400) = 79\,800$ ainda passe de $100{\cdot}400$. É o elo genérico que exige o "+1" em $\lfloor 100/c_1 \rfloor + 1$.

> **Sobre a versão original do item (e) e o item (f) retirado.** Na versão original da lista, o item (e) dizia "$O(n^2)$" (e não $\theta(n^2)$) com "é possível existir". Aquela versão também era V, mas por outro motivo: um algoritmo que faz exatamente $100n$ operações em toda entrada é $O(n^2)$ ($100n \le 100n^2$ para $n \ge 1$). A versão de 09/09 trazia ainda um item (f), retirado em 13/09, que perguntava se, com $O(n^2)$ no pior caso, há *obrigatoriamente* alguma entrada suficientemente grande com pelo menos $100n$ operações. A resposta era F — o algoritmo que só devolve $A[1]$ gasta um número constante K de operações, com $K < 100$; ele é $O(n^2)$ e nenhuma entrada chega a $100n$. Quem inverte a resposta é a troca de "é possível" por "obrigatoriamente" sob $O(n^2)$ (V → F); sob $\theta(n^2)$ as duas perguntas são V, porque o piso obriga. A resolução completa do (f) está em `geradores/dados/lista01/q1ef-limpo.md`.

Questão 2

> Enunciado
>
> Considere n representando um valor variável e k é um valor constante. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação *Big O*. Dê um limite assintótico tão firme quanto possível. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
>
> Os cinco códigos estão transcritos literalmente, cada um no início do seu item abaixo.

Todo laço `for (x = a; x <= b; x = x+1)` com m iterações custa 1 (inicialização) + $(m+1)$ testes + m incrementos + $m\cdot(\text{custo do corpo})$, ou seja $2m + 2 + m\cdot\text{corpo}$. É a regra 4 do guia do enunciado, e é dela que sai tudo abaixo. **Atenção ao ponto que mais engana:** o custo de um laço inclui os testes de condição, que ocorrem *mesmo quando o corpo não executa nenhuma vez*.

Código 1 — $O(n^2)$, e o limite é justo: $\Theta(n^2)$

> Enunciado — código 1 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i = 1; i <= n; i = i + 1)
3  {
4      for (j = 1; j <= n; j = j + 1)
5      {
6          soma = soma + 1;
7      }
8  }
```

O laço interno tem n iterações e corpo de custo 1, logo custa $2n + 2 + n = 3n + 2$. O laço externo tem n iterações e esse corpo:

$$T_1(n) = 1 + [2n + 2 + n(3n + 2)] = 3n^2 + 4n + 3.$$

Como $3n^2 + 4n + 3 \le 10n^2$ para todo $n \ge 1$, vale $T_1(n) = O(n^2)$ com $c = 10$, $n_0 = 1$; e $T_1(n) \ge 3n^2$, logo também $\Omega(n^2)$. Portanto $\Theta(n^2)$.

Código 2 — $O(n^2)$, e o limite é justo: $\Theta(n^2)$

> Enunciado — código 2 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i = 1; i <= n; i = i + 1)
3  {
4      for (j = 1; j <= n; j = j + 1)
5      {
6          for (c = j; c <= k; c = c + 1)
7          {
8              soma = soma + 1;
9          }
10     }
11 }
```

Este é o código-armadilha da questão, e ele engana nos *dois* sentidos.

O laço mais interno quase não contribui. Ele vai de $c = j$ até k, com k constante, logo executa $r_j = \max(0, k - j + 1)$ vezes. Como j percorre até n, a partir de $j = k+1$ ele **não executa nenhuma vez**. Suponho $k \ge 1$ (o enunciado só diz que k é constante; se $k \le 0$, todo r_j é 0, logo $K = 0$ e $T_2(n) = 4n^2 + 4n + 3$ — mesma classe; a fórmula $k(k+1)/2$ abaixo daria valor errado para $k \le -2$). Somando sobre j, para $n \ge k$:

$$K = \Sigma_{j=1..n}\, r_j = k + (k-1) + \cdots + 1 = k(k+1)/2 \ \text{— uma constante, independente de n.}$$

Mas o laço do meio continua custando n por iteração de i. Ele é executado n vezes e cada iteração paga o teste da condição e o incremento, tenha ou não trabalho interno. Custo do laço j: $1 + (n+1) + n + \Sigma_j(3r_j + 2) = 4n + 2 + 3K$. Logo

$$T_2(n) = 1 + [2n + 2 + n(4n + 2 + 3K)] = 4n^2 + (4 + 3K)n + 3, \quad \text{com } K = k(k+1)/2, \text{ válido para } n \ge k.$$

Como K é constante, $T_2(n) = \Theta(n^2)$: o teto sai de $T_2(n) \le (11 + 3K)n^2$ e o piso de $T_2(n) \ge 4n^2$, ambos para $n \ge \max(1, k)$. A constante 11 não é folga gratuita: com $c = 7 + 3K$ a desigualdade falharia em $n = 1$ sempre que $k \le 1$, pois $T_2(1) = 11 + 3K > 7 + 3K$.

> Duas conclusões erradas são comuns aqui. Dizer $O(n^3)$ por haver três laços aninhados é frouxo — o enunciado pede o limite *mais firme possível*, e o terceiro laço só contribui uma constante. E dizer $\Theta(n)$, contando apenas as execuções de `soma = soma + 1`, ignora os n^2 testes do laço j, que dominam o custo.

Código 3 — O(n²), e o limite é justo: Θ(n²)

Enunciado — código 3 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i = 1; i <= n; i = i + 1)
3  {
4      for (j = 1; j <= i; j = j + 1)
5      {
6          soma = soma + 1;
7      }
8  }
```

O laço interno agora tem i iterações, custando $2i + 2 + i = 3i + 2$. Somando sobre o laço externo e usando $\Sigma\, i = n(n+1)/2$:

$$T_3(n) = 1 + [2n + 2 + \Sigma_{i=1..n}(3i + 2)] = 1 + 2n + 2 + 3{\cdot}n(n+1)/2 + 2n = (3/2)n^2 + (11/2)n + 3.$$

Teto: $T_3(n) \le 10n^2$ para $n \ge 1$. Piso: $T_3(n) \ge (3/2)n^2$. Portanto $\Theta(n^2)$.

> O laço interno é executado em média $(n+1)/2$ vezes, e não n — o somatório $\Sigma\, i = n(n+1)/2$ é a origem da constante $3/2$ em vez de 3. A *classe* é a mesma do código 1; o que muda é o fator constante.

Código 4 — O(n²), e o limite é justo: Θ(n²)

Enunciado — código 4 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i= 1; i <= n; i = i + 1)
3  {
4      for (j = i; j <= n; j = j + 1)
5      {
6          c:= 1;
7          while ( c <= k )
8          {
9              soma = soma + 1;
10             c = c * 2;
11         }
12     }
13 }
```

O `while` é O(1). Ele duplica c a partir de 1 até ultrapassar k, logo executa $m_k = \lfloor \lg k \rfloor + 1$ vezes, **supondo k ≥ 1** (para $k \le 0$ o teste falha de saída e o laço não executa, isto é, $m_k = 0$; note que $\lg k$ nem está definido aí). Como k é *constante*, m_k é constante: o bloco formado pela atribuição `c := 1` seguida do `while` custa $3m_k + 2 = O(1)$ — o `while` sozinho custa $3m_k + 1$, e a unidade restante é a atribuição, que fica fora dele.

O laço j vai de i até n, portanto tem $L = n - i + 1$ iterações, e $\Sigma_{i=1..n}\, L = n(n+1)/2$. Fazendo a conta:

$$T_4(n) = (3m_k + 4){\cdot}n(n+1)/2 + 4n + 3.$$

O par de laços forma um triângulo: são $n(n+1)/2$ pares (i, j), cada um com custo constante.

Testemunhas. De $n(n+1)/2 \ge n^2/2$ vem o piso $T_4(n) \ge ((3m_k+4)/2){\cdot}n^2$, com $c_1 = (3m_k+4)/2$ e $n_0 = 1$. De $n(n+1)/2 \le n^2$, $4n \le 4n^2$ e $3 \le 3n^2$ (todas para $n \ge 1$) vem o teto $T_4(n) \le (3m_k+11){\cdot}n^2$, com $c_2 = 3m_k+11$ e $n_0 = 1$ — teto justo, com igualdade em $n = 1$. Portanto $\Theta(n^2)$, o mesmo do código 3, apenas com constante maior.

Código 5 — $O(n^2 \log n)$, e o limite é justo: $\Theta(n^2 \lg n)$

Enunciado — código 5 (transcrição literal, com a numeração de linhas da lista)

```
 1  soma = 0;
 2  for ( i= 1; i <= n; i = i + 1)
 3  {
 4      for (j = 1; j <= n; j = j + 1)
 5      {
 6          c = 1;
 7          while ( c <= n )
 8          {
 9              soma = soma + 1;
10              c = c * 5;
11          }
12      }
13  }
```

Aqui está a diferença em relação ao código 4: o `while` testa contra n, não contra k. Multiplicando por 5 a cada passo, ele executa $m(n) = \lfloor \log_5 n \rfloor + 1$ vezes, que *depende de n* e vale $\Theta(\log n)$. Note que a base é irrelevante assintoticamente, pois $\log_5 n = \lg n / \lg 5$.

Os dois laços externos dão n^2 pares (i, j), cada um pagando o `while`:

$$T_5(n) = n^2 \cdot (3m(n) + 4) + 4n + 3 = \Theta(n^2 \log n).$$

Formalmente, de $\log_5 n \le m(n) \le \log_5 n + 1$ saem o piso $T_5(n) \ge (3/\lg 5) \cdot n^2 \lg n$ e o teto $T_5(n) \le 12 n^2 \lg n$ para $n \ge 6$.

Código	Custo exato (operações primitivas)	Resposta (Big O, mais firme)	Por quê
1	$3n^2 + 4n + 3$	**$O(n^2)$**	dois laços completos, corpo $O(1)$
2	$4n^2 + (4+3K)n + 3$	**$O(n^2)$**	3º laço só soma constante $K = k(k+1)/2$; os n^2 testes do laço j dominam
3	$(3/2)n^2 + (11/2)n + 3$	**$O(n^2)$**	triângulo: $\Sigma i = n(n+1)/2$
4	$(3m_k+4) \cdot n(n+1)/2 + 4n + 3$	**$O(n^2)$**	triângulo × while constante (k é constante)
5	$n^2(3m(n)+4) + 4n + 3$	**$O(n^2 \log n)$**	while depende de n: $\Theta(\log n)$ por par (i, j)

Questão 3

Enunciado

Demonstre as seguintes propriedades relativas à notação assintótica.

a. $f(n) = 2^{117}$, $f(n) \in O(1)$
b. $f(n) = n^2 + 2 * n$, $f(n) \in O(n^2)$
c. $f(n) = n^2 - 2 * n + 3$, $f(n) \in \theta(n^2)$
d. $f(n) = 5 * n$, $f(n) \in \Omega(n)$
e. $f(n) = n^2 + 2 * n - 8$, $f(n) \in \Omega(n^2)$

As três definições do CLRS, que é o que se deve exibir em cada item:

Notação	Definição	O que exibir
$O(g(n))$	$\exists\, c > 0, \exists\, n_0 > 0 : 0 \le f(n) \le c \cdot g(n), \forall\, n \ge n_0$	um c e um n_0
$\Omega(g(n))$	$\exists\, c > 0, \exists\, n_0 > 0 : 0 \le c \cdot g(n) \le f(n), \forall\, n \ge n_0$	um c e um n_0
$\Theta(g(n))$	$\exists\, c_1, c_2 > 0, \exists\, n_0 > 0 : 0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n), \forall\, n \ge n_0$	c_1, c_2 e n_0

Duas exigências que derrubam prova: as constantes são escolhidas *antes* e **não podem depender de n**; e a definição exige $f(n) \ge 0$ na região $n \ge n_0$ — o que restringe a escolha de n_0 nos itens com termo negativo.

(a) $f(n) = 2^{117} \in O(1)$

Testemunhas: $c = 2^{117}$ e $n_0 = 1$.

Para todo $n \ge 1$ tem-se $0 \le 2^{117} \le 2^{117} \cdot 1$, com igualdade. A desigualdade não depende de n porque f é constante, então vale trivialmente em toda a região. Qualquer $c \ge 2^{117}$ também serve. ∎

> **O tamanho da constante não importa.** $2^{117} \approx 1{,}66 \cdot 10^{35}$ é um número enorme, mas não depende de n; a definição de O só exige que c seja uma constante positiva fixada antes de n, e 2^{117} é. (Versões anteriores desta resolução liam "2117": o expoente tinha se perdido na extração do texto do PDF. Corrigido em 06/10/2026.)

(b) $f(n) = n^2 + 2n \in O(n^2)$

Testemunhas: $c = 3$ e $n_0 = 1$.

Para $n \geq 1$ vale $n \leq n^2$, logo $2n \leq 2n^2$. Somando n^2 aos dois lados:

$$0 \leq n^2 + 2n \leq n^2 + 2n^2 = 3n^2, \quad \text{para todo } n \geq 1.$$

A não-negatividade é imediata, pois $n^2 + 2n > 0$ para $n \geq 1$. ∎

(c) $f(n) = n^2 - 2n + 3 \in \Theta(n^2)$

Testemunhas: $c_1 = 1/2$, $c_2 = 2$ e $n_0 = 1$.

Não-negatividade. O discriminante de $n^2 - 2n + 3$ é $4 - 12 = -8 < 0$ e o coeficiente líder é positivo, logo $f(n) > 0$ para todo n real. Nenhuma restrição vem daqui.

Limite superior ($c_2 = 2$). Basta $n^2 - 2n + 3 \leq 2n^2$, isto é, $0 \leq n^2 + 2n - 3$, ou $0 \leq (n+3)(n-1)$, verdadeiro para todo $n \geq 1$.

Limite inferior ($c_1 = 1/2$). Basta $\frac{1}{2}n^2 \leq n^2 - 2n + 3$, isto é, $0 \leq \frac{1}{2}n^2 - 2n + 3$, ou $0 \leq n^2 - 4n + 6$. Esse polinômio tem discriminante $16 - 24 = -8 < 0$ e líder positivo, logo é positivo para todo n.

Assim, $\frac{1}{2}n^2 \leq n^2 - 2n + 3 \leq 2n^2$ para todo $n \geq 1$. ∎

> Observe que $c_2 = 1$ *não* funcionaria com $n_0 = 1$: em $n = 1$ temos $f(1) = 2 > 1 = 1 \cdot g(1)$. Com $c_2 = 1$ seria preciso $n_0 = 2$. A checagem em $n = n_0$ exatamente é o que separa uma prova correta de uma quase-correta.

(d) $f(n) = 5n \in \Omega(n)$

Testemunhas: $c = 5$ e $n_0 = 1$.

Para todo $n \geq 1$ vale $0 \leq 5 \cdot n \leq 5n$, com igualdade. Qualquer $0 < c \leq 5$ também serve, pois então $c \cdot n \leq 5n$. ∎

(e) $f(n) = n^2 + 2n - 8 \in \Omega(n^2)$

Testemunhas: $c = 1$ e $n_0 = 4$.

É preciso $0 \leq 1 \cdot n^2 \leq n^2 + 2n - 8$. A segunda desigualdade equivale a $0 \leq 2n - 8$, isto é, $n \geq 4$. A primeira exige $f(n) \geq 0$: como $f(n) = (n+4)(n-2)$, isso vale para $n \geq 2$, e em particular para $n \geq 4$. Logo $n^2 \leq n^2 + 2n - 8$ para todo $n \geq 4$. ∎

> **Por que $n_0 = 4$ e não menos.** Este item é o que mais derruba prova, porque o n_0 é justo — não há folga:

n	$f(n) = n^2+2n-8$	$c \cdot g(n) = n^2$	$n^2 \leq f(n)$?
1	-5	1	não — e $f(n)$ é *negativa*, violando $0 \leq f(n)$
2	0	4	não
3	7	9	não
4	16	16	sim, com igualdade

> Escrever "$n_0 = 1$" aqui é erro, e é um erro que passa despercebido numa leitura rápida. Alternativamente, com $c = 1/2$ bastaria $n_0 = 3$: $\frac{1}{2}n^2 \leq n^2 + 2n - 8$ equivale a $0 \leq n^2 + 4n - 16$, válido para $n \geq 2{,}47\ldots$, ou seja, $n \geq 3$.

Questão 4

> Enunciado
> Suponha que um algoritmo para um certo problema consome $O(n^2)$ unidades de tempo, sendo n o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome apenas $\Omega(n)$ unidades de tempo. Devo ficar impressionado(a)?

Não. A afirmação do amigo é quase vazia, e o "apenas" do enunciado é justamente a isca.

Ω é limite inferior. Dizer que um algoritmo consome $\Omega(n)$ é dizer que ele gasta *pelo menos* da ordem de n. Isso não é promessa de eficiência — é o contrário: é uma garantia sobre o quão ruim ele é, no mínimo. Para se impressionar seria preciso um **teto** (uma afirmação em O) melhor que $O(n^2)$, e o amigo não deu nenhum.

A afirmação é compatível com um algoritmo péssimo. Um algoritmo $\Theta(n^5)$ também é $\Omega(n)$, pois $n^5 \geq n$ para $n \geq 1$. O mesmo vale para $\Theta(2^n)$. Ou seja, a frase do amigo não exclui nenhum desses casos.

O que ela de fato informa. Lida como cota do pior caso do algoritmo dele, ela pode não ser *totalmente* vazia: exclui todo custo que não fique, a partir de algum ponto, acima de algum múltiplo positivo $c \cdot n$ — por exemplo, um algoritmo $\Theta(\lg n)$ como a busca binária, que *não* é $\Omega(n)$. Tudo o que ela carrega é desse tipo, um *piso*, e por isso é *desfavorável* a ele: o amigo está dizendo que seu algoritmo gasta pelo menos da ordem de n. E se o problema em questão já exige tempo $\Omega(n)$ de qualquer algoritmo (por exemplo, quando a resposta depende de todos os n itens), a frase não informa nada.

> Cuidado com o argumento tentador "todo algoritmo que lê a entrada inteira já é $\Omega(n)$, logo a frase não diz nada". Ler a entrada inteira de fato implica $\Omega(n)$; o defeito do argumento é a premissa oculta de que *todo* algoritmo para o problema precisa ler a entrada inteira. Existem problemas que admitem algoritmos sublineares, que não leem tudo — e para eles a frase exclui alguma coisa. Para os problemas que exigem ler tudo, o argumento está certo e a frase é vazia, como diz o parágrafo acima.

Questão 5

> Enunciado
>
> Meu algoritmo para um certo problema consome $O(n^2)$ unidades de tempo, sendo n o tamanho de uma instância. Meu amigo diz ter outro algoritmo para o mesmo problema que consome $O(n * \lg n)$ unidades de tempo. Devo ficar impressionado(a)?

Sim, se eu souber algo a mais sobre o meu algoritmo — por exemplo, que o meu $O(n^2)$ é justo. Desta vez a comparação é ao menos entre grandezas do mesmo tipo (dois tetos), e $n \lg n$ é assintoticamente melhor que n^2. Mas atenção: das duas afirmações em O, tomadas isoladamente, não se conclui ganho nenhum — é a Ressalva 1 abaixo, e ela é o coração da questão.

Entre os dois tetos, o ganho assintótico é real. Vale $n \lg n = o(n^2)$ — a razão $n \lg n / n^2 = \lg n / n$ é decrescente para $n \geq 3$ e, como $n / \lg n$ é ilimitada (provado logo abaixo), fica menor que qualquer $\varepsilon > 0$ a partir de algum ponto —, e a recíproca falha: n^2 *não* é $O(n \lg n)$. De fato, suponha por absurdo que existam $c > 0$ e $n_0 \geq 2$ (podemos supor $n_0 \geq 2$, para que $\lg n > 0$) com $n^2 \leq c \cdot n \lg n$ para $n \geq n_0$. Dividindo por $n \lg n > 0$:

$$n / \lg n \leq c, \text{ para todo } n \geq n_0.$$

Mas $n / \lg n$ é ilimitada: tomando $n = 2^k$ temos $n / \lg n = 2^k/k$, e como $2^k \geq k^2$ para $k \geq 4$, segue $2^k/k \geq k$. Basta então escolher $k > \max(c, 4)$ com $2^k \geq n_0$ para contradizer a desigualdade. ∎

Ressalva 1 — O é teto, dos dois lados. As duas frases sozinhas não bastam: é preciso saber algo a mais sobre o meu algoritmo. *Basta* que o meu $O(n^2)$ seja justo, isto é, que meu algoritmo seja de fato $\Theta(n^2)$; mais geralmente, basta que meu custo de pior caso $W(n)$ cresça estritamente mais que $n \lg n$, isto é, $n \lg n = o(W(n))$ — um algoritmo $\Theta(n^{1{,}5})$, por exemplo. Aí o custo do amigo, no máximo $c \cdot n \lg n$, é $o(W(n))$. *Não* basta que W apenas não seja $O(n \lg n)$: um custo que oscila — n^2 nos n pares e n nos ímpares — não é $O(n \lg n)$, e mesmo assim um algoritmo $\Theta(n \lg n)$ do amigo perderia em todos os n ímpares suficientemente grandes. Se minha análise foi frouxa e ele é na verdade $\Theta(n \lg n)$, as duas garantias declaradas deixam de separar os algoritmos. Note a diferença: nesse caso eu perco a capacidade de *concluir* o ganho — não que o ganho deixe de existir. Ele pode existir ainda assim (o do amigo pode ser $\Theta(n)$), eu é que não consigo mais afirmá-lo a partir das duas frases.

Ressalva 2 — o teto dele também pode ser folgado. Simetricamente, $O(n \lg n)$ não impede que o algoritmo do amigo seja ainda melhor, por exemplo $\Theta(n)$. A afirmação dele é um limite superior, não uma descrição exata.

Ressalva 3 — constantes ocultas e o tamanho real de n. A notação assintótica descarta constantes, e elas importam na prática. Se meu algoritmo custa $2n^2$ e o dele $1000 \cdot n \lg n$, o meu é mais rápido em toda a faixa $2 \leq n \leq 6311$: para $n \geq 2$, $2n^2 < 1000\, n \lg n$ equivale a $n < 500 \lg n$, o que vale exatamente nessa faixa (em $n = 1$ o dele custa 0, pois $\lg 1 = 0$). O cruzamento relevante é $n^* \approx 6311{,}93$, de modo que de $n = 6312$ em diante o dele passa à frente. Se as instâncias reais forem menores que isso, o ganho assintótico não se converte em ganho prático.

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Análise de Algoritmos:

Não há um conjunto de regras para analisar um algoritmo. No entanto, conforme [AHO et al., 1983] podemos seguir o seguinte guia, considerando o modelo computacional *Random Access Machine* (RAM):

1. O custo de execução de uma instrução de atribuição, de leitura ou de escrita é O(1);
2. O custo de execução de uma sequência de instruções é determinado pela instrução de maior custo na sequência de instruções;
3. O custo de execução de um comando de decisão/seleção é composto pelo custo de execução das instruções no corpo do comando de decisão/seleção mais o custo para avaliar a condição. O custo para avaliar a condição é O(1), e
4. O custo de execução de um comando de repetição (laço/loop) é a soma do custo de execução do corpo do comando de repetição adicionado ao custo de avaliar a condição de término do comando de execução, multiplicado pelo número de iterações do comando de repetição. O custo para avaliar a condição de término é O(1).

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n + 1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$

onde c é uma constante arbitrária inteira ≥ 2.

Referências

- [AHO et al., 1983] AHO, A. V., HOPCORFT, J. E., and ULLMAN, J. D. (1983). Data Structures and Algorithms. Addison-Wesley.

Sobre esta resolução. As classes assintóticas da Questão 2 e as testemunhas (c, n_0) da Questão 3 foram conferidas por medição: os cinco laços foram transcritos em Python e suas operações primitivas contadas por força bruta, e cada desigualdade da Questão 3 foi testada em 300 000 valores de n. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final.

Atualização de 09/09/2026. Os itens (e) e (f) da Questão 1 foram refeitos para a versão revisada da lista. Texto-fonte verificado: `geradores/dados/lista01/q1ef-limpo.md`. Primeira rodada: 4 verificadores, 3 apontaram defeito numa nota lateral (a cota da busca linear estava errada na contagem usual); corrigida, a reverificação deu 4 de 4 sem defeito. Números conferidos em `verificacao/atualizacao-2026-09-chao-deterministico.py`.

Ajuste de 13/09/2026. O professor retirou o item (f) da Questão 1. O bloco dele saiu desta resolução (a versão de 09/09 está em `geradores/dados/lista01/historico/`), e o item (e) ganhou uma nota sobre a variante retirada. Verificação: 4 verificadores; um apontou que, no item (b), "n comparações" com c = 1 só vale contando comparações de chave — corrigido —, e a reverificação deu 4 de 4 sem defeito.

Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.

Projeto e Análise de Algoritmos — Lista de Exercícios II

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

---

> **Convenção de custo.** O custo é medido em *operações primitivas* no modelo RAM, atribuindo 1 unidade a cada atribuição, comparação e incremento, conforme o guia de análise fornecido no enunciado. **O custo de um laço inclui os testes de condição**, que ocorrem mesmo quando o corpo não executa — contar apenas o corpo mais interno leva a resposta errada. Escreve-se n^* para pontos de cruzamento entre duas funções de custo, símbolo distinto do n_0 que aparece nas definições assintóticas: são constantes independentes.

> Cabeçalho e instruções da lista (transcrição literal)
>
> Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios II · Formiga,MG, 16 de agosto de 2026
>
> **Instruções:**
>
> 1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
> 2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
> 3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1

> Enunciado
>
> Considere as informações abaixo e responda o que se pede. Justifique suas respostas à luz da teoria da complexidade de algoritmos. Questão inspirada em exercícios análogos propostos em [BRASSARD and BRATLEY, 1996].
>
> a. Considere os algoritmos A e B com funções de complexidade no pior caso, respectivamente, $\theta(n^3)$ e $\theta(n^2)$. Em alguns conjuntos de dados, o tempo de execução do algoritmo A é *menor* que o tempo de execução do algoritmo B, quando testados no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador). Forneça três possíveis explicações para esse fenômeno.
> b. Um pesquisador tem a sua disposição 2 algoritmos para resolver um determinado problema. Um algoritmo possui função de complexidade $O(n^2)$ e outro $O(n^3)$. Esse pesquisador escolheu o algoritmo com complexidade $O(n^3)$. Cite 4 razões que poderiam ter motivado essa opção.
> c. Considere os algoritmos A e B para resolver um determinado problema. O algoritmo A exige $10^8 * n$ instruções e o algoritmo B exige $10 * n^2$ instruções. Pode-se afirmar que o algoritmo A é sempre preferível em relação ao algoritmo B?

(a) Três explicações

As três abaixo são **mecanismos distintos**: a primeira é sobre os fatores constantes, a segunda sobre *quais entradas* foram testadas, e a terceira sobre o modelo de custo não corresponder à máquina real.

(i) Constantes ocultas e termos de menor ordem. Θ só afirma a existência de constantes, sem fixar seus valores: $T_A(n) = \Theta(n^3)$ significa apenas que existem $a_1, a_2, n_0 > 0$ com $a_1 n^3 \le T_A(n) \le a_2 n^3$ para $n \ge n_0$.

Tome $T_A(n) = n^3 + 10$ e $T_B(n) = 1000n^2$. As duas respeitam as classes dadas, e

$$T_A(n) < T_B(n) \iff n^3 + 10 < 1000n^2, \quad \text{verdadeiro para todo } 1 \le n \le 999.$$

Em $n = 999$: A faz 997 003 009 operações e B faz 998 001 000 — A vence. Em $n = 1000$ a ordem se inverte (1 000 000 010 contra 1 000 000 000). O cruzamento está em $n^* \approx 1000$, e ele é determinado pelas *constantes*, não pelo n_0 das definições — que pode perfeitamente valer 1.

(ii) Θ descreve o pior caso, e os dados testados podem não ser o pior caso de A. Aqui o mecanismo é outro: não são as constantes, é *a escolha das entradas*.

$T_A(n) = \Theta(n^3)$ no pior caso limita o *máximo* sobre as entradas de tamanho n; não diz nada sobre uma entrada específica. Se A for adaptativo — o *quicksort*, por exemplo, só degrada para $\Theta(n^2)$ com pivôs ruins — pode existir uma família de entradas em que o custo efetivo *de A* seja $c \cdot n + O(1)$. Se, ao mesmo tempo, o conjunto testado for justamente o pior caso de B, cujo custo ali é $d \cdot n^2$, então A é mais rápido sempre que

$$c \cdot n + O(1) < d \cdot n^2, \text{ o que vale para todo } n \text{ suficientemente grande.}$$

> Repare que o limiar *não* é simplesmente $n > c/d$: com um termo aditivo $e > 0$, a condição $c \cdot n + e < d \cdot n^2$ só vale a partir de $n > (c + \sqrt{(c^2 + 4de)})/(2d)$. Com $c = 3$, $d = 1$, $e = 100$, isso dá $n \ge 12$, e não $n > 3$. O que importa aqui é só que $c \cdot n + O(1) = o(n^2)$, logo existe um ponto a partir do qual A vence.

Não há contradição: as duas afirmações falam de conjuntos de entradas diferentes. Uma cota de pior caso é um teto sobre o máximo, e nada obriga A a gastar n^3 nesta família em particular.

(iii) O modelo RAM não captura a hierarquia de memória. O terceiro mecanismo é que o modelo de custo está incompleto para a máquina real.

O modelo RAM cobra $O(1)$ por acesso à memória, independentemente de *onde* o dado esteja. Na máquina real, um acesso em cache custa da ordem de 1 ns e um acesso à memória principal custa da ordem de 100 ns — duas ordens de grandeza. Um algoritmo $\Theta(n^3)$ com acesso sequencial (favorável à pré-busca e à localidade de cache) pode executar em menos tempo de parede que um $\Theta(n^2)$ com acesso aleatório, mesmo realizando mais operações — **até o ponto de cruzamento induzido por esse fator**.

> O ganho é um fator *constante*, e fator constante não inverte $\Theta(n^3)$ contra $\Theta(n^2)$ para todo n. Comparando $n^3/100$ com n^2, os dois empatam exatamente em $n = 100$ (10 000 de cada lado) e a partir daí o cúbico volta a perder — em $n = 500$ já perde por cinco vezes.

As duas classificações continuam corretas: o que falha é a tradução de *número de operações* para *tempo*, que o modelo assume proporcional.

(b) Quatro razões

As quatro apoiam-se em eixos independentes: o tamanho das instâncias, a justeza da cota, os fatores constantes, e recursos que não são tempo.

(i) O tamanho das instâncias é limitado pela natureza do problema. Se a aplicação garante $n \le 100$ sempre — por exemplo, um tabuleiro de tamanho fixo, um número de sensores instalados, um grau máximo de polinômio —, o regime assintótico *nunca é atingido*. A garantia "para todo $n \ge n_0$" pode simplesmente não se aplicar a nenhuma entrada real. Note que, com $n \le 100$, o custo absoluto dos dois é pequeno ($n^3 \le 10^6$ operações, milissegundos), de modo que a classe assintótica deixa de ser o critério de decisão e os outros passam a decidir. Não é que o cúbico ganhe: com constantes iguais ele perde em todo $n \ge 2$ — e em $n = 100$ perde por um fator 100. O que muda é que perder por 100× algo que leva microssegundos deixa de importar.

(ii) O é apenas teto — a cota $O(n^3)$ pode ser frouxa. Este eixo é sobre a qualidade da *análise*, não do algoritmo. Todo $f \in O(n^2)$ também está em $O(n^3)$, pois $n^2 \le n^3$ para $n \ge 1$. Logo $O(n^3)$ pode ser apenas a melhor cota que alguém conseguiu *provar*, enquanto o algoritmo é de fato $\Theta(n^2)$ ou melhor. Comparar dois algoritmos pelos seus O não é comparação válida: O limita por cima, e um teto frouxo não é evidência de lentidão.

(iii) Os fatores constantes são muito desiguais. Aqui n não é limitado e as cotas são justas; o que decide são as constantes. Com $T_1(n) = 5000n^2$ e $T_2(n) = 2n^3$:

$$2n^3 < 5000n^2 \iff n < 2500.$$

Ou seja, o algoritmo cúbico faz *menos* operações para todo $1 \le n \le 2499$. Em $n = 2500$ há **empate exato** (31 250 000 000 operações de cada lado), e só a partir de $n = 2501$ o quadrático passa à frente. Se a faixa de interesse do pesquisador estiver abaixo de 2500, a escolha do cúbico é a correta.

(iv) Critérios que não são tempo de execução. O modelo RAM contabiliza apenas operações; a decisão de engenharia envolve outros recursos. O algoritmo $O(n^3)$ pode usar $O(1)$ de espaço enquanto o $O(n^2)$ usa $\Theta(n^2)$ e simplesmente não cabe em memória; pode ser muito mais simples de implementar e manter, portanto menos sujeito a erro; pode ser numericamente mais estável; pode já estar implementado, validado e em produção; ou pode ser trivialmente paralelizável, reduzindo o tempo de parede apesar de mais operações totais.

(c) Não

Aqui $10^8 n$ e $10n^2$ são contagens *exatas* de instruções, e não cotas assintóticas — então a comparação é direta, sem constantes ocultas.

$$10^8 \cdot n < 10 \cdot n^2 \iff 10^8 < 10n \iff n > 10^7 \quad \text{(dividindo por } n \ge 1\text{, que é positivo).}$$

n	$A = 10^8 n$	$B = 10n^2$	preferível
10^6	10^{14}	10^{13}	**B** — dez vezes menos instruções
10^7	10^{15}	10^{15}	empate exato
10^8	10^{16}	10^{17}	**A**

Assintoticamente $T_A = \Theta(n)$ e $T_B = \Theta(n^2)$, logo A vence para n suficientemente grande. Mas "suficientemente grande" aqui significa $n > 10^7$, um limiar altíssimo causado pela constante 10^8. Como a preferência depende do tamanho da instância, **não** se pode afirmar que A é sempre preferível.

Questão 2

> Enunciado
>
> Considere a teoria da complexidade de algoritmos e responda o que se pede. Justifique formalmente suas respostas. Respostas sem justificativas não serão consideradas.
>
> a. Explique por que a frase "O tempo de execução do algoritmo A é no mínimo $O(n^2)$"não faz sentido.
> b. Se um algoritmo A tem tempo de execução $\theta(n^2)$ sobre qualquer entrada de tamanho n, ele pode ter tempo de execução $\Omega(n\log n)$ no pior caso?

(a) A frase mistura piso com teto — e, na leitura mais generosa, é vazia

1. Incoerência de sentido. Por definição, $T(n) = O(n^2)$ significa que existem $c > 0$ e $n_0 > 0$ com $0 \le T(n) \le c \cdot n^2$ para todo $n \ge n_0$. É um **teto**: diz que T não passa de $c \cdot n^2$. Já "no mínimo" pede uma afirmação da forma $T(n) \ge (algo)$, que é exatamente o papel de Ω. Não existe, na definição de O, nenhuma desigualdade do tipo $T(n) \ge \ldots$ para "no mínimo" qualificar.

2. A frase é vazia. Pode-se tentar salvá-la lendo "no mínimo $O(n^2)$" como "$T(n) = O(g(n))$ para alguma g pelo menos tão grande quanto n^2". Mesmo assim não informa nada, porque as classes O são ordenadas por inclusão e crescem *para cima*:

$$O(n^2) \subset O(n^3) \subset O(2^n) \subset \cdots$$

Logo "pelo menos $O(n^2)$" aponta para as classes *maiores*, que são afirmações mais **fracas**. Formalmente: seja T o tempo de um algoritmo qualquer e defina $g(n) = T(n) + n^2$. Então $g(n) \ge n^2$ para todo $n \ge 1$, e vale $T(n) = O(g(n))$ com $c = 1$ e $n_0 = 1$, pois $0 \le T(n) \le T(n) + n^2 = 1 \cdot g(n)$. Como T era arbitrária, *todo* algoritmo satisfaz a frase. Uma afirmação satisfeita por todos os objetos do universo não separa nada.

Redação correta: "o tempo de execução de A é $\Omega(n^2)$".

(b) Sim, pode — e de fato tem. Não há contradição.

Hipótese. Para toda entrada de tamanho n, o custo de A é $\Theta(n^2)$: existem $c_1, c_2 > 0$ e n_0 com $0 \le c_1 n^2 \le T(n) \le c_2 n^2$ para $n \ge n_0$.

Passo 1 — o pior caso herda os limites. Seja $T_{pior}(n)$ o máximo de $T(x)$ sobre as entradas de tamanho n. O máximo de valores todos contidos em $[c_1 n^2, c_2 n^2]$ também pertence a esse intervalo, logo $T_{pior}(n) = \Theta(n^2)$.

Passo 2 — lema: $\lg n \le n$ para $n \ge 1$. Basta $2^n \ge n$, por indução. Base: $2^1 = 2 \ge 1$. Passo: se $2^n \ge n$ com $n \ge 1$, então $2^{n+1} = 2 \cdot 2^n \ge 2n = n + n \ge n + 1$. Como lg é crescente, $n = \lg(2^n) \ge \lg n$.

Passo 3 — testemunhas. Multiplicando o lema por $n > 0$: $n \cdot \lg n \le n^2$. Tome $c = c_1$ e $n_0' = \max(n_0, 2)$ (o 2 evita o caso degenerado $\lg 1 = 0$). Para todo $n \ge n_0'$:

$$T_{pior}(n) \ge c_1 n^2 \ge c_1 \cdot n \lg n, \quad \text{isto é,} \quad T_{pior}(n) = \Omega(n \lg n). \ \blacksquare$$

Por que não é contraditório. $\Omega(n \lg n)$ *não* significa "o algoritmo gasta n lg n". Significa "gasta pelo menos da ordem de n lg n" — e nada impede que gaste muito mais. Como n^2 domina $n \lg n$, um custo $\Theta(n^2)$ satisfaz o piso com folga.

> **A afirmação é verdadeira, mas frouxa.** Ela não é a cota inferior mais firme, que é $\Omega(n^2)$. De fato T_{pior} não é $\Theta(n \lg n)$: se fosse, existiria c_3 com $c_1 n^2 \le c_3 \cdot n \lg n$, ou seja $n / \lg n \le c_3 / c_1$ para todo n grande — falso, pois $n / \lg n$ é ilimitada (em $n = 2^k$ vale $2^k / k$, que cresce sem limite).

Questão 3

> Enunciado
>
> Prove que as sentenças abaixo são verdadeiras ou falsas, usando para isso as definições das notações assintóticas e/ou contra exemplos. Considere que as funções f(n) e g(n) são assintoticamente não negativas.
>
> a. $f(n) \in O(g(n))$ se e somente se $g(n) \in \Omega(f(n))$
> b. Se $f(n) \in O(g(n))$ então $g(n) \in O(f(n))$.
> c. Se $f(n) \in O(u(n))$ e $g(n) \in O(v(n))$ então $f(n) + g(n) \in O(u(n) + v(n))$.
> d. Se $f(n) \in O(g(n))$ e $g(n) \in O(f(n))$ então $f(n) \in \theta(g(n))$.

Em provas de propriedade *geral* as constantes não podem ser apenas declaradas: têm de ser **construídas** a partir das que a hipótese fornece, e nomeadas sem colisão de símbolos.

(a) Verdadeira

As duas notações são a *mesma* desigualdade lida dos dois lados. Sendo um "se e somente se", provam-se as duas direções.

($\Longrightarrow$) Se $f \in O(g)$, existem $c > 0$ e n_0 com $0 \le f(n) \le c \cdot g(n)$ para $n \ge n_0$. Fixe $d = 1/c$ e $m_0 = n_0$ — legítimo porque $c > 0$ estritamente, e nenhuma delas depende de n. Dividindo por $c > 0$: $d \cdot f(n) \le g(n)$. Com $f(n) \ge 0$ e $d > 0$ vem $d \cdot f(n) \ge 0$. Logo $0 \le d \cdot f(n) \le g(n)$, que é $g \in \Omega(f)$.

($\Longleftarrow$) Se $g \in \Omega(f)$, existem $d' > 0$ e m_0' com $0 \le d' \cdot f(n) \le g(n)$. Fixe $c' = 1/d'$ e $n_0' = m_0'$. Dividindo por $d' > 0$: $f(n) \le c' \cdot g(n)$; e de $0 \le d' \cdot f(n)$ segue $f(n) \ge 0$. Logo $f \in O(g)$. $\blacksquare$

(b) Falsa

Contraexemplo: $f(n) = n$ e $g(n) = n^2$, ambas não negativas para $n \ge 1$.

A hipótese vale. Com $c = 1$ e $n_0 = 1$: de $n \ge 1$ vem $n^2 \ge n$, logo $0 \le n \le 1 \cdot n^2$ para $n \ge 1$, isto é, $f \in O(g)$.

A conclusão falha. Suponha por absurdo que existam $c > 0$ e n_0 com $0 \le n^2 \le c \cdot n$ para $n \ge n_0$. Dividindo por $n > 0$: $n \le c$ para todo $n \ge \max(n_0, 1)$. Tome a testemunha $n^* = \max(n_0, 1) + \lfloor c \rfloor + 1$: ela cumpre $n^* \ge \max(n_0, 1)$, então a desigualdade deveria valer, mas $n^* > c$. Contradição. Logo $g \notin O(f)$. $\blacksquare$

> Coerente com (a): $f \in O(g)$ garante apenas $g \in \Omega(f)$. Nada obriga f a ser também teto de g.

(c) Verdadeira

De $f \in O(u)$: existem $c_1 > 0$, n_1 com $0 \le f(n) \le c_1 u(n)$ para $n \ge n_1$. De $g \in O(v)$: existem $c_2 > 0$, n_2 com $0 \le g(n) \le c_2 v(n)$ para $n \ge n_2$.

Construa $C = máx(c_1, c_2)$ e $N = máx(n_1, n_2)$. Para todo $n \ge N$:

$$f(n) + g(n) \le c_1 u(n) + c_2 v(n) \le C \cdot u(n) + C \cdot v(n) = C \cdot (u(n) + v(n)).$$

O passo $c_1 u(n) \le C \cdot u(n)$ exige $u(n) \ge 0$, e isso sai da própria hipótese — o enunciado só fala de f e g: de $0 \le f(n) \le c_1 u(n)$ com $c_1 > 0$ vem $u(n) \ge 0$ para todo $n \ge n_1$; do mesmo modo $v(n) \ge 0$ para $n \ge n_2$, e as duas valem para $n \ge N$. E $f(n) + g(n) \ge 0$ porque ambas são não negativas. Logo $f + g \in O(u + v)$ com testemunhas C e N. ∎

(d) Verdadeira — é o Teorema 3.1 do CLRS

De $f \in O(g)$: existem $b > 0$, n_1 com $0 \le f(n) \le b \cdot g(n)$ para $n \ge n_1$.

De $g \in O(f)$: existem $a > 0$, n_2 com $0 \le g(n) \le a \cdot f(n)$ para $n \ge n_2$; dividindo por $a > 0$, $f(n) \ge (1/a) \cdot g(n)$.

Construa $c_1 = 1/a$, $c_2 = b$ e $n_0 = máx(n_1, n_2)$. Para todo $n \ge n_0$:

$$0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n),$$

que é exatamente a definição de $f \in \Theta(g)$. ∎

> Note que os símbolos foram batizados a e b justamente para não colidirem com c_1 e c_2, que são as constantes *construídas* para o Θ.

Questão 4

> Enunciado
>
> Considere n representando um valor variável e k é um valor constante. Analise os códigos abaixo e encontre a complexidade assintótica em cada caso usando a notação *Big O*. Dê um limite assintótico tão firme quanto possível. Ao realizar a análise, considere o pior caso. Justifique suas respostas. Respostas sem justificativas não serão consideradas.
>
> Os cinco códigos estão transcritos literalmente, cada um no início do seu item abaixo.

Código 1 — O(log n), e o limite é justo: Θ(log n)

> Enunciado — código 1 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  i = 1;
3  While ( i <= n )
4  {
5      soma = soma + 1;
6      i = i * 10;
7  }
```

Após t multiplicações vale $i = 10^t$ (começando em $i = 1$). O corpo executa enquanto

$$10^t \le n \iff t \le \log_{10} n,$$

e como t é inteiro começando em 0, o corpo executa exatamente $\lfloor \log_{10} n \rfloor + 1$ vezes. Somando o custo constante de fora do laço e o teste final que falha (1 unidade), $T(n) = a + b \cdot (\lfloor \log_{10} n \rfloor + 1) + 1$ — na contagem do guia, $a = 2$ e $b = 3$, isto é, $T(n) = 3(\lfloor \log_{10} n \rfloor + 1) + 3$.

Teto. Para $n \ge 10$ vale $\log_{10} n \ge 1$, logo $\lfloor \log_{10} n \rfloor + 1 \le 2 \log_{10} n \le 2 \lg n$. Com $c = a + 4b$ e $n_0 = 10$ obtém-se $T(n) \le c \cdot \lg n$. *Piso.* $\lfloor \log_{10} n \rfloor + 1 > \lg n / \lg 10$, logo $c_1 = b / \lg 10$ e $n_0 = 1$. Portanto $\Theta(\log n)$.

Código 2 — O(log n), e o limite é justo: Θ(log n)

> Enunciado — código 2 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  i = n;
3  While ( i > 0 )
4  {
5      soma = soma + i;
6      i = i / 2; // considere divisao inteira
7  }
```

Com divisão inteira, após t divisões vale $i = \lfloor n / 2^t \rfloor$. O corpo executa enquanto $\lfloor n / 2^t \rfloor \ge 1$, isto é, $2^t \le n$, ou $t \le \lg n$. Logo o corpo executa exatamente $\lfloor \lg n \rfloor + 1$ vezes. Conferência: $n = 1$ dá 1 iteração; $n = 8$ dá 4 ($i = 8, 4, 2, 1$).

Teto. $\lfloor \lg n \rfloor + 1 \le 2 \lg n$ para $n \ge 2$; tome $c = a + 3b$, $n_0 = 2$. *Piso.* $\lfloor \lg n \rfloor + 1 > \lg n$; tome $c_1 = b$, $n_0 = 1$. Portanto $\Theta(\log n)$.

> Os códigos 1 e 2 são logarítmicos por mecanismos *opostos*: no primeiro a variável multiplica por 10 subindo até n; no segundo divide por 2 descendo até 0. As bases são irrelevantes assintoticamente, pois $\log_{10} n = \lg n / \lg 10$.

Código 3 — O(n), e o limite é justo: Θ(n) no pior caso

Enunciado — código 3 (transcrição literal, com a numeração de linhas da lista)

```
1  x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O (1)
2  soma = 0;
3  for (i = 1; i <= n; i = i + 1)
4  {
5      if ( i != x )
6      {
7          soma = soma + i;
8      }
9      else
10     {
11         for (j = 1; j <= i; j = j + 1)
12         {
13             soma = soma + j * x;
14         }
15     }
16 }
```

A condição é `i != x`, então o laço interno está no `else` e só é alcançado quando $i == x$ — o que acontece *exatamente uma vez* em toda a execução, já que x é um único valor de [1..n].

O laço externo custa $a{\cdot}n + a'$. O laço interno roda uma única vez, na iteração $i = x$, com $j = 1..x$, custando $b{\cdot}x + b'$. Logo

$$T(n, x) = a{\cdot}n + b{\cdot}x + d.$$

T é crescente em x, e $x \in [1..n]$, então o **pior caso é x = n**: $T_{pior}(n) = (a + b)n + d$. Testemunhas: $c_1 = a$, $c_2 = a + b + d$, $n_0 = 1$. Portanto $\Theta(n)$.

> **Por que não é Θ(n²).** O somatório $\Sigma_{i=1..n}\, i = n(n+1)/2$, que produziria custo quadrático, só apareceria se o laço interno rodasse para *todo* i. Aqui ele roda para um único i, e o somatório colapsa num único termo $x \le n$.

Código 4 — O(n²), e o limite é justo: Θ(n²) no pior caso

Enunciado — código 4 (transcrição literal, com a numeração de linhas da lista)

```
1  x = escolha (1,n); // retorna um valor inteiro no intervalo [1..n] em O(1)
2  soma = 0;;
3  for (i = 1; i <= n; i = i + 1)
4  {
5      if (i == x)
6      {
7          soma = soma + i;
8      }
9      else
10     {
11         for (j = 1; j <= i; j = j + 1)
12         {
13             soma = soma + j * x;
14         }
15     }
16 }
```

A única diferença em relação ao código 3 é a condição, agora `i == x` — e ela inverte a análise. O laço interno continua no `else`, mas agora o `else` é alcançado para *todo* $i \neq x$, ou seja, em $n - 1$ das n iterações.

Número total de execuções do corpo interno, em forma fechada:

$$\Sigma_{i=1,\, i\neq x}^{n} i = n(n+1)/2 - x.$$

Logo $T(n, x) = A{\cdot}n(n+1)/2 - B{\cdot}x + C{\cdot}n + D$, que é *decrescente* em x: cada unidade a mais em x retira exatamente uma passagem pelo laço interno. O **pior caso é x = 1**. Com $n(n+1)/2 \le n^2$ para $n \ge 1$ obtém-se o teto, e $n(n+1)/2 \ge n^2/2$ o piso. Portanto $\Theta(n^2)$.

Testemunhas. Na contagem do guia (1 unidade por atribuição, teste e incremento), o pior caso custa exatamente $W(n) = T(n, 1) = (3/2)n^2 + (13/2)n$ — conferido por força bruta para $n = 1..149$, e $x = 1$ é o pior em todo n testado. Para $n \ge 1$, $(3/2)n^2 \le W(n) \le (3/2)n^2 + (13/2)n^2 = 8n^2$: $c_1 = 3/2$, $c_2 = 8$, $n_0 = 1$ (em $n = 1$: $1{,}5 \le 8 \le 8$).

> **Os piores casos são opostos:** $x = n$ no código 3 e $x = 1$ no código 4. Responder os dois igual erra um deles.

Código 5 — O(n log n), e o limite é justo: Θ(n log n)

Enunciado — código 5 (transcrição literal, com a numeração de linhas da lista)

```
1  soma = 0;
2  for (i = 1; i <= n; i = i + 1)
3  {
4      for (j = 1; j <= i; j = 3*j)
5      {
6          for (k = 1; k <= 25; k = k + 1)
7          {
8              soma = soma + 1;
9          }
10     }
11 }
```

O laço mais interno é O(1): ele vai de 1 a **25**, um limite constante que não depende de n — 25 iterações sempre.

O laço do meio é logarítmico: j multiplica por 3 a cada passo, valendo 3^t após t passos. Ele executa enquanto $3^t \le i$, ou seja $\lfloor \log_3 i \rfloor + 1$ vezes.

Somando sobre o laço externo:

$$T(n) = \Theta\left(\sum_{i=1..n} (\lfloor \log_3 i \rfloor + 1) \right) = \Theta(n \log n).$$

Teto. $\lfloor \log_3 i \rfloor + 1 \le \log_3 n + 1 \le 2 \log_3 n$ para $i \le n$ e $n \ge 3$; somando sobre n valores, $T(n) = O(n \log n)$. *Piso.* Para os $n/2$ valores com $i > n/2$ vale $\lfloor \log_3 i \rfloor + 1 > \log_3(n/2)$, logo $T(n) = \Omega(n \log n)$. A base 3 só altera a constante, pois $\log_3 n = \lg n / \lg 3$.

Testemunhas. Na contagem do guia, $T(n) = 4n + 3 + 79 \cdot S(n)$, com $S(n) = \Sigma_{i=1..n} (\lfloor \log_3 i \rfloor + 1)$: cada volta do laço do meio custa 79 (seu teste, $k = 1$, 25 × 3 do laço interno, o teste final dele e $j = 3 \cdot j$) — conferido por força bruta. *Teto:* para $n \ge 3$, $S(n) \le 2n \cdot \log_3 n$ e $4n + 3 \le 7n \le 7n \cdot \log_3 n$, logo $T(n) \le 165 \cdot n \cdot \log_3 n = (165/\lg 3) \cdot n \lg n$. *Piso:* os $\lceil n/2 \rceil \ge n/2$ índices $i > n/2$ têm $\lfloor \log_3 i \rfloor + 1 > \log_3(n/2) \ge \frac{1}{2} \cdot \log_3 n$ (a última desigualdade vale para $n \ge 4$), logo $T(n) \ge 79 \cdot (n/4) \cdot \log_3 n = (79/(4 \lg 3)) \cdot n \lg n$. Testemunhas: $c_1 = 79/(4 \lg 3) \approx 12{,}46$, $c_2 = 165/\lg 3 \approx 104{,}10$, $n_0 = 4$ — em $n = 4$: $99{,}7 \le T(4) = 493 \le 832{,}8$; conferidas por força bruta de $n = 4$ a $2 \cdot 10^5$.

Código	Iterações do laço decisivo	Resposta (Big O, mais firme)	Por quê
1	$\lfloor \log_{10} n \rfloor + 1$	**O(log n)**	i multiplica por 10 subindo até n
2	$\lfloor \lg n \rfloor + 1$	**O(log n)**	i divide por 2 descendo até 0
3	interno roda 1 vez (i = x)	**O(n)**	`if (i != x)` põe o laço no else; pior caso x = n
4	interno roda n − 1 vezes	**O(n²)**	`if (i == x)` inverte tudo; pior caso x = 1
5	$\lfloor \log_3 i \rfloor + 1$ por i	**O(n log n)**	laço de 25 é constante; o do meio é log

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Análise de Algoritmos:

Não há um conjunto de regras para analisar um algoritmo. No entanto, conforme [AHO et al., 1983] podemos seguir o seguinte guia, considerando o modelo computacional *Random Access Machine* (RAM):

1. O custo de execução de uma instrução de atribuição, de leitura ou de escrita é O(1);
2. O custo de execução de uma sequência de instruções é determinado pela instrução de maior custo na sequência de instruções;
3. O custo de execução de um comando de decisão/seleção é composto pelo custo de execução das instruções no corpo do comando de decisão/seleção mais o custo para avaliar a condição. O custo para avaliar a condição é O(1), e
4. O custo de execução de um comando de repetição (laço/loop) é a soma do custo de execução do corpo do comando de repetição adicionado ao custo de avaliar a condição de término do comando de execução, multiplicado pelo número de iterações do comando de repetição. O custo para avaliar a condição de término é O(1).

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n + 1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$

onde c é uma constante arbitrária inteira ≥ 2.

Referências

- [AHO et al., 1983] AHO, A. V., HOPCORFT, J. E., and ULLMAN, J. D. (1983). Data Structures and Algorithms. Addison-Wesley.
- [BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). Fundamentals of Algorithms. Prentice-Hall.

Sobre esta resolução. As classes assintóticas da Questão 4 foram conferidas por medição: os cinco códigos foram transcritos em Python e todas as operações primitivas contadas por força bruta, com varredura exaustiva de x para confirmar qual valor realiza o pior caso nos códigos 3 e 4. Os pontos de cruzamento citados na Questão 1 foram verificados numericamente, incluindo os empates exatos em $n = 2500$ e $n = 10^7$. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final. Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.

Projeto e Análise de Algoritmos — Lista de Exercícios III

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Lista revisada pelo professor** (arquivo de 09/09/2026; o cabeçalho da lista manteve a data original). Entraram duas questões novas — a Questão 3 (moeda falsa) e a Questão 5 (passo indutivo sem base) — e a antiga Questão 3 passou a ser a Questão 4, sem mudança de conteúdo. A versão anterior está arquivada em `listas-de-exercicios/versoes-anteriores/`.

> **Estrutura das provas por indução.** Cada item da Questão 4 segue o mesmo esqueleto: enunciar *P(n)*; verificar a **base** no valor inicial que o enunciado exige, com os dois lados avaliados; escrever a **hipótese indutiva**; e provar o **passo**, marcando o ponto exato em que a hipótese é aplicada. Nas recorrências, o passo parte da *definição recursiva* e chega à fórmula fechada — partir da fórmula fechada querendo chegar nela mesma seria circular. A Questão 3 aplica o mesmo esqueleto a um *procedimento* (a hipótese é aplicada a um subconjunto das moedas), e a Questão 5 mostra o que sobra quando só o passo vale e a base falha.

> Cabeçalho e instruções da lista (transcrição literal)
>
> Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios III · Formiga,MG, 23 de agosto de 2026
>
> **Instruções:**
>
> 1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
> 2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
> 3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1

> Enunciado
>
> Alice e Bob estão discutindo sobre o desempenho de seus algoritmos de ordenação. Alice afirma que o seu algoritmo de ordenação executando em tempo $\theta(n * \lg n)$ é melhor que algoritmo de Bob que executa em tempo $\theta(n^2)$. Para decidir qual é melhor algoritmo, implementaram e executaram ambos os algoritmos no mesmo ambiente (mesma arquitetura, mesmo sistema operacional, mesma linguagem de programação e mesmo compilador) em uma série de dados. Para espanto de Alice, descobriram que quando $n < 100$ o algoritmo $\theta(n^2)$ é mais rápido e apenas quando $n \geq 100$ é que o algoritmo $\theta(n * \lg n)$ é melhor. Explique como essa situação pode ocorrer.

Não há contradição alguma. Alice está certa *assintoticamente*, e a medição também está certa *na faixa testada*. As duas coisas convivem porque Θ descarta exatamente aquilo que decide o desempenho para n pequeno.

1. O que Θ afirma — e o que não afirma

Por definição, $T(n) = \Theta(g(n))$ significa que existem $c_1, c_2 > 0$ e $n_0 > 0$ com $0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n)$ para todo $n \geq n_0$. Disso decorrem três coisas:

(i) as constantes c_1, c_2 são arbitrárias — nada limita seu tamanho; (ii) termos de menor ordem desaparecem, pois $5n \lg n + 500n + 20000$ também é $\Theta(n \lg n)$; e (iii) a garantia só vale a partir de um n_0, que pode ser grande.

Ou seja, "Θ(n lg n) é melhor que Θ(n²)" é afirmação sobre o *limite*, não sobre cada valor de n.

2. Um exemplo concreto que reproduz o cenário

Tome

$$T_A(n) = 15{\cdot}n{\cdot}\lg n + 20 \ \text{(Alice)} \quad \text{e} \quad T_B(n) = n^2 \ \text{(Bob)}.$$

T_A é $\Theta(n \lg n)$ e T_B é $\Theta(n^2)$, exatamente como Alice e Bob anunciaram. A única diferença estrutural é a constante 15 e o termo constante 20 — precisamente o que Θ apaga. O 20 tem leitura natural: é o custo fixo de preparação, como alocar o vetor auxiliar de um mergesort.

n	$T_A = 15n \lg n + 20$	$T_B = n^2$	mais rápido
1	20	1	Bob
10	518	100	Bob
50	4 253	2 500	Bob
99	9 865	9 801	Bob
100	9 986	10 000	**Alice**
200	22 952	40 000	**Alice**
1000	149 507	1 000 000	**Alice**

> O termo constante 20 não é decoração: sem ele, com $T_A(n) = 15n \lg n$, teríamos $T_A(1) = 15{\cdot}1{\cdot}\lg 1 = 0$, porque $\lg 1 = 0$. Alice "venceria" em $n = 1$ com tempo zero — um artefato degenerado que contradiz o enunciado, já que ele afirma que Bob é mais rápido para *todo* $n < 100$. Com o termo constante, Bob vence em todos os inteiros de 1 a 99, sem furos.

3. O ponto de cruzamento, algebricamente

Ignorando o termo constante para achar a ordem de grandeza do cruzamento, resolve-se

$$15{\cdot}n{\cdot}\lg n = n^2 \iff 15{\cdot}\lg n = n \iff n \,/\, \lg n = 15,$$

onde a divisão por $\lg n$ exige $\lg n > 0$, isto é, $n \geq 2$ — restrição que precisa ser declarada.

Seja $\varphi(n) = n \,/\, \lg n$. Derivando, $\varphi'(n) = \ln 2{\cdot}(\ln n - 1)/(\ln n)^2$, que é positiva quando $\ln n > 1$, ou seja $n > e \approx 2{,}718$. Logo φ é estritamente crescente **a partir de n = 3**, e só nessa região a equação $\varphi(n) = 15$ tem solução única.

$$\varphi(99) = 14{,}93 < 15 < 15{,}05 = \varphi(100).$$

> Esse *bracket* localiza a raiz da equação **simplificada** $15n{\cdot}\lg n = n^2$, que é $n \approx 99{,}563$. A raiz da equação **completa** $15n{\cdot}\lg n + 20 = n^2$ é outra: $n \approx 99{,}819$. O bracket correspondente é $T_A(99) = 9\,864{,}6 > 9\,801 = T_B(99)$ e $T_A(100) = 9\,985{,}8 < 10\,000 = T_B(100)$.

Resolvendo numericamente $T_A(n) = T_B(n)$, o cruzamento cai em $n \approx 99{,}82$ — entre 99 e 100. Portanto o **primeiro inteiro** em que Alice vence é $n = 100$, exatamente o limiar observado.

Por que não volta a inverter. Seja $h(n) = T_B(n) - T_A(n) = n^2 - 15n{\cdot}\lg n - 20$. Sua derivada é $h'(n) = 2n - 15{\cdot}\lg n - 15/\ln 2$, que é negativa só para $n \leq 53$ e positiva a partir de $n = 54$. Logo h é crescente em todo $n \geq 54$ e, como $h(99) < 0 < h(100)$, ela troca de sinal **uma única vez**. Assim Bob vence exatamente em $1 \leq n \leq 99$, sem furos, e Alice de $n = 100$ em diante, sem reversão.

> **Cuidado com dois limiares diferentes.** O número 100 é o *ponto de cruzamento entre* T_A *e* T_B, causado pela constante 15. Ele **não é o n_0 mínimo** de nenhuma das duas declarações Θ — nelas basta $n_0 = 2$ — e nada na definição de Θ o determina. (Rigorosamente, n_0 não é único: qualquer valor maior também serve, inclusive 100. O que se quer dizer é que o cruzamento não é o menor n_0 possível, e que ele vem da constante 15, não da definição.) Chamar o cruzamento de "n_0 do par" confunde duas constantes que não têm relação: uma liga um algoritmo à sua própria classe, a outra compara dois algoritmos.

4. Outros fatores que contribuem

Termos de menor ordem. O custo real de Alice poderia ser $5n \lg n + 500n + 3000$, que continua sendo $\Theta(n \lg n)$. Para n pequeno o $500n$ e a constante dominam; para n grande somem diante de $5n \lg n$.

Fatores que o modelo RAM não enxerga. O modelo cobra custo unitário por operação, mas na máquina real o algoritmo $\Theta(n^2)$ pode ter localidade de referência muito melhor (varre vetores contíguos, que cabem em cache), pode ser iterativo — sem o custo de empilhar chamadas recursivas — e pode operar *in place*, sem alocar memória auxiliar nem pagar cópias. Todos esses custos entram na constante multiplicativa do algoritmo assintoticamente melhor.

É exatamente o caso do *insertion sort* ($\Theta(n^2)$ no pior caso) contra o *mergesort* ($\Theta(n \lg n)$): para entradas pequenas o insertion sort é mais rápido na prática, e por isso implementações reais de mergesort e quicksort trocam para insertion sort quando o subproblema fica abaixo de um limiar de algumas dezenas de elementos.

5. Conclusão

A afirmação de Alice é sobre o comportamento quando $n \to \infty$, e é verdadeira: a partir de $n = 100$ seu algoritmo é mais rápido, e a vantagem cresce sem limite, pois $T_B(n)/T_A(n) \approx n/(15 \lg n) \to \infty$. A observação empírica apenas revela onde está o ponto de cruzamento desse par de implementações — e quem o determina são as constantes, que a notação Θ deliberadamente ignora.

Questão 2

> Enunciado
>
> Considere
>
> $$f(n) = \{\ n \quad \text{se } n \text{ é ímpar};\quad n^2 \quad \text{se } n \text{ é par}\ \}$$
>
> Apresente um limite assintótico para a função f tão firme (restrito/justo) quanto possível, usando a notação Big O.

Resposta: $f(n) = O(n^2)$, com testemunhas $c = 1$ e $n_0 = 1$.

n	1	2	3	4	5	6	7	8	9	10
f(n)	1	4	3	16	5	36	7	64	9	100

(1) Limite superior: $f(n) = O(n^2)$

Testemunhas $c = 1$ e $n_0 = 1$, fixadas antes e independentes de n. Seja $n \geq 1$:

Caso n par: $f(n) = n^2 \leq 1 \cdot n^2$, com igualdade.
Caso n ímpar: $f(n) = n$. De $1 \leq n$, multiplicando pelo positivo n, vem $n \leq n^2$. Logo $f(n) \leq 1 \cdot n^2$.

Em ambos $f(n) \geq 0$. Portanto $0 \leq f(n) \leq 1 \cdot n^2$ para todo $n \geq 1$. ∎

(2) Esse teto não pode ser trocado por um menor: f(n) não é O(n)

Nos índices *pares* a função vale exatamente n^2, então nenhuma função g com $g(n) = o(n^2)$ pode servir de teto.

Suponha por absurdo $f(n) = O(n)$: existem $c > 0$ e n_0 com $f(n) \leq c \cdot n$ para $n \geq n_0$. Escolha um n **par** com $n > \max(n_0, c)$ — existe, pois há infinitos pares. Aí $f(n) = n^2$, e a desigualdade daria $n^2 \leq c \cdot n$; dividindo por $n > 0$, $n \leq c$, contradizendo $n > c$. ∎

O mesmo argumento vale para qualquer g com $g(n) = o(n^2)$. Mais que isso: entre as funções não decrescentes — como todas as da hierarquia do anexo —, n^2 é o menor teto a menos de constante. Se g é não decrescente e $f(n) \leq c \cdot g(n)$ para $n \geq n_0$, então nos n pares $g(n) \geq n^2/c$, e nos ímpares com $n - 1 \geq n_0$ vale $g(n) \geq g(n-1) \geq (n-1)^2/c \geq n^2/(4c)$; logo $g = \Omega(n^2)$. (Sem a restrição, a própria f seria um teto mais firme: $f = O(f)$, mas n^2 não é $O(f)$.) Nesse sentido, n^2 é o teto justo.

(3) Ponto central: f(n) *não* é Θ(n²)

Θ exige teto **e** piso da mesma ordem. O teto vale, mas o piso não — e é isso que a questão testa.

Suponha por absurdo $f(n) = \Omega(n^2)$: existem $c > 0$ e n_0 com $c \cdot n^2 \leq f(n)$ para $n \geq n_0$. Escolha um n **ímpar** com $n > \max(n_0, 1/c)$. Aí $f(n) = n$, e a desigualdade daria $c \cdot n^2 \leq n$; dividindo por $n > 0$, $c \cdot n \leq 1$, isto é $n \leq 1/c$ — contradizendo $n > 1/c$. ∎

> Note que a constante c não salva a situação por menor que seja: ela apenas *afasta* o primeiro ponto de falha, sem eliminá-lo. Para $c = 1$ a desigualdade falha já em $n = 3$; para $c = 0{,}01$ falha a partir de $n = 101$; para $c = 0{,}0001$, a partir de $n = 10\,001$. Sempre restam infinitos ímpares onde falha.

Como $f(n) \geq n$ para todo $n \geq 1$, vale $f(n) = \Omega(n)$. Assim $n \leq f(n) \leq n^2$: a função fica espremida entre as duas ordens **sem ser Θ de nenhuma das duas**, porque oscila entre elas conforme a paridade.

Questão 3

> Enunciado
>
> Têm-se 2^n moedas de ouro, sendo uma delas falsa, com peso menor do que as demais. Dispõe-se de uma balança de dois pratos, sem nenhum peso. Vamos mostrar, por indução sobre n, que é possível achar a moeda falsa com n pesagens.

Bissecção: divide-se em duas metades, pesa-se uma contra a outra, e a metade mais leve é um problema do mesmo tipo com a metade das moedas. A indução formaliza isso — e o ponto delicado é enunciar $P(n)$ para *qualquer* conjunto de moedas, porque a hipótese será aplicada a um subconjunto.

1. Suposições e a proposição

Suposições declaradas. (1) Todas as moedas verdadeiras têm o mesmo peso p; a falsa tem peso $q < p$. ("As demais" são as verdadeiras; o enunciado diz que a falsa pesa menos que elas.) (2) Uma pesagem põe um conjunto de moedas em cada prato e observa um de três resultados: prato esquerdo mais pesado, prato direito mais pesado, ou equilíbrio. (3) "Com n pesagens" é lido como "com no máximo n pesagens"; o procedimento abaixo usa exatamente n. (4) n é inteiro, $n \geq 1$ (se $0 \in \mathbb{N}$, o caso $n = 0$ também vale e é tratado à parte).

Proposição. $P(n)$: para todo conjunto M de 2^n moedas no qual exatamente uma é falsa, de peso q, e as demais têm todas o mesmo peso $p > q$, é possível identificar a moeda falsa de M usando n pesagens.

> A quantificação "para todo conjunto M" é essencial: no passo, a hipótese de indução será aplicada a um **subconjunto** das moedas originais, não ao conjunto original.

2. A indução

Base (n = 1). M tem $2^1 = 2$ moedas. Põe-se uma em cada prato (1 pesagem). Um prato tem peso q e o outro, p; como $q < p$, a balança não equilibra, e a moeda do prato mais leve é a falsa. *P(1)* vale, com 1 pesagem. (Caso $n = 0$, se $0 \in \mathbb{N}$: M tem $2^0 = 1$ moeda, que é a falsa; 0 pesagens. *P(0)* vale.)

Hipótese de indução. Seja $n \geq 1$ fixado e suponha *P(n)*: em qualquer conjunto de 2^n moedas com exatamente uma falsa, mais leve, e as demais de mesmo peso, acha-se a falsa com n pesagens.

Passo. Seja M um conjunto qualquer de 2^{n+1} moedas com exatamente uma falsa (peso q) e as demais de peso $p > q$. Como $2^{n+1} = 2 \cdot 2^n$, divide-se M em dois conjuntos disjuntos M_1 e M_2 com 2^n moedas cada, e põe-se um em cada prato (1 pesagem). A falsa está em exatamente um dos dois; chame-o de M_i e o outro de M_j. Então

$$\text{peso}(M_i) = (2^n - 1)\cdot p + q \quad \text{e} \quad \text{peso}(M_j) = 2^n \cdot p, \quad \text{logo} \quad \text{peso}(M_j) - \text{peso}(M_i) = p - q > 0.$$

A balança não equilibra e o prato mais leve é o de M_i: a pesagem revela qual grupo contém a falsa. M_i é um conjunto de 2^n moedas com exatamente uma falsa, de peso q, e as demais de peso $p > q$ — exatamente a situação de *P(n)*. Pela hipótese de indução, *aplicada ao conjunto* M_i, a falsa de M_i (que é a falsa de M) é identificada com n pesagens. Total: $1 + n = n + 1$ pesagens. Logo *P(n+1)*.

Conclusão. Pelo princípio da indução (base *P(1)*, passo $P(n) \to P(n+1)$ para todo $n \geq 1$), *P(n)* vale para todo $n \geq 1$: com 2^n moedas, acha-se a falsa com n pesagens. ∎ (Se $0 \in \mathbb{N}$, a base *P(0)* com o mesmo passo, aplicado também em $n = 0$ — divide 2 moedas em grupos de 1, pesa, e *P(0)* identifica a falsa no grupo mais leve —, dá o resultado para todo $n \geq 0$.)

> **Fencepost.** Passo com $n = 1$ (o menor em que é aplicado, na leitura $n \geq 1$): 4 moedas → dois grupos de 2 → 1 pesagem → grupo mais leve de 2 moedas → base: 1 pesagem. Total $2 = n + 1$. O número de pesagens satisfaz $W(1) = 1$, $W(n+1) = W(n) + 1$, logo $W(n) = n = lg(2^n)$, o logaritmo do número de moedas.

3. Observações

Neste procedimento o resultado "equilíbrio" nunca ocorre: os dois pratos têm o mesmo número de moedas e só um contém a falsa.

n pesagens bastam, mas não são o mínimo para n ≥ 3. Cada pesagem tem 3 resultados possíveis, e há 2^n possibilidades para a posição da falsa; logo qualquer estratégia precisa de pelo menos $\lceil \log_3 2^n \rceil$ pesagens, e dividir em três grupos aproveita o resultado "equilíbrio". Exemplo: 8 moedas ($n = 3$) em 2 pesagens — pesa-se 3 contra 3; se equilibrar, a falsa está entre as 2 restantes, e pesa-se uma contra a outra; se não, está entre as 3 do prato mais leve, e pesa-se uma contra outra dessas três: o prato mais leve indica a falsa e, no equilíbrio, a falsa é a terceira. Para todo $n \geq 3$ isso dá uma estratégia com menos de n pesagens: faz-se a bissecção por $n - 3$ pesagens, até restar um grupo de $2^3 = 8$ moedas que contém a falsa, e resolve-se esse grupo em 2 pesagens como acima; total $(n - 3) + 2 = n - 1 < n$. O enunciado pede apenas que n pesagens sejam suficientes, que é o que a indução mostra.

Questão 4

Enunciado

Prove as assertivas abaixo usando o método da indução matemática.

a. $P(n) : n! > 2^n$, $\forall n \geq 4$

b. $P(n) : 2 + 4 + 6 + \ldots + 2 * n = n^2 + n$, $\forall n \geq 1$

c. $P(n) : 1 + 2 + 3 + \ldots + n = n * (n + 1)/2$, $\forall n \geq 1$

d. $T(n) = \{ 1 \text{ se } n = 1; \quad T(n - 1) + n \text{ se } n \geq 2 \}$
   A fórmula fechada para essa recorrência é $T(n) = n * (n + 1)/2$

e. $T(n) = \{ 1 \text{ se } n = 1; \quad 2 * T(n - 1) + 1 \text{ se } n \geq 2 \}$
   A fórmula fechada para essa recorrência é $T(n) = 2^n - 1$

(a) n! > 2^n, para todo n ≥ 4

P(n): $n! > 2^n$.

Base (n = 4). $4! = 24$ e $2^4 = 16$. Como $24 > 16$, *P(4)* é verdadeira.

Hipótese indutiva. Suponha $n! > 2^n$ para algum $n \geq 4$.

Passo. Queremos $(n+1)! > 2^{n+1}$. Da definição de fatorial,

$$(n+1)! = (n+1)\cdot n! > (n+1)\cdot 2^n \quad \leftarrow \text{aqui a hipótese é aplicada}$$

Como $n \geq 4$, tem-se $n + 1 \geq 5 > 2$, logo

$$(n+1)\cdot 2^n > 2\cdot 2^n = 2^{n+1}.$$

Encadeando, $(n+1)! > 2^{n+1}$, isto é, *P(n+1)*. ∎

> **A base em 4 é justa, não folgada.** A afirmação é *falsa* antes disso: $1! = 1 > 2$ é falso; $2! = 2 > 4$ é falso; $3! = 6 > 8$ é falso. Só em $n = 4$ ela passa a valer. Começar a base em $n = 1$ não seria apenas erro de forma — seria "provar" algo falso. E note que a restrição $n \geq 4$ é usada exatamente uma vez no passo: para garantir $n + 1 > 2$.

(b) $2 + 4 + 6 + \ldots + 2n = n^2 + n$, para todo $n \geq 1$

$P(n)$: $\Sigma_{i=1..n}\, 2i = n^2 + n$.

> **Base ($n = 1$).** Esquerda: $2 \cdot 1 = 2$. Direita: $1^2 + 1 = 2$. Iguais.
>
> **Hipótese indutiva.** Suponha $2 + 4 + \ldots + 2n = n^2 + n$ para algum $n \geq 1$.
>
> **Passo.** A soma até $2(n+1)$ é
>
> $$(2 + 4 + \ldots + 2n) + 2(n+1) = (n^2 + n) + 2n + 2 \quad \leftarrow \text{hipótese aplicada}$$
> $$= n^2 + 3n + 2.$$
>
> E a fórmula avaliada em $n+1$ dá
>
> $$(n+1)^2 + (n+1) = n^2 + 2n + 1 + n + 1 = n^2 + 3n + 2.$$
>
> As duas expressões coincidem, logo $P(n+1)$. ∎

(c) $1 + 2 + 3 + \ldots + n = n(n+1)/2$, para todo $n \geq 1$

$P(n)$: $\Sigma_{i=1..n}\, i = n(n+1)/2$.

> **Base ($n = 1$).** Esquerda: 1. Direita: $1 \cdot 2/2 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $1 + 2 + \ldots + n = n(n+1)/2$ para algum $n \geq 1$.
>
> **Passo.**
>
> $$(1 + 2 + \ldots + n) + (n+1) = n(n+1)/2 + (n+1) \quad \leftarrow \text{hipótese aplicada}$$
>
> Fatorando $(n+1)$:
>
> $$= (n+1)\cdot(n/2 + 1) = (n+1)\cdot(n+2)/2,$$
>
> que é a fórmula avaliada em $n+1$. ∎

(d) $T(n) = T(n-1) + n$, $T(1) = 1 \implies T(n) = n(n+1)/2$

$P(n)$: $T(n) = n(n+1)/2$.

> **Base ($n = 1$).** Pela definição, $T(1) = 1$. Pela fórmula, $1 \cdot (1+1)/2 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $T(n-1) = (n-1)\cdot n/2$ para algum $n \geq 2$.
>
> **Passo.** Como $n \geq 2$, vale o caso recursivo da definição — e é *dele* que se parte:
>
> $$T(n) = T(n-1) + n = (n-1)\cdot n/2 + n \quad \leftarrow \text{hipótese aplicada}$$
> $$= (n^2 - n)/2 + 2n/2 = (n^2 + n)/2 = n(n+1)/2.$$ ∎

> Primeiros valores pela recorrência: 1, 3, 6, 10, 15, 21, 28, 36 — que coincidem com $n(n+1)/2$.

(e) $T(n) = 2\cdot T(n-1) + 1$, $T(1) = 1 \implies T(n) = 2^n - 1$

$P(n)$: $T(n) = 2^n - 1$.

> **Base ($n = 1$).** Pela definição, $T(1) = 1$. Pela fórmula, $2^1 - 1 = 1$. Iguais.
>
> **Hipótese indutiva.** Suponha $T(n-1) = 2^{n-1} - 1$ para algum $n \geq 2$.
>
> **Passo.** Como $n \geq 2$, parte-se do caso recursivo da definição:
>
> $$T(n) = 2\cdot T(n-1) + 1 = 2\cdot(2^{n-1} - 1) + 1 \quad \leftarrow \text{hipótese aplicada}$$
> $$= 2^n - 2 + 1 = 2^n - 1.$$ ∎

> O fecho da conta é o $-2 + 1 = -1$: é ali que a fórmula se encaixa, e é o passo que não se deve pular. Primeiros valores pela recorrência: 1, 3, 7, 15, 31, 63, 127, 255 — que coincidem com $2^n - 1$.

> **Sobre a estrutura dos itens (d) e (e).** Nos dois, o passo começa aplicando a *definição recursiva* e termina na fórmula fechada. O caminho inverso — escrever a fórmula em n e manipulá-la até chegar em algo verdadeiro — é circular: usaria a tese para provar a tese. A distinção é sutil na leitura e decisiva na correção.

Questão 5

> Enunciado
>
> Seja a sentença aberta em $\mathbb{N}$:
>
> $$P(n): \quad 1 + 2 + \cdots + n = n(n+1)/2 + 1,$$
>
> mostre que:
> (i) Qualquer que seja $n \in \mathbb{N}$, se $P(n)$ é verdadeira, então $P(n+1)$ é verdadeira.
> (ii) $P(n)$ não é verdadeira para nenhum valor de $n \in \mathbb{N}$.

O passo vale, a base não — e sem base a indução não conclui nada. Seja $S(n) = 1 + 2 + \cdots + n = \Sigma_{k=1..n}\, k$. O enunciado não diz se $0 \in \mathbb{N}$; a solução cobre as duas convenções. Para $n = 0$, $S(0)$ é a soma vazia, igual a 0. Assim, $P(n)$ é a afirmação $S(n) = n(n+1)/2 + 1$.

(i) Para todo n ∈ ℕ, P(n) → P(n+1)

Seja $n \in \mathbb{N}$ arbitrário e suponha $P(n)$, isto é, $S(n) = n(n+1)/2 + 1$. Como $S(n+1) = S(n) + (n+1)$ (separação do último termo; vale também para $n = 0$, pois $S(1) = 0 + 1$),

$$\begin{aligned} S(n+1) &= S(n) + (n+1) = n(n+1)/2 + 1 + (n+1) \quad \leftarrow \text{hipótese P(n)} \\ &= [n(n+1) + 2(n+1)]/2 + 1 = (n+1)(n+2)/2 + 1. \end{aligned}$$

Como $(n+1)((n+1)+1)/2 = (n+1)(n+2)/2$, isto é exatamente $P(n+1)$. Como n foi arbitrário, a implicação vale para todo $n \in \mathbb{N}$. ∎

> A prova de (i) é direta e usa apenas a hipótese $P(n)$ e a definição de S; não usa (ii) nem a fórmula verdadeira de $S(n)$.

(ii) P(n) é falsa para todo n ∈ ℕ

Lema. Para todo $n \in \mathbb{N}$, $S(n) = n(n+1)/2$.

> **Base.** $S(1) = 1 = 1 \cdot 2/2$; e, se $0 \in \mathbb{N}$, $S(0) = 0 = 0 \cdot 1/2$.
>
> **Hipótese.** Para um $n \in \mathbb{N}$ fixado, $S(n) = n(n+1)/2$.
>
> **Passo.** $S(n+1) = S(n) + (n+1) = n(n+1)/2 + (n+1) = (n+1)(n+2)/2$ ← hipótese aplicada. Logo o Lema vale para todo $n \in \mathbb{N}$. ∎

(É a mesma identidade da Questão 4(c) desta lista, aqui estendida a $n = 0$.)

Conclusão de (ii). Para qualquer $n \in \mathbb{N}$, pelo Lema, o lado esquerdo de $P(n)$ vale $S(n) = n(n+1)/2$, e o lado direito vale $n(n+1)/2 + 1 = S(n) + 1$. Como $S(n) \neq S(n) + 1$, $P(n)$ é falsa. Como n foi arbitrário, $P(n)$ não é verdadeira para nenhum $n \in \mathbb{N}$. ∎

> **Por que (ii) não se prova só olhando a base.** Verificar que $P(1)$ é falsa ($1 \neq 2$) mostra que a base falha, mas *não* mostra que $P(n)$ é falsa para todo n: uma propriedade pode falhar nos primeiros valores e valer depois (por exemplo, $n! > 2^n$ é falsa em $n = 1, 2, 3$ e verdadeira para todo $n \geq 4$ — Questão 4(a) desta lista). Por isso (ii) precisa de um argumento que valha para **todo** n, e não só da base. Aqui esse argumento é o Lema, que mostra que os dois lados diferem por exatamente 1 em todo n. Não é o único caminho: a conta de (i) é reversível — como $S(n+1) = S(n) + (n+1)$ e $(n+1)(n+2)/2 + 1 = [n(n+1)/2 + 1] + (n+1)$, vale $S(n+1) = (n+1)(n+2)/2 + 1$ se e somente se $S(n) = n(n+1)/2 + 1$ —, logo $P(n+1) \to P(n)$, isto é, $\neg P(n) \to \neg P(n+1)$; uma indução sobre $\neg P$, com base $\neg P(1)$ (ou $\neg P(0)$), também prova (ii). Nos dois caminhos há base **e** passo: a falha da base, sozinha, não basta.

O que a questão ensina

(i) e (ii) juntos mostram que o passo indutivo sozinho não prova nada: aqui $P(n) \to P(n+1)$ vale para todo n e, mesmo assim, $P(n)$ é falsa para todo n. O que falta é um ponto de partida: $P(1)$ (ou $P(0)$) é falsa e, por (ii), nenhum $P(m)$ é verdadeiro, de modo que a cadeia de implicações nunca é disparada, qualquer que seja o valor inicial escolhido. A questão mostra que a base é indispensável: um passo válido, sozinho, não prova nada.

Informações que podem ser úteis

> Anexo da lista (transcrição literal)
>
> **Somatórios e Logaritmos:**
>
> - $\Sigma_{i=1}^{n}\,(i) = n(n+1)/2$
> - $\Sigma_{i=1}^{n}\,(a^i) = a(a^n - 1)/(a - 1)$
> - $\log\,(a * b) = \log a + \log b$
> - $\log\,(a/b) = \log a - \log b$
> - $\log_b a = \log_c a / \log_c b$
> - $\log_b a^c = c * \log_b a$
> - $\lg a = \log_2 a$
>
> **Hierarquia de funções:**
>
> A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:
>
> $$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$
>
> onde c é uma constante arbitrária inteira ≥ 2.

Atualização de 09/09/2026. As Questões 3 e 5 são novas; a Questão 4 é a antiga Questão 3, com o mesmo texto. Textos-fonte verificados: `geradores/dados/lista03/q3-moedas-limpo.md` e `q5-passo-sem-base-limpo.md`. Na Questão 3, 4 de 4 verificadores sem defeito; uma observação lateral ("não são o mínimo para n ≥ 3") só estava demonstrada para n = 3 e ganhou a construção geral, reverificada com 4 de 4. Na Questão 5, 3 de 4 verificadores derrubaram uma nota que dizia que (ii) "exige" o Lema — falso, há o caminho por ¬P —; corrigida, 4 de 4 na reverificação. A bissecção foi simulada para n = 0…12 em toda posição da moeda falsa, e a estratégia de n − 1 pesagens para n = 3…11; as identidades da Questão 5 em n = 0…10^5 (`verificacao/atualizacao-2026-09-chao-deterministico.py`).
Sobre esta resolução. As fórmulas fechadas das recorrências (d) e (e) da Questão 4 foram conferidas por recursão direta em n = 1…2999 e n = 1…599 respectivamente; as somas de (b) e (c) em n = 1…4999; e a fronteira de (a) verificada item a item, confirmando que a afirmação é falsa em n = 1, 2, 3 e verdadeira a partir de n = 4. As constantes da Questão 1 foram calibradas e testadas em toda a faixa n = 1…200, incluindo o caso degenerado $\lg 1 = 0$. Na Questão 2, a ausência de cota inferior de ordem n^2 foi verificada para c = 1, 0,5, 0,01 e 0,0001. Os scripts estão em `resolucoes/verificacao/`. As demonstrações foram

submetidas a verificadores independentes em contexto isolado, que receberam apenas o enunciado e a prova final. Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.

Projeto e Análise de Algoritmos — Lista de Exercícios IV

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2 · Lista de 30/08/2026, atualizada em 09/09/2026
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Como ler esta resolução.** As Questões 1 e 2 seguem o mesmo esqueleto exigido pela disciplina: enunciar *P(n)*, verificar a **base** com os dois lados avaliados, escrever a **hipótese** e provar o **passo** marcando o ponto exato em que a hipótese é usada. Toda afirmação de Θ vem com c_1, c_2, n_0 numéricos, conferidos em $n = n_0$ exatamente. A Questão 3 é de lógica, não de conta: ali o que se pede é localizar o elo que falha. As Questões 4 e 5 entraram na versão de 09/09/2026: a 4 é mais uma indução sobre somatório; a 5 é, como a 3, uma prova defeituosa para diagnosticar — e o defeito é outro.

> Cabeçalho e instruções da lista (transcrição literal)
>
> Instituto Federal de Educação, Ciência e Tecnologia · Curso: Bacharelado em Ciência da Computação · Disciplina: Projeto e Análise de Algoritmos · Professor: Mário Luiz Rodrigues Oliveira · Atividade: Lista Exercícios IV · Formiga,MG, 30 de agosto de 2026
>
> **Instruções:**
>
> 1. Essa lista não será avaliada e, portanto, não precisa ser entregue. Resolva os exercícios como forma de fixação do conteúdo ministrado.
> 2. Caso você ache que falta algum detalhe nas especificações, você deverá fazer as suposições que julgar necessárias e escrevê-las junto à resolução do trabalho. Pode acontecer também que a descrição dessa atividade contenha dados e/ou especificações supérfluas para sua solução. Utilize sua capacidade de julgamento para separar o supérfluo do necessário.
> 3. Não basta escrever apenas o resultado. É necessário mostrar a resolução ou o raciocínio utilizado para responder às questões. Portanto, justifique todas as suas respostas.

Questão 1 — provas por indução

> Enunciado
>
> Use o método da indução matemática para provar as assertivas abaixo.
>
> (a) $S(n) : 1 + 3 + 5 + \ldots + (2 * n - 1) = n^2$, $\forall n \geq 1$
>
> (b) $T(n) = 2$ se $n = 1$; $T(n) = 2T(n-1)$ se $n \geq 2$. A fórmula fechada para essa recorrência é $T(n) = 2^n$
>
> (c) $T(n) = 1$ se $n = 1$; $T(n) = T(n-1) + 1$ se $n \geq 2$. A fórmula fechada para essa recorrência é $T(n) = n$
>
> (d) $T(n) = 1$ se $n = 1$; $T(n) = T(n-1) + 3$ se $n \geq 2$. A fórmula fechada para essa recorrência é $T(n) = 3 * n - 2$
>
> (e) $T(n) = 1$ se $n = 1$; $T(n) = T(n-1) + 2$ se $n \geq 2$. A fórmula fechada para essa recorrência $\in \theta(n)$

> **Uma observação de domínio que vale para (b), (c), (d) e (e), e que quase todo mundo pula.** Cada recorrência está definida sobre os inteiros $n \geq 1$ — *não* sobre $n \geq 0$. A primeira cláusula fixa o valor em $n = 1$; a segunda, *T(n) = … T(n−1) …*, vem sob a condição *"se $n \geq 2$"* e só pode ser aplicada quando o argumento é ≥ 2. Nos passos abaixo calcula-se $T(n+1)$ com $n \geq 1$; como $n \geq 1$ implica $n+1 \geq 2$, a segunda cláusula é aplicável — e é aí, e só aí, que a restrição de domínio é consumida. Uma prova que aplicasse a cláusula recursiva em $n = 1$ concluiria coisas sobre $T(0)$, que não existe.

(a) $1 + 3 + 5 + \ldots + (2n - 1) = n^2$

Proposição. Para cada inteiro $n \geq 1$, seja $P(n)$: $\sum_{k=1}^{n} (2k - 1) = n^2$. O somatório é a notação precisa para "1 + 3 + 5 + … + (2n − 1)"; em particular, para $n = 1$ a soma tem **um único** termo — a reticência não representa termo algum. É nesse sentido que a base é verificada.

Base (n = 1). Os dois lados, calculados separadamente: lado esquerdo = $2 \cdot 1 - 1 = 1$; lado direito = $1^2 = 1$. Como $1 = 1$, $P(1)$ é verdadeira.

Hipótese (h.i.). Seja $n \geq 1$ um inteiro *fixado* e suponha $\sum_{k=1}^{n} (2k - 1) = n^2$.

Passo. Separa-se o somatório até $n+1$ no somatório até n mais o termo $k = n+1$ — separação válida para todo $n \geq 0$, inclusive quando a soma restante é vazia:

$$\sum_{k=1}^{n+1} (2k - 1) = \left[\sum_{k=1}^{n} (2k - 1) \right] + (2(n+1) - 1)$$
$$= n^2 + (2(n+1) - 1) \leftarrow \text{(h.i.) aplicada ao colchete}$$
$$= n^2 + 2n + 1 = (n + 1)^2. \blacksquare$$

Detalhe que vale um ponto: a condição de fronteira que *de fato* importa aqui não é a do somatório — separar o último termo vale mesmo quando o que sobra é a soma vazia — e sim a convenção declarada acima, de que em $n = 1$ a soma tem um único termo. É ali que a base do item se decide.

(b) $T(1) = 2, T(n) = 2T(n-1) \implies T(n) = 2^n$

Base (n = 1). $T(1) = 2$ pela primeira cláusula; $2^1 = 2$. Como $2 = 2$, vale.

Hipótese. Seja $n \geq 1$ fixado e suponha $T(n) = 2^n$.

Passo. Como $n \geq 1$, então $n+1 \geq 2$, e *é por isso* que a segunda cláusula é aplicável ao argumento $n+1$:

$$T(n+1) = 2 \cdot T((n+1) - 1) = 2 \cdot T(n) = 2 \cdot 2^n \leftarrow \text{(h.i.)}$$
$$= 2^{n+1}. \blacksquare$$

> **A armadilha do item.** A mesma recorrência com base $T(1) = 1$ resolveria para 2^{n-1}, não 2^n. Aqui a base vale 2, e é a base que fixa a constante multiplicativa. Esse é exatamente o tipo de troca que o professor faz entre a lista e a prova (compare com a Q2(a) da prova de 2025, em que o + virou − e a resposta se inverteu).

(c) $T(1) = 1, T(n) = T(n-1) + 1 \implies T(n) = n$

> **Base (n = 1).** $T(1) = 1$ pela primeira cláusula; e $n = 1$. Como $1 = 1$, vale.
>
> **Hipótese.** Seja $n \geq 1$ fixado e suponha $T(n) = n$.
>
> **Passo.** Como $n \geq 1$, então $n+1 \geq 2$ e a segunda cláusula é aplicável ao argumento $n+1$:
>
> $$T(n+1) = T((n+1) - 1) + 1 = T(n) + 1 = n + 1 \leftarrow \text{(h.i.)} \quad \blacksquare$$

(d) $T(1) = 1, T(n) = T(n-1) + 3 \implies T(n) = 3n - 2$

> **Base (n = 1).** $T(1) = 1$ pela primeira cláusula; $3 \cdot 1 - 2 = 3 - 2 = 1$. Como $1 = 1$, vale.
>
> **Hipótese.** Seja $n \geq 1$ fixado e suponha $T(n) = 3n - 2$.
>
> **Passo.** Como $n \geq 1$, então $n+1 \geq 2$ e a segunda cláusula é aplicável:
>
> $$T(n+1) = T((n+1) - 1) + 3 = T(n) + 3 = (3n - 2) + 3 \leftarrow \text{(h.i.)}$$
> $$= 3n + 1 = 3n + 3 - 2 = 3(n + 1) - 2. \quad \blacksquare$$

> Dois pontos que valem nota. Primeiro: a base é *calculada*, não declarada — é exatamente ali que uma fórmula quase certa como $3n - 3$ ou $3n - 1$ seria refutada. Segundo: a constante da hipótese (o -2) reaparece **intacta** na conclusão. Constante que "cresce" no meio do passo é o erro mais comum da técnica.

(e) $T(1) = 1, T(n) = T(n-1) + 2 \implies T(n) \in \Theta(n)$

O enunciado pede a classe Θ. O caminho é em duas fases: primeiro a indução dá a forma fechada, depois as testemunhas dão o Θ.

Fase 1 — forma fechada: $T(n) = 2n - 1$. De onde vem o palpite: desenrolando a recorrência, $T(n) = T(n-1) + 2 = T(n-2) + 2 \cdot 2 = \ldots = T(1) + 2(n-1) = 1 + 2n - 2 = 2n - 1$. O desenrolar é *heurística*, não demonstração — quem demonstra é a indução abaixo, que parte da definição recursiva e chega à fórmula.

> **Base (n = 1).** $T(1) = 1$ pela primeira cláusula; $2 \cdot 1 - 1 = 1$. Como $1 = 1$, vale.
>
> **Hipótese.** Seja $n \geq 1$ fixado e suponha $T(n) = 2n - 1$.
>
> **Passo.** Como $n \geq 1$, então $n+1 \geq 2$ e a segunda cláusula é aplicável:
>
> $$T(n+1) = T((n+1) - 1) + 2 = T(n) + 2 = (2n - 1) + 2 \leftarrow \text{(h.i.)}$$
> $$= 2n + 1 = 2n + 2 - 1 = 2(n + 1) - 1. \quad \blacksquare$$

Fase 2 — de $T(n) = 2n - 1$ para $T(n) \in \Theta(n)$.

Pela definição (CLRS 3ª ed., §3.1), $\Theta(g(n))$ é o conjunto das funções f para as quais **existem** constantes positivas c_1, c_2, n_0 tais que, **para todo** $n \geq n_0$, vale $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$. A ordem dos quantificadores é $\exists c_1\ \exists c_2\ \exists n_0\ \forall n \geq n_0$: as três constantes são escolhidas primeiro, de uma vez, e não podem depender de n.

Testemunhas: $c_1 = 1$, $c_2 = 2$, $n_0 = 1$. Todas positivas ($1 > 0$, $2 > 0$, $1 > 0$) — condição da definição, não decorrência dela — e nenhuma contém n. Aqui $f(n) = T(n) = 2n - 1$ e $g(n) = n$.

> **Conferência em $n = n_0 = 1$, exatamente.** $c_1 \cdot g(1) = 1 \cdot 1 = 1$; $T(1) = 2 \cdot 1 - 1 = 1$ (e $T(1) = 1$ pela recorrência — os dois caminhos concordam); $c_2 \cdot g(1) = 2 \cdot 1 = 2$. Logo $0 \leq 1 \leq 1 \leq 2$. Vale em n_0, com igualdade na cota inferior — permitido, pois a definição usa ≤, não <.
>
> **Verificação para todo $n \geq 1$** (algébrica, não amostral). Seja n inteiro com $n \geq 1$.
>
> (i) $c_1 \cdot g(n) = n \geq 1 > 0$, portanto $0 \leq c_1 \cdot g(n)$.
>
> (ii) $n \leq 2n - 1$ equivale, subtraindo n dos dois lados, a $0 \leq n - 1$, isto é $1 \leq n$ — verdadeira por hipótese.
>
> (iii) $2n - 1 \leq 2n$ equivale, subtraindo $2n$ dos dois lados, a $-1 \leq 0$ — verdadeira incondicionalmente.
>
> Reunindo: para todo $n \geq 1$, $0 \leq 1 \cdot n \leq 2n - 1 = T(n) \leq 2 \cdot n$. Portanto $T(n) = 2n - 1 \in \Theta(n)$. $\blacksquare$

> $\Theta(n)$ é teto *e* piso; não diz que $T(n) = n$. De fato $T(n) \neq n$ para $n \geq 2$, e ainda assim $T \in \Theta(n)$ — é precisamente para isso que servem c_1 e c_2.

Item	Resultado provado	Detalhe que decide o item
(a)	$\sum_{k=1}^{n}(2k-1) = n^2,\ \forall n \geq 1$	base n = 1: 1 = 1; a soma tem um só termo
(b)	$T(n) = 2^n,\ \forall n \geq 1$	a base T(1) = 2 é o que dá 2^n, e não 2^{n-1}
(c)	$T(n) = n,\ \forall n \geq 1$	—
(d)	$T(n) = 3n - 2,\ \forall n \geq 1$	base n = 1: 3·1 − 2 = 1 = T(1); o −2 sobrevive ao passo
(e)	$T(n) = 2n - 1 \in \Theta(n)$	$c_1 = 1$, $c_2 = 2$, $n_0 = 1$; em n_0: $0 \leq 1 \leq 1 \leq 2$

Questão 2 — limites assintóticos por Teorema Mestre

> Enunciado
>
> Indique o limite assintótico usando a notação Θ, para cada uma das equações de recorrência abaixo. Considere o caso base $T(1) = 1$ ou $T(n) = c$, para alguma constante c e $\forall n \le n_0$, para alguma constante n_0. Suponha, também, que n uma potência exata de b. Escolha o valor adequado para a constante b conforme a equação de recorrência a ser resolvida. Justifique todas as suas respostas. Respostas sem justificativas não serão consideradas.
>
> (a) $T(n) = 2T(n/3) + 1$ (b) $T(n) = 5T(n/4) + n$ (c) $T(n) = 7T(n/7) + n$ (d) $T(n) = 9T(n/3) + n^2$ (e) $T(n) = 8T(n/2) + n^3$

O teorema na forma que a prova entrega

A prova desta disciplina fornece o Teorema Mestre na forma de **Brassard & Bratley**, que compara a com b^k — não $f(n)$ com $n^{\log_b a}$, como o CLRS. Hipóteses, como o formulário da prova as enuncia: $a \ge 1$ e $b \ge 2$ naturais, $k \ge 0$, $f(n) = c \cdot n^k$ com $c > 0$ real, n potência exata de b, e **T assintoticamente não decrescente**. A tabela abaixo está **na mesma ordem em que a prova imprime os casos** — assim "primeiro/segundo/terceiro caso" aponta para a linha certa com a folha na mesa.

caso	Comparação	Conclusão	Leitura
1º	$a < b^k$	$\Theta(n^k)$	a raiz domina
2º	$a = b^k$	$\Theta(n^k \cdot \lg n)$	todos os níveis custam o mesmo
3º	$a > b^k$	$\Theta(n^{\log_b a})$	as folhas dominam

O valor de b **não é escolhido arbitrariamente**: ele é lido no argumento da chamada recursiva. Se a chamada é $T(n/3)$, então $b = 3$. "Escolher o valor adequado de b", como diz o enunciado, significa usar essa base ao supor $n = b^m$. Dentro de Θ a base do logaritmo é irrelevante, pois $\log_x n = \lg n / \lg x$ e $1/\lg x$ é constante positiva; escreve-se $\lg = \log_2$.

Lema da expansão — onde a hipótese "n é potência exata de b" é usada

Aplicar a fórmula não é justificar. Como o enunciado garante que n é potência exata de b, dá para exibir a **forma fechada exata** e extrair dela as testemunhas. Isso vale para os cinco itens de uma vez.

> O lema abaixo é enunciado com $c = 1$, que é o caso dos cinco itens desta questão. Para $c > 0$ qualquer, o termo motor fica multiplicado por c, mas o termo do caso base a^m **não** — de modo que $T_c(n) = a^m + c \cdot (T_1(n) - a^m)$, e *não* $c \cdot T_1(n)$. Por isso as testemunhas não se reescalam por um simples fator c: no caso $a = b^k$ ficam $c_1 = c/\lg b$, $c_2 = (1+c)/\lg b$, $n_0 = b$; no caso $a > b^k$ ficam $c_1 = 1$, $c_2 = 1 + c \cdot B$, $n_0 = 1$. As classes Θ, que é o que importa aqui, não mudam.

Lema. Seja $T(1) = 1$ e $T(n) = a \cdot T(n/b) + n^k$ para $n > 1$. Se $n = b^m$ com $m \ge 0$ inteiro, então

$$T(b^m) = a^m + \sum_{i=0}^{m-1} a^i \cdot b^{k(m-i)}$$

> **Base (m = 0).** O lado direito vale $a^0 = 1$ (a soma é vazia) e $T(b^0) = T(1) = 1$. Confere.
>
> **Passo.** Suponha a fórmula válida para m. Então
>
> $$T(b^{m+1}) = a \cdot T(b^m) + (b^{m+1})^k = a \cdot a^m + \sum_{i=0}^{m-1} a^{i+1} b^{k(m-i)} + b^{k(m+1)}$$
>
> Trocando o índice para $j = i+1$, a soma vira $\sum_{j=1}^{m} a^j b^{k(m+1-j)}$; e o termo isolado $b^{k(m+1)}$ é exatamente a parcela $j = 0$ dessa mesma soma (substituindo $j = 0$ em $a^j b^{k(m+1-j)}$ obtém-se $1 \cdot b^{k(m+1)}$). Logo
>
> $$T(b^{m+1}) = a^{m+1} + \sum_{j=0}^{m} a^j b^{k(m+1-j)}, \blacksquare$$
>
> que é a fórmula para $m+1$.

> **Onde a hipótese entra.** Em cada um dos m desdobramentos, o argumento $b^m / b^i = b^{m-i}$ é *inteiro*, e a recursão atinge o caso base $T(1)$ após exatamente $m = \log_b n$ níveis. Sem essa hipótese apareceriam pisos e tetos $\lfloor n/b \rfloor$, e a igualdade acima seria apenas aproximada — a conclusão Θ continuaria valendo, a forma fechada exata não.

Somando a série geométrica. Escrevendo $b^{k(m-i)} = b^{km} \cdot (b^{-k})^i$, tem-se $\sum_{i=0}^{m-1} a^i b^{k(m-i)} = b^{km} \sum_{i=0}^{m-1} (a/b^k)^i$, com $b^{km} = (b^m)^k = n^k$ e $a^m = (b^m)^{\log_b a} = n^{\log_b a}$.

Caso $a = b^k$. A razão é 1, a soma tem m parcelas iguais a 1:

$$\text{(F1)} \quad T(n) = (m+1) \cdot n^k = n^k \cdot (\log_b n + 1)$$

Caso $a \ne b^k$. A soma geométrica de razão $a/b^k \ne 1$ vale $((a/b^k)^m - 1)/((a/b^k) - 1)$. Multiplicando numerador e denominador por b^k:

$$\text{(F2)} \quad T(n) = A \cdot n^{\log_b a} - B \cdot n^k, \quad \text{com } A = a/(a - b^k) \text{ e } B = b^k/(a - b^k)$$

Dois corolários que dão as testemunhas

Corolário 1 — caso $a > b^k$. Aqui $a - b^k > 0$, logo A e B são positivos, e $A - B = (a - b^k)/(a - b^k) = 1$. Como $a > b^k$ implica $\log_b a > k$, o expoente $k - \log_b a$ é negativo, e portanto $n^{k - \log_b a} \le 1$ para todo $n \ge 1$. Escrevendo $T(n) = n^{\log_b a} \cdot (A - B \cdot n^{k - \log_b a})$:

$$1 \cdot n^{\log_b a} = (A - B) \cdot n^{\log_b a} \le T(n) \le A \cdot n^{\log_b a}, \quad \forall\, n = b^m \ge 1$$

ou seja $\mathbf{c_1 = 1, c_2 = A, n_0 = 1}$. Isso é $\Theta(n^{\log_b a})$. Checagem de consistência: em $n = 1$ a fórmula dá $T(1) = A - B = 1$, que é o caso base dado.

Corolário 2 — caso $a = b^k$. De (F1), $T(n) = n^k \cdot (\lg n / \lg b + 1)$. Para $n \ge b$ vale $\lg n \ge \lg b$, isto é $1 \le \lg n / \lg b$; portanto

$$(1/\lg b)\cdot n^{k}\lg n \le T(n) \le (2/\lg b)\cdot n^{k}\lg n, \quad \forall\, n = b^{m} \ge b$$

ou seja $\mathbf{c_1 = 1/\lg b}$, $\mathbf{c_2 = 2/\lg b}$, $\mathbf{n_0 = b}$. Isso é $\Theta(n^k \lg n)$.

> **A restrição $n_0 = b$ não é cosmética — é o ponto de fencepost do item.** Com $n_0 = 1$ teríamos $\lg 1 = 0$, e a cota *superior* viraria $T(1) = 1 \le c_2 \cdot 0 = 0$, que é falsa. Escrever $n_0 = 1$ aqui perde o item inteiro.

(a) $T(n) = 2T(n/3) + 1$

Identificação. A chamada é $T(n/3) \Rightarrow b = 3$. Há duas chamadas $\Rightarrow a = 2$. O termo não recursivo é $1 = 1\cdot n^0 \Rightarrow k = 0$, $c = 1$. *(Armadilha: o "+1" é n^0, não n^1.)*

Comparação. $b^k = 3^0 = 1$. Como $2 > 1$, tem-se $a > b^k$: **terceiro caso**.

$\mathbf{T(n) \in \Theta(n^{\log_3 2})}$, com $\log_3 2 \approx 0{,}630930$. A forma exata $\log_3 2$ é a resposta; o decimal é só orientação.

> **Confirmação por expansão** ($n = 3^m$). Aqui $a/b^k = 2$, e (F2) dá $A = 2/(2-1) = 2$, $B = 1/(2-1) = 1$:
>
> $$T(3^m) = 2^{m+1} - 1, \quad \text{isto é} \quad T(n) = 2\cdot n^{\log_3 2} - 1$$
>
> Conferindo pela recorrência: $T(3) = 2\cdot T(1) + 1 = 3 = 2^2 - 1$; $T(9) = 2\cdot 3 + 1 = 7 = 2^3 - 1$.
>
> **Testemunhas** (Corolário 1, $A = 2$): $\mathbf{c_1 = 1}$, $\mathbf{c_2 = 2}$, $\mathbf{n_0 = 1}$. A cota inferior $n^{\log_3 2} \le 2n^{\log_3 2} - 1$ equivale a $n^{\log_3 2} \ge 1$, verdadeira exatamente para $n \ge 1$. Em $n = n_0 = 1$: $1 \le 1 \le 2$.

> **Cruzando com o CLRS.** $n^{\log_b a} = n^{\log_3 2}$ e $f(n) = 1 = O(n^{\log_3 2 - \varepsilon})$ para $\varepsilon = 0{,}5 > 0$ (pois $0{,}630930 - 0{,}5 = 0{,}130930 > 0$). Caso 1 do CLRS $\Rightarrow$ mesma resposta.

(b) $T(n) = 5T(n/4) + n$

Identificação. $b = 4$, $a = 5$, $f(n) = n = 1\cdot n^1 \Rightarrow k = 1$, $c = 1$.

Comparação. $b^k = 4$. Como $5 > 4$: **terceiro caso**.

$\mathbf{T(n) \in \Theta(n^{\log_4 5})}$, com $\log_4 5 \approx 1{,}160964$.

> **Confirmação por expansão** ($n = 4^m$). $a - b^k = 1$, logo $A = 5$, $B = 4$:
>
> $$T(4^m) = 5\cdot 5^m - 4\cdot 4^m, \quad \text{isto é} \quad T(n) = 5\cdot n^{\log_4 5} - 4n$$
>
> Conferindo: $T(4) = 5\cdot 1 + 4 = 9$ e $5\cdot 5 - 4\cdot 4 = 9$; $T(16) = 5\cdot 9 + 16 = 61$ e $125 - 64 = 61$.
>
> **Testemunhas** (Corolário 1, $A = 5$): $\mathbf{c_1 = 1}$, $\mathbf{c_2 = 5}$, $\mathbf{n_0 = 1}$. A cota inferior $5n^{\alpha} - 4n \ge n^{\alpha}$ (com $\alpha = \log_4 5$) equivale a $n^{\alpha-1} \ge 1$, com $\alpha - 1 \approx 0{,}161 > 0$ — verdadeira para todo $n \ge 1$. Em $n = n_0 = 1$: $T(1) = 5 - 4 = 1$, e $1 \le 1 \le 5$. Em $n = 4$: $5 \le 9 \le 25$.

> **Um susto que a forma fechada desfaz.** A razão $T(n)/n^{\log_4 5}$ vale 1,80 · 2,44 · 2,95 · 3,36 · 3,69 · 3,95 · 4,16 · 4,33 em $n = 4^1 \ldots 4^8$ — ainda subindo, o que *parece* divergência e sugeriria um fator logarítmico faltando. Não falta: a forma fechada dá $T(n)/n^{\alpha} = 5 - 4\cdot n^{1-\alpha}$, monótona crescente e **limitada por 5**. A convergência é lenta só porque $1 - \alpha \approx -0{,}161$ é pequeno em módulo. Ler tabela sem a forma fechada engana.

(c) $T(n) = 7T(n/7) + n$

Identificação. $b = 7$, $a = 7$, $k = 1$, $c = 1$.

Comparação. $b^k = 7 = a$: **segundo caso**.

$\mathbf{T(n) \in \Theta(n \lg n)}$.

> **Confirmação por expansão** ($n = 7^m$). Como $a = b^k$, cada nível custa o mesmo: no nível i há 7^i subproblemas de tamanho 7^{m-i}, custo total $7^i \cdot 7^{m-i} = 7^m = n$ por nível, para $i = 0, \ldots, m-1$; as $7^m = n$ folhas custam $T(1) = 1$ cada. Por (F1):
>
> $$T(7^m) = (m+1)\cdot 7^m, \quad \text{isto é} \quad T(n) = n\cdot(\log_7 n + 1) = (1/\lg 7)\cdot n \lg n + n$$
>
> Conferindo: $T(7) = 7\cdot 1 + 7 = 14 = 2\cdot 7$; $T(49) = 7\cdot 14 + 49 = 147 = 3\cdot 49$.
>
> **Testemunhas** (Corolário 2, $\lg 7 \approx 2{,}807355$): $\mathbf{c_1 = 1/\lg 7 \approx 0{,}356207}$, $\mathbf{c_2 = 2/\lg 7 \approx 0{,}712414}$, $\mathbf{n_0 = 7}$. Em $n = n_0 = 7$ exatamente: $n \lg n = 7\cdot\lg 7 \approx 19{,}6515$; $c_1\cdot 19{,}6515 \approx 7{,}0$ e $c_2\cdot 19{,}6515 \approx 14{,}0$. Como $T(7) = 14$, vale $7 \le 14 \le 14$ — a cota superior é atingida **com igualdade** em n_0, o que a definição permite (usa $\le$), mas mostra que c_2 não pode ser reduzido mantendo $n_0 = 7$.

> **Por que NÃO é $\Theta(n)$ — a resposta errada plausível do conjunto.** Com $a = b = 7$ a intuição sugere "n subproblemas de custo unitário, logo $\Theta(n)$". A forma fechada refuta: $T(n)/n = \log_7 n + 1$, que vale 2, 3, 4, 5 em $n = 7, 49, 343, 2401$ — **diverge**. Não existe c com $T(n) \le c\cdot n$ para todo n grande, logo $T \notin O(n)$ e, a fortiori, $T \notin \Theta(n)$. O fator $\lg n$ vem do *número de níveis*, não do custo por nível.

(d) $T(n) = 9T(n/3) + n^2$

Identificação. $b = 3$ (lido no argumento $n/3$), $a = 9$, $f(n) = n^2 = 1\cdot n^2 \Rightarrow k = 2$.

Comparação. $b^k = 3^2 = 9 = a$: **segundo caso**.

$\mathbf{T(n) \in \Theta(n^2 \lg n)}$.

Confirmação por expansão ($n = 3^m$). No nível i há 9^i subproblemas de tamanho 3^{m-i}, custo $9^i \cdot (3^{m-i})^2 = 9^i \cdot 9^{m-i} = 9^m = n^2$ — custo idêntico em todos os níveis. Por (F1), usando $9^m = (3^m)^2 = n^2$:

$$T(3^m) = (m+1)\cdot 9^m, \quad \text{isto é} \quad T(n) = n^2 \cdot (\log_3 n + 1)$$

Conferindo: $T(3) = 9\cdot 1 + 9 = 18 = 2\cdot 9$; $T(9) = 9\cdot 18 + 81 = 243 = 3\cdot 81$.

Testemunhas (Corolário 2, $lg\ 3 \approx 1{,}584963$): $\mathbf{c_1 = 1/lg\ 3 \approx 0{,}630930}$, $\mathbf{c_2 = 2/lg\ 3 \approx 1{,}261860}$, $\mathbf{n_0 = 3}$. Em $n = n_0 = 3$ exatamente: $n^2\ lg\ n = 9\cdot lg\ 3 \approx 14{,}2647$; as cotas dão $\approx 9{,}0$ e $\approx 18{,}0$; com $T(3) = 18$, vale $9 \le 18 \le 18$.

(e) $T(n) = 8T(n/2) + n^3$

Identificação. $b = 2$, $a = 8$, $k = 3$.

Comparação. $b^k = 2^3 = 8 = a$: **segundo caso**.

$T(n) \in \Theta(n^3 \lg n)$.

Confirmação por expansão ($n = 2^m$). No nível i: 8^i subproblemas de tamanho 2^{m-i}, custo $8^i \cdot (2^{m-i})^3 = 8^m = n^3$ por nível. Por (F1), com $b = 2$ não há fator de conversão ($log_2\ n = lg\ n$):

$$T(2^m) = (m+1)\cdot 8^m, \quad \text{isto é} \quad T(n) = n^3 \cdot (\lg n + 1) = n^3 \lg n + n^3$$

Conferindo: $T(2) = 8\cdot 1 + 8 = 16 = 2\cdot 8$; $T(4) = 8\cdot 16 + 64 = 192 = 3\cdot 64$.

Testemunhas (Corolário 2, $lg\ 2 = 1$): $\mathbf{c_1 = 1}$, $\mathbf{c_2 = 2}$, $\mathbf{n_0 = 2}$. Em $n = n_0 = 2$ exatamente: $n^3\ lg\ n = 8\cdot 1 = 8$; as cotas dão 8 e 16; com $T(2) = 16$, vale $8 \le 16 \le 16$.

O caso base não muda nenhuma das respostas

O enunciado admite $T(1) = 1$ *ou* $T(n) = c$ para todo $n \le n_{base}$. Refazendo o Lema com $T(1) = c$, o único termo afetado é $a^m \cdot T(1) = c\cdot a^m$:

$$\text{(a) } (c+1)n^{\log_3 2} - 1 \ \cdot\ \text{(b) } (c+4)n^{\log_4 5} - 4n \ \cdot\ \text{(c) } n(\log_7 n + c) \ \cdot\ \text{(d) } n^2(\log_3 n + c) \ \cdot\ \text{(e) } n^3(\lg n + c)$$

Cuidado com o símbolo. Aqui c é a constante do *caso base*, não o coeficiente do termo motor — e os dois papéis dão testemunhas diferentes. Com $T(1) = c$ a razão $T(n)/(n^k\ lg\ n)$ vale $1/lg\ b + c/lg\ n$, cujo ínfimo é $1/lg\ b$ e **não depende de c**; no regime do termo motor a razão é $1/lg\ n + c/lg\ b$, com ínfimo $c/lg\ b$. Trocar um pelo outro produz uma cota inferior falsa: no item (e) com $c = 3$ tem-se $T(4) = 320$, e a cota $c/lg\ b$ exigiria 384.

Assim, no caso $a = b^k$ com $T(1) = c$ as testemunhas são $\mathbf{c_1 = 1/lg\ b}$, $\mathbf{c_2 = (1+c)/lg\ b}$, $\mathbf{n_0 = b}$; e no caso $a > b^k$, como $T_c(n) = T_1(n) + (c-1)\cdot a^m$ e $A - B = 1$, ficam $\mathbf{c_1 = c}$, $\mathbf{c_2 = c + B}$, $\mathbf{n_0 = 1}$. Em todos os casos, $c > 0$ muda c_1 e c_2 apenas por constantes — nunca a classe. Nos casos de igualdade o fator $lg\ n$ sobrevive porque vem do **número de níveis** ($log_b\ n$), que independe do caso base. Interromper a recursão em $n \le n_{base}$ em vez de $n = 1$ corta apenas $log_b\ n_{base} = O(1)$ níveis — os mais profundos da árvore de recursão.

item	recorrência	a	b	k	b^k	caso	forma fechada ($n = b^m$, $T(1)=1$)	resposta
(a)	$2T(n/3)+1$	2	3	0	1	3º ($a > b^k$)	$2n^{\log_3 2} - 1$	$\boldsymbol{\Theta(n^{\log_3 2})} \approx \Theta(n^{0{,}6309})$
(b)	$5T(n/4)+n$	5	4	1	4	3º ($a > b^k$)	$5n^{\log_4 5} - 4n$	$\boldsymbol{\Theta(n^{\log_4 5})} \approx \Theta(n^{1{,}1610})$
(c)	$7T(n/7)+n$	7	7	1	7	2º ($a = b^k$)	$n(log_7\ n + 1)$	$\boldsymbol{\Theta(n\ lg\ n)}$
(d)	$9T(n/3)+n^2$	9	3	2	9	2º ($a = b^k$)	$n^2(log_3\ n + 1)$	$\boldsymbol{\Theta(n^2\ lg\ n)}$
(e)	$8T(n/2)+n^3$	8	2	3	8	2º ($a = b^k$)	$n^3(lg\ n + 1)$	$\boldsymbol{\Theta(n^3\ lg\ n)}$

Ressalva de escopo. As formas fechadas exatas valem apenas sobre $n = b^m$. Para n arbitrário, com pisos, a conclusão Θ continua válida, mas a igualdade exata não. As testemunhas exibidas certificam o Θ nessa subsequência — que é exatamente o que o enunciado pede ao mandar supor potência exata de b.

Questão 3 — a prova por indução defeituosa

Enunciado

Considere a seguinte prova por indução matemática:

1. **Proposição.** Dado um conjunto de n pessoas, se pelo menos uma pessoa tem a nacionalidade chinesa, então todas os demais pessoas também são chinesas.
2. **Caso base (n = 1):** Em um conjunto com uma pessoa, se ela é chinesa, então todas as pessoas do conjunto são chinesa. Portanto, a afirmação é verdadeira.
3. **Hipótese de indução:**
   - Suponha que a proposição é verdadeira para $n = 3$;
   - Sejam T_1, T_2, T_3 e T_4 quatro pessoas das quais pelo menos uma tem nacionalidade chinesa (suponha que é T_1);
   - Supondo que o conjunto $\{T_1, T_2, T_3\}$ é a hipótese de que é verdadeiro para $n = 3$, então T_2 e T_3 são pessoas com nacionalidade chinesa;
   - Analogamente para $\{T_1, T_2, T_4\}$, tem-se que T_2 e T_4 são pessoas com nacionalidade chinesa.

   Portanto, as quatro pessoas tem nacionalidade chinesa! A generalização da construção para k e $k + 1$ é a prova desejada.

Pergunta: Essa prova está correta? Justifique formalmente sua resposta.

Transcrição literal do PDF do professor, inclusive as concordâncias “todas os demais pessoas” e “são chinesa”.

Não, a prova não está correta. São dois defeitos, e vale separá-los. O *imediato*, no texto tal como escrito: ele supõe *P(3)*, que nunca foi derivada da base *P(1)*, e apresenta a conclusão como fato categórico — de modo que o máximo que estabelece é a implicação *P(3)* → *P(4)*. O *de fundo*, que aparece ao completar a generalização que o texto delega a um "analogamente": a proposição que ela pretende demonstrar é **falsa**. Como o Princípio da Indução Finita é uma regra de inferência correta — de premissas verdadeiras nunca extrai conclusão falsa —, uma "demonstração" de proposição falsa tem obrigatoriamente ao menos uma premissa não estabelecida. Abaixo: a refutação da proposição, a localização exata da premissa que falha, os defeitos adicionais de redação e o diagnóstico estrutural.

0. Notação

Para cada inteiro $n \geq 1$, seja

P(n): "em todo conjunto *C* de *n* pessoas, se existe pelo menos uma pessoa chinesa em *C*, então todas as pessoas de *C* são chinesas".

Duas observações que decidem tudo o que vem depois: *P(n)* é uma afirmação **universal sobre todos os conjuntos** de tamanho *n*, e cada instância é uma **implicação** (antecedente: "há pelo menos um chinês"; consequente: "todos são chineses").

1. A proposição é falsa — contraexemplo

Tome $n = 2$ e $C = \{T_1, T_2\}$, com T_1 de nacionalidade chinesa e T_2 de nacionalidade brasileira. O antecedente é verdadeiro (T_1); o consequente é falso (T_2 não é). Implicação com antecedente verdadeiro e consequente falso é falsa: **P(2) é falsa**. O mesmo contraexemplo — $C = \{T_1$ *chinês*, T_2, ..., T_n *não-chineses*$\}$ — refuta *P(n)* para qualquer $n \geq 2$ para o qual existam *n* pessoas; e é *P(2)* que o resto desta resposta usa.

n	1	2	3	4	5	…
P(n)	**V**	F	F	F	F	…

> A tabela supõe pessoas em número suficiente para formar cada conjunto — num universo com menos de *n* pessoas, *P(n)* seria vacuamente verdadeira, por falta de conjunto a testar. Isso não socorre a prova: o que ela precisaria é de *P(2)*, e *P(2)* é falsa em qualquer universo com ao menos uma pessoa chinesa e ao menos uma não-chinesa.

Como a conclusão pretendida — "*P(n)* para todo *n*" — é falsa, o veredito está encerrado. O que resta é responder *onde*.

2. Localização exata do defeito

2.1 Reconstrução do passo. O texto não escreve o passo genérico — diz "a generalização … é a prova desejada". Mas a construção exibida em $3 \to 4$ tem uma generalização natural (e o item 2.4 mostra que a conclusão não depende de qual escolha de elementos removidos se adote):

> Seja $k \geq 1$ e suponha *P(k)*. Seja $S = \{T_1, ..., T_{k+1}\}$ com pelo menos uma pessoa chinesa; sem perda de generalidade, T_1. Defina $A = S \setminus \{T_{k+1}\}$ e $B = S \setminus \{T_k\}$. Então $|A| = |B| = k$ e $A \cup B = S$. Aplicando *P(k)* a *A* e a *B*, e usando $A \cup B = S$, conclui-se *P(k+1)*.

No texto, $k = 3$: $A = \{T_1,T_2,T_3\}$ e $B = \{T_1,T_2,T_4\}$.

2.2 A condição exata de aplicabilidade da hipótese. *P(k)* é uma *implicação*. Aplicá-la a um conjunto *X* com $|X| = k$ só produz "todos de *X* são chineses" se o **antecedente estiver verificado** para *X* — isto é, se soubermos que *X* contém pelo menos uma pessoa chinesa. Ter $|X| = k$ não basta.

A sutileza é que a lista de pessoas sabidamente chinesas **cresce** ao longo do argumento. No início a única testemunha é T_1; mas ao aplicar *P(k)* a *A*, que contém T_1, *todo* elemento de *A* passa a ser sabidamente chinês — e então qualquer elemento de $A \cap B$ serve de testemunha para a segunda aplicação. Com a rotulação de 2.1 — *A* é o primeiro subconjunto a receber a hipótese —, o passo exige $T_1 \in A$ e $A \cap B \neq \varnothing$. Mas $T_1 \in A \cup B = S$ sempre vale, de modo que chamar de *A* aquele que contém T_1 é escolha de *rótulo*, não exigência. A condição substantiva — a única que pode falhar — é:

$$\mathbf{A \cap B \neq \varnothing}$$

(Exigir $T_1 \in A \cap B$ também funcionaria, mas é forte demais: para $k = 3$ o par $A = \{T_1,T_2,T_3\}$, $B = \{T_2,T_3,T_4\}$ fecha o passo sem que T_1 esteja em *B*.)

Na construção do texto, a conta é imediata:

$$A \cap B = S \setminus \{T_k, T_{k+1}\} = \{T_1, ..., T_{k-1}\}, \quad \text{com} \quad |A \cap B| = k - 1$$

Logo $A \cap B \neq \varnothing$ se e somente se $k \geq 2$. A cardinalidade nem depende da rotulação: $|A| = |B| = k$ e $|A \cup B| = k + 1$ forçam $|A \cap B| = 2k - (k+1) = k - 1$.

2.3 O caso k = 1, onde tudo desaba. Para $k = 1$ temos $S = \{T_1, T_2\}$ com T_1 chinês, e a construção dá $A = \{T_1\}$, $B = \{T_2\}$.

> Aplicar *P(1)* a *A* é legítimo, mas conclui apenas "T_1 é chinês" — nada de novo.
>
> Aplicar *P(1)* a *B* é **ilegítimo**: não se sabe que T_2 é chinês — esse é exatamente o fato que se quer provar. O antecedente de *P(1)* para *B* não está verificado, e de *P(1)* nada se deduz sobre *B*. Insistir seria usar a tese para provar a tese.
>
> Consequentemente $S = A \cup B$ não fica coberto: sobre T_2 não se obteve nada.

Conclusão: a implicação P(1) → P(2) não é demonstrada pela construção. É o único elo faltante — e, pelo item 1, é também um elo **falso** (*P(1)* verdadeira, *P(2)* falsa). Não há remendo possível: não se trata de prova malfeita de algo verdadeiro, e sim da tentativa de provar algo falso.

2.4 O defeito não depende da rotulação. Qualquer versão dessa construção obtém os dois subconjuntos de tamanho k removendo de S dois elementos *distintos* (removendo o mesmo duas vezes teríamos $A = B$ e $A \cup B \neq S$, e o argumento nem cobriria S). Removendo dois elementos distintos de um conjunto de $k+1 = 2$ elementos, os subconjuntos resultantes são forçosamente os singletons $\{T_1\}$ e $\{T_2\}$, que são **disjuntos** — e sem interseção não há como transportar testemunha de um para o outro, qualquer que seja a rotulação.

3. Defeitos adicionais de redação

(a) A hipótese foi enunciada para um valor fixo. O texto escreve "suponha que a proposição é verdadeira para $n = 3$". Isso não é hipótese de indução legítima: a hipótese é "seja $k \geq n_0$ *arbitrário* e suponha $P(k)$". Duas regras trabalham juntas — descarregar a hipótese $P(k)$ é o que produz a implicação $P(k) \to P(k+1)$, e a arbitrariedade de k é o que autoriza a *generalização universal*, $\forall k\,(P(k) \to P(k+1))$. Fixando $n = 3$, a primeira ainda se aplica — assumir $P(3)$, derivar $P(4)$ e descarregar produz legitimamente a implicação $P(3) \to P(4)$ — mas a segunda não: sem k arbitrário não há generalização universal. É por isso que o texto fica preso a essa única implicação concreta.

(b) P(3) nunca foi estabelecida — e é falsa. O caso base provado é $P(1)$; nada liga $P(1)$ a $P(3)$. Sutileza importante: *supor* $P(3)$ dentro de uma prova condicional não é falácia — obtém-se legitimamente a implicação $P(3) \to P(4)$. A falácia está em **concluir o consequente de uma prova condicional sem ter estabelecido o antecedente** — modus ponens sem a premissa menor: a frase "Portanto, as quatro pessoas tem nacionalidade chinesa!" é apresentada como fato categórico quando só vale sob uma hipótese não demonstrada.

(c) O rótulo está trocado. O que o texto chama de "hipótese de indução" é o *passo* indutivo, e apenas na instância concreta $3 \to 4$. Enunciar a hipótese dentro do passo é prática comum e não seria defeito; o defeito é o rótulo esconder que ali está sendo feita justamente a parte que precisaria de k arbitrário — e a quantificação universal sobre k nunca aparece.

(d) "A generalização … é a prova desejada" é um buraco, não um passo. É o padrão "analogamente / claramente": a frase ocupa exatamente o lugar onde a demonstração deveria estar — e é precisamente onde a prova falha. Se o autor tivesse escrito o passo genérico com os quantificadores explícitos, seria obrigado a justificar que $A \cap B \neq \varnothing$ — na construção do texto, que $T_1 \in B$ — e o caso $k = 1$ apareceria imediatamente.

(e) Base e passo não se encaixam. A base é $n = 1$; o passo só funciona de $k = 2$ em diante.

4. Diagnóstico estrutural: por que o dominó não cai

O texto do professor não escreve o passo genérico — exibe só a instância $3 \to 4$ e delega o resto a um "analogamente". Completando-o como em 2.1, com a aplicabilidade da hipótese verificada como em 2.2, dá para separar o que fica de pé do que não fica. A indução é uma cadeia de implicações ancorada na base:

$$P(1) \to P(2) \to P(3) \to P(4) \to \dots$$

Peça	Status
Base: $P(1)$	verdadeira ✔
Passo: $P(k) \to P(k+1)$ para todo $k \geq 2$	demonstrável — mas pela reconstrução de 2.1, não pelo texto ✔
Elo $P(1) \to P(2)$	**não demonstrado — e falso ✘**

O primeiro dominó cai e os do segundo em diante estão bem posicionados uns em relação aos outros, mas **ninguém empurra o segundo**. A única instância $P(n)$ inferível dessas premissas é $P(1)$.

Por que "base válida + passo válido para quase todo k" não basta. O argumento decisivo é um contramodelo. Tome um mundo com pelo menos uma pessoa chinesa e pelo menos uma não-chinesa — T_1 chinês, T_2 brasileiro, e quantas mais se queira. Confira nele as premissas oferecidas:

> (i) $P(1)$ é verdadeira, logo a base vale — de fato $P(1)$ é verdadeira em todo mundo;
>
> (ii) $P(k) \to P(k+1)$ vale para todo $k \geq 2$, logo o passo vale — a derivação de 2.1 é correta sempre que $k \geq 2$, qualquer que seja o mundo;
>
> (iii) e, no entanto, $\forall n\, P(n)$ é falsa, pois $P(2)$ falha pelo contraexemplo do item 1.

Existe, portanto, um modelo que satisfaz *todas* as premissas oferecidas e refuta a conclusão. Logo essas premissas não implicam a conclusão — nenhuma reescrita da prova poderia salvá-la. E, coerentemente, a única implicação da cadeia que é falsa nesse modelo é exatamente $P(1) \to P(2)$, a que o texto não demonstrou.

> **Lição geral.** Uma única lacuna em $k = m$ corta a cadeia, e nada a respeito de $n > m$ fica demonstrado — a menos que se prove independentemente uma nova base acima de m. Aqui não há base a resgatar: qualquer nova base teria de recuperar $P(2)$ para cobrir $n = 2$, e $P(2)$ é falsa pelo contraexemplo do item 1.

5. O nome do padrão, e a regra de higiene

É o disfarce brasileiro de um clássico. A falácia foi levantada por George Pólya em *Induction and Analogy in Mathematics* (1954), na forma "quaisquer n moças têm olhos da mesma cor"; a formulação que ficou famosa — "todos os cavalos têm a mesma cor" — é creditada a Joel E. Cohen (1961). A anatomia é idêntica à desta questão: base em $n = 1$ e passo por dois subconjuntos de tamanho $n-1$, que só têm elemento em comum quando $n - 1 \geq 2$.

O que o Princípio da Indução Finita realmente exige. Seja $P(n)$ propriedade sobre os inteiros e $n_0 \in \mathbb{Z}$. Se (1) $P(n_0)$ é verdadeira e (2) para **todo** inteiro $k \geq n_0$ vale $P(k) \to P(k+1)$, então $P(n)$ é verdadeira para todo $n \geq n_0$. A quantificação em (2) é universal e *sem exce-*

ções: o passo tem de valer para todo k a partir de n_0, começando no próprio n_0. Uma demonstração do passo que use, ainda que tacitamente, uma hipótese extra — aqui $k \geq 2$, disfarçada na suposição não declarada de que $A \cap B \neq \varnothing$ (na construção do texto, de que $T_1 \in B$) — estabelece apenas o passo *restrito* a esse domínio.

> **Regra de higiene, que é o que transfere para a prova.** Em toda prova por indução, **substitua $k = n_0$ literalmente no passo e refaça a conta**, em vez de confiar no k genérico. Neste exercício, $k = 1$ destrói a prova em duas linhas — e foi por não fazer esse teste, escondendo o caso geral atrás de um "a generalização … é a prova desejada", que o argumento pareceu convincente. É o mesmo ataque de *fencepost* que decide as Questões 1 e 2.

Questão 4 — soma dos quadrados

> Enunciado
>
> Prove a proposição $P(n) = 1^2 + 2^2 + 3^2 + \ldots + n^2 = n * (n + 1) * (2n + 1)/6$ usando o método da indução matemática.

$1^2 + 2^2 + \ldots + n^2 = n(n+1)(2n+1)/6$, para todo $n \geq 1$

Suposições declaradas. (1) O enunciado escreve "P(n) = …", mas $P(n)$ é uma proposição, não um número. Lê-se $P(n)$: $1^2 + 2^2 + \ldots + n^2 = n(n+1)(2n+1)/6$. (2) O enunciado não fixa o domínio de n. Supõe-se n inteiro, $n \geq 1$, que é o menor valor em que a soma escrita "$1^2 + 2^2 + \ldots + n^2$" tem ao menos o termo 1^2. (A identidade também vale em $n = 0$, lendo o lado esquerdo como soma vazia, igual a 0: $0 \cdot 1 \cdot 1/6 = 0$. Isso não é necessário para o que se pede.) (3) Em notação de somatório, $P(n)$: $\Sigma_{k=1}^{n} k^2 = n(n+1)(2n+1)/6$.

Princípio usado. Se $P(1)$ é verdadeira e, para todo inteiro $n \geq 1$, $P(n)$ implica $P(n+1)$, então $P(n)$ é verdadeira para todo inteiro $n \geq 1$.

> **Base (n = 1).** Lado esquerdo: $1^2 = 1$. Lado direito: $1 \cdot (1+1) \cdot (2 \cdot 1+1)/6 = 1 \cdot 2 \cdot 3/6 = 6/6 = 1$. Como $1 = 1$, $P(1)$ é verdadeira.
>
> **Hipótese de indução.** Seja $n \geq 1$ um inteiro fixado, e suponha $P(n)$: $\Sigma_{k=1}^{n} k^2 = n(n+1)(2n+1)/6$.
>
> **Passo.** Quer-se $P(n+1)$: $\Sigma_{k=1}^{n+1} k^2 = (n+1)((n+1)+1)(2(n+1)+1)/6 = (n+1)(n+2)(2n+3)/6$. Separa-se o último termo da soma:
>
> $$\Sigma_{k=1}^{n+1} k^2 = [\Sigma_{k=1}^{n} k^2] + (n+1)^2 = n(n+1)(2n+1)/6 + (n+1)^2 \quad \leftarrow \text{hipótese aplicada ao colchete}$$
> $$= [n(n+1)(2n+1) + 6(n+1)^2]/6 = (n+1) \cdot [n(2n+1) + 6(n+1)]/6 \quad \leftarrow \text{fator comum } (n+1)$$
> $$= (n+1) \cdot (2n^2 + n + 6n + 6)/6 = (n+1)(2n^2 + 7n + 6)/6.$$
>
> Fatorando o trinômio: $(n+2)(2n+3) = 2n^2 + 3n + 4n + 6 = 2n^2 + 7n + 6$. Logo $\Sigma_{k=1}^{n+1} k^2 = (n+1)(n+2)(2n+3)/6$, que é exatamente o lado direito de $P(n+1)$. Portanto $P(n)$ implica $P(n+1)$, para todo $n \geq 1$.
>
> **Conclusão.** Pelo princípio da indução matemática, $P(n)$ é verdadeira para todo inteiro $n \geq 1$. ∎

> **Conferência de fencepost.** $n = 1$: $1 = 1$ (base). $n = 2$: $1 + 4 = 5$ e $2 \cdot 3 \cdot 5/6 = 5$. $n = 3$: $1 + 4 + 9 = 14$ e $3 \cdot 4 \cdot 7/6 = 14$. O passo com $n = 1$ (o menor valor em que é aplicado): $1 \cdot 2 \cdot 3/6 + 2^2 = 1 + 4 = 5 = 2 \cdot 3 \cdot 5/6$. É a mesma identidade do IFMG, ex. 1, do banco de questões (lá escrita a partir de 0^2).

Questão 5 — a prova defeituosa sobre árvores

> Enunciado
>
> Considere a proposição P(n): Toda árvore com n vértices possui exatamente n-1 arestas.
>
> Um estudante apresentou a seguinte prova por indução.
>
> **Caso base.** Para n = 1, uma árvore possui um único vértice e nenhuma aresta. Portanto, 0 = 1 − 1, e a afirmação é verdadeira para n = 1.
>
> **Hipótese de indução.** Suponha que toda árvore com n vértices possua exatamente n−1 arestas.
>
> **Passo indutivo.** Considere uma árvore T com n vértices. Pela hipótese de indução, T possui n − 1 arestas. Acrescente a T um novo vértice v e ligue-o, por meio de uma única aresta, a algum vértice de T. O novo vértice v é uma folha, e o grafo resultante continua sendo uma árvore. Assim, a nova árvore possui n + 1 vértices e (n − 1) + 1 = n arestas. Como n = (n + 1) − 1, conclui-se que toda árvore com n + 1 vértices possui n arestas. Logo, pelo princípio da indução matemática, toda árvore com n vértices possui exatamente n − 1 arestas.
>
> **Pergunta:** Embora a afirmação seja verdadeira, a prova acima é logicamente incorreta. Identifique precisamente o erro no passo indutivo e explique por que a construção apresentada não é suficiente para demonstrar que toda árvore com n + 1 vértices possui n arestas.

O erro está no passo indutivo: ele parte de uma árvore T com n vértices e constrói, pendurando uma folha, UMA árvore com n + 1 vértices; prova assim a propriedade apenas para as árvores de n + 1 vértices obtidas por essa construção, e conclui para TODAS, sem provar que toda árvore com n + 1 vértices pode ser obtida assim. P(n+1) é universal e exige partir de uma árvore ARBITRÁRIA T' com n + 1 vértices; o caminho natural é reduzi-la a n vértices. A construção não basta porque a mesma forma de argumento "prova" a afirmação falsa "todo grafo com n ≥ 2 vértices, todos de grau ≥ 1, é conexo" (contraexemplo: duas arestas disjuntas, 4 vértices). O conserto: toda árvore com n + 1 ≥ 2 vértices tem uma folha v (caminho máximo), T' − v é árvore com n vértices, e a hipótese dá |E(T')| = (n − 1) + 1 = n.

> **Convenções.** Grafo simples, finito. Árvore = grafo conexo e acíclico. Folha = vértice de grau 1. n é inteiro, $n \geq 1$.

1. O que o passo indutivo precisa provar

Fixado $n \geq 1$ e suposta a hipótese de indução, o passo precisa estabelecer $P(n+1)$, que é uma afirmação universal:

$$P(n+1): \quad \text{para TODA árvore T' com n + 1 vértices, } |E(T')| = n.$$

Para provar uma afirmação universal, parte-se de um objeto ARBITRÁRIO da classe — uma árvore T' qualquer com $n+1$ vértices — e mostra-se que ele tem a propriedade.

2. O que o passo do estudante efetivamente prova

O texto parte de uma árvore T com n vértices, escolhe um vértice u de T, e forma $T+v$ (novo vértice v e aresta uv). Mostra que $T+v$ tem $(n-1)+1=n$ arestas. O que fica provado é:

para toda árvore T com n vértices e todo vértice u de T, a árvore T + v tem n arestas.

Isto é, a propriedade vale para as árvores de $n+1$ vértices QUE SÃO OBTIDAS por esta construção. A frase final — "conclui-se que toda árvore com n + 1 vértices possui n arestas" — troca "toda árvore construída assim" por "toda árvore". Essa troca só seria legítima se estivesse provado que TODA árvore com $n+1$ vértices pode ser obtida pendurando uma folha em alguma árvore de n vértices. Esse fato não é enunciado nem provado em lugar nenhum do texto.

3. O erro, com precisão

O passo começa numa árvore de n vértices e CONSTRÓI uma de $n+1$ ($n \to n+1$). Construir não é, por si só, inválido; mas, assim, o passo cobre apenas a imagem da construção, e conclui sobre a classe inteira sem mostrar que a imagem é a classe inteira. O caminho que evita essa lacuna é o inverso: começar numa árvore ARBITRÁRIA de $n+1$ vértices e REDUZI-LA a uma de n vértices, onde a hipótese se aplica ($n+1 \to n$).

É um erro de quantificador: prova-se "$\forall T\ \forall u$: T + v tem n arestas" e conclui-se "$\forall T'$ com n + 1 vértices: T' tem n arestas", sem o elo "$\forall T'\ \exists T\ \exists u$: $T' = T + v$" (igualdade a menos do nome dos vértices). (Esse padrão é conhecido em inglês como *build-up error*, "erro de construção".)

4. Por que a construção não basta: a mesma forma de argumento prova uma falsidade

Se o argumento do estudante fosse válido pela sua forma, a mesma forma provaria qualquer coisa com a mesma estrutura. Considere a afirmação FALSA

C(n): todo grafo com n vértices, todos de grau ≥ 1, é conexo (n ≥ 2).

"Prova" com a mesma forma. **Base (n = 2):** o único grafo simples com 2 vértices, ambos de grau ≥ 1, é a aresta única, que é conexo; $C(2)$ vale. **Hipótese:** todo grafo com n vértices, todos de grau ≥ 1, é conexo. **"Passo":** tome um grafo G com n vértices, todos de grau ≥ 1; pela hipótese, G é conexo. Acrescente um novo vértice v ligado por uma aresta a algum vértice de G. O novo grafo tem $n+1$ vértices, todos de grau ≥ 1, e é conexo (v está ligado a G, que é conexo). "Logo" todo grafo com $n+1$ vértices, todos de grau ≥ 1, é conexo.

Mas $C(4)$ é falsa: o grafo com vértices {1, 2, 3, 4} e arestas {1,2} e {3,4} tem todos os graus iguais a 1 e não é conexo. A construção nunca produz esse grafo: ela parte de um grafo de 3 vértices com todos os graus ≥ 1, e remover qualquer vértice de {1,2},{3,4} deixa um vértice de grau 0 — logo ele não é imagem da construção. O passo "construtivo" só enxergou os grafos que ele mesmo gera.

Portanto a forma de argumento usada pelo estudante é inválida; que a proposição $P(n)$ sobre árvores seja verdadeira é um fato que a prova do estudante não estabelece.

5. O conserto: partir de uma árvore arbitrária com n + 1 vértices

Lema 1. Toda árvore com $m \geq 2$ vértices tem uma folha.
Prova. Seja T' árvore com $m \geq 2$ vértices. Como T' é conexa e tem pelo menos dois vértices, tem pelo menos uma aresta; logo existem caminhos com pelo menos uma aresta. Seja $P = x_0x_1\ldots x_k$ um caminho de comprimento máximo em T' (existe, pois T' é finita e todo caminho tem no máximo m vértices); $k \geq 1$. Todo vizinho de x_0 está em P: se x_0 tivesse um vizinho y fora de P, $yx_0x_1\ldots x_k$ seria caminho mais longo. Se x_0 fosse vizinho de x_i com $i \geq 2$, então $x_0x_1\ldots x_ix_0$ seria um ciclo, impossível em árvore. Logo o único vizinho de x_0 é x_1, e $grau(x_0) = 1$. ∎

Lema 2. Se v é folha de uma árvore T' com $m \geq 2$ vértices, então $T' - v$ (remove-se v e a única aresta incidente a v) é uma árvore com $m-1$ vértices.
Prova. Acíclico: todo ciclo de $T' - v$ seria ciclo de T'. Conexo: sejam x, y vértices de $T' - v$. Em T' existe um caminho de x a y. Um vértice interno de um caminho tem grau ≥ 2 dentro dele; como $grau(v) = 1$ e $v \notin \{x, y\}$, v não está nesse caminho, que portanto está inteiro em $T' - v$. ∎

> **Passo correto.** Seja $n \geq 1$ e suponha a hipótese de indução (toda árvore com n vértices tem $n-1$ arestas). Seja T' uma árvore ARBITRÁRIA com $n+1$ vértices. Como $n+1 \geq 2$, pelo Lema 1 T' tem uma folha v. Pelo Lema 2, $T = T' - v$ é árvore com n vértices; pela hipótese de indução, $|E(T)| = n-1$. Como v tem grau 1, T' tem exatamente uma aresta a mais que T: $|E(T')| = (n-1)+1 = n$. Como T' era arbitrária, $P(n+1)$ vale. Com a base do estudante (que está correta), a indução fica completa. ∎

6. Observações

Para árvores, a imagem da construção É a classe inteira: pelos Lemas 1 e 2, toda árvore T' com $n+1 \geq 2$ vértices é $(T' - v) + v$. Por isso a conclusão é verdadeira. Mas esse fato é exatamente o conteúdo dos Lemas 1 e 2 — o trabalho que o estudante deixou de fazer.

A afirmação do estudante de que "o grafo resultante continua sendo uma árvore" é verdadeira (conexo porque v se liga a T conexa; acíclico porque um ciclo passando por v exigiria $grau(v) \geq 2$, e um ciclo que não passa por v estaria inteiro em T, que é acíclica), mas foi dada sem justificativa. É uma lacuna menor; não é ela que invalida a prova.

A base e a hipótese de indução do estudante estão corretas; o erro está só no passo.

Comparação com a Questão 3. As duas são provas defeituosas, mas por motivos diferentes. Na Questão 3 o passo *não vale* num valor específico (*k = 1*), e a proposição é falsa. Aqui o passo vale para cada árvore construída, e a proposição é verdadeira: o defeito é concluir sobre uma classe inteira a partir de uma construção que não se provou cobrir a classe.

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n + 1)/2$
- $\Sigma_{i=1}^{n} (a^{i}) = a(a^{n} - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^{c} = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^{c} < c^{n} < n! < n^{n}$$

onde c é uma constante arbitrária inteira ≥ 2.

Apêndice — o exercício deixado em aberto no slide da semana 5

Contexto — slides de MC458 (UNICAMP), "Indução matemática no projeto de algoritmos", Exemplo 3

Dada uma árvore binária *A* com *n* nós, calcular o fator de balanceamento (f.b.) de cada nó — a altura da subárvore esquerda menos a da direita. Convenção: árvore vazia tem altura *−1* e f.b. zero. A hipótese de indução **fortalecida** ("sabemos calcular f.b. *e alturas* de árvores com menos que *n* nós") leva ao algoritmo:

No slide ("Exemplo 3 - Algoritmo"), transcrição literal:

```
FatorAltura (A)
▷ Entrada: Uma árvore binária A com n ≥ 0 nós
▷ Saída: A árvore A os f.b.s nos seus nós e a altura de A
1.   se n = 0 então h ← −1
2.   senão
3.        seja r a raiz de A
3.        h_e ← FatorAltura(A_e)
4.        h_d ← FatorAltura(A_d)
5.        ▷ armazena o f.b. na raiz em r.f
6.        r.f ← h_e − h_d
7.        h ← max(h_e, h_d) + 1
8.   devolva h
```

O slide numera duas linhas como "3." e termina em 8. **Nesta resolução as linhas são citadas pela numeração corrida abaixo** (1 a 9), que é a mesma do slide até a primeira linha 3 e soma 1 dali em diante:

```
FatorAltura(A) — numeração corrida usada nesta resolução
1. se n = 0 então h ← −1
2. senão
3.      seja r a raiz de A
4.      he ← FatorAltura(Ae)
5.      hd ← FatorAltura(Ad)
6.      ▷ armazena o f.b. na raiz em r.f
7.      r.f ← he − hd
8.      h ← max(he, hd) + 1
9. devolva h
```

Os dois slides seguintes ("Exemplo 3 - Complexidade"), transcrição literal:

"Seja T(n) o número de operações executadas pelo algoritmo para calcular os f.b.s e a altura de uma árvore A de n nós. Então $T(n) = \Theta(1)$, se $n = 0$; $T(n) = T(n_e) + T(n_d) + \Theta(1)$, se $n > 0$, onde n_e, n_d são os números de nós das subárvores esquerda e direita."

"O pior caso da recorrência parece ser quando, ao longo da recursão, um de n_e, n_d é sempre igual a zero (e o outro é sempre igual a $n - 1$). Chega-se então a: $T(n) = T(1) + \Sigma_{i=2}^{n} \Theta(1) = (n + 1)\Theta(1) + n\Theta(1) \in \Theta(n)$."

E então o slide deixa em aberto:

Exercício: "Há algum ganho em complexidade quando ambos n_e e n_d são aproximadamente *n/2* ao longo da recursão?"

Não há ganho algum de complexidade de tempo — e o resultado é mais forte que "mesma classe assintótica": o custo é **idêntico**, não apenas da mesma ordem. O que o balanceamento melhora, e melhora muito, é a **profundidade da pilha**, que cai de $\Theta(n)$ para $\Theta(\lg n)$ — ganho de espaço, não de tempo.

1. Modelo de custo

Duas constantes positivas, fixas e **uniformes sobre todas as chamadas** — é o que os $\Theta(1)$ do slide escondem, e explicitá-las é necessário para poder somar um número *variável* delas:

$a > 0$: custo de uma chamada com A vazia (linha 1 e a devolução)

$b > 0$: custo da parte não recursiva de uma chamada com A não vazia (linha 1 e linhas 3, 7, 8, 9)

A recorrência estrutural exata é $T(A) = a$ se A é vazia, e $T(A) = b + T(A_e) + T(A_d)$ caso contrário.

2. O caso balanceado pelo Teorema Mestre

Com $n_e \approx n_d \approx n/2$ a recorrência vira $T(n) = 2 \cdot T(n/2) + \Theta(1)$. Para não colidir com as constantes de custo a e b, escrevo os parâmetros do teorema como α (número de chamadas recursivas), β (fator de redução) e k, na forma $T(n) = \alpha \cdot T(n/\beta) + c \cdot n^k$, com o critério $\alpha \gtrless \beta^k$.

parâmetro	valor	de onde vem
α	2	linhas 4 e 5: a esquerda *e* a direita são visitadas
β	2	cada subárvore tem $\approx$ n/2 nós
k	0	o trabalho fora da recursão é constante: $c \cdot n^0$

$\beta^k = 2^0 = 1$ e $\alpha = 2$, logo $\alpha > \beta^k$ — o **terceiro** dos três casos, na ordem em que o formulário da prova os imprime. Portanto

$$\mathbf{T(n) = \Theta(n^{\log_\beta \alpha}) = \Theta(n^{\log_2 2}) = \Theta(n)}$$

> Detalhe de rigor: "$T(n) = 2 \cdot T(n/2) + \Theta(1)$" é uma aproximação — as subárvores somam $n - 1$ nós, não n, e o formulário pede termo motor *literalmente* $c \cdot n^k$. A forma exata cabe no formulário. Numa árvore perfeitamente balanceada ($n = 2^{h+1} - 1$), cada filho tem $(n-1)/2$ nós e $T(n) = 2 \cdot T((n-1)/2) + b$, com $T(0) = a$. Trocando $U(m) = T(m - 1)$, fica $U(m) = 2 \cdot U(m/2) + b$ com $U(1) = a$ e $m = n + 1$ potência exata de 2: $\alpha = 2$, $\beta = 2$, $k = 0$, termo motor $b \cdot m^0$ com $b > 0$, e U positiva e crescente. Terceiro caso: $U(m) = \Theta(m)$, isto é, $T(n) = \Theta(n)$. Para formas só aproximadamente balanceadas, quem decide é o teorema da seção 3, que vale para toda árvore.

3. A razão estrutural: a forma da árvore é irrelevante para o custo

As duas contas acima cobrem apenas os dois extremos. O que segue vale para **toda** árvore binária, e é a resposta de verdade.

Teorema. Para toda árvore binária A com $n \geq 0$ nós, $\mathbf{T(A) = (n+1) \cdot a + n \cdot b}$, qualquer que seja a forma de A.

> **Prova, por indução forte em n.**
>
> **Base (n = 0).** A é vazia; por definição $T(A) = a$. A fórmula dá $(0+1)a + 0 \cdot b = a$. ✓
>
> **Passo (n ≥ 1).** Suponha a fórmula válida para toda árvore com estritamente menos que n nós — a hipótese *forte* é necessária, porque não se sabe como os $n-1$ nós se repartem. Seja A com n nós, subárvores A_e com n_e nós e A_d com n_d. Como a raiz não está em nenhuma das duas, $n_e + n_d = n - 1$; e como $n \geq 1$, vale $n_e, n_d \leq n-1 < n$, logo a hipótese se aplica às duas. Então
>
> $$\begin{aligned} T(A) &= b + [(n_e+1)a + n_e b] + [(n_d+1)a + n_d b] \\ &= b + (n_e + n_d + 2) \cdot a + (n_e + n_d) \cdot b \\ &= b + (n+1) \cdot a + (n-1) \cdot b \quad \leftarrow \text{usando } n_e + n_d = n - 1 \\ &= (n+1) \cdot a + n \cdot b. \ \blacksquare \end{aligned}$$

Corolário. Tomando $a = b = 1$, o número total de invocações de `FatorAltura` numa árvore de n nós é **exatamente $2n + 1$**: n chamadas com árvore não vazia (uma por nó) e $n + 1$ com árvore vazia — que são exatamente os $n+1$ ponteiros nulos de uma árvore binária de n nós (contando, em $n = 0$, o ponteiro para a raiz). Casos pequenos: $n = 0 \to 1$; $n = 1 \to 3$ (a raiz e os dois filhos vazios).

Consequência. Escrever "$T(n)$" para um algoritmo cujo custo dependesse da *forma* da entrada só faria sentido como pior caso. Aqui o custo depende **só de n**: $T(n)$ é função genuína de n, e pior caso = melhor caso = caso médio. Não há forma de árvore boa nem ruim.

Testemunhas. Com $T(n) = (a+b)n + a$: $\mathbf{c_1 = a + b}$, $\mathbf{c_2 = 2a + b}$, $\mathbf{n_0 = 1}$. Em $n = n_0 = 1$ exatamente: $T(1) = 2a + b$; a cota inferior dá $a + b \leq 2a + b \iff 0 \leq a$ ✓ (estrito, pois $a > 0$), e a superior $2a + b \geq 2a + b$ ✓ com igualdade. No modelo unitário ($a = b = 1$): $T(n) = 2n + 1$, $c_1 = 2$, $c_2 = 3$, $n_0 = 1$; em $n = 1$: $2 \leq 3 \leq 3$; em $n = 5$: $10 \leq 11 \leq 15$.

4. Por que tinha de ser assim — e o contraste que torna isso não trivial

O algoritmo faz, **por nó**, uma quantidade fixa de trabalho: uma subtração, um `max`, uma soma. Nenhum nó é visitado duas vezes e nenhum é pulado — é um percurso em pós-ordem disfarçado de recorrência. A forma da árvore *redistribui* as n unidades de trabalho entre os níveis, mas não muda quantas são.

O contraste: se o passo de combinação custasse $\Theta(\text{tamanho da subárvore})$ em vez de $\Theta(1)$ — como no MergeSort —, a forma importaria muito: cadeia daria $\Theta(n^2)$ e balanceada $\Theta(n \lg n)$. O que anula o efeito da forma aqui é especificamente o custo de combinação ser *constante*.

Cota inferior do problema. Qualquer algoritmo correto precisa escrever um f.b. em cada um dos n nós, logo executa $\Omega(n)$ operações (testemunhas: ao menos 1 escrita por nó, isto é $c = 1$, $n_0 = 1$). Como `FatorAltura` é $O(n)$, ele é **assintoticamente ótimo**. Nenhum balanceamento baixa a classe: o gargalo é o tamanho da saída, não a estratégia.

5. O que muda e o que não muda

Aspecto	Cadeia	Balanceada	Muda?
Tempo assintótico	$\Theta(n)$	$\Theta(n)$	não
Número exato de chamadas	$2n + 1$	$2n + 1$	não — *idênticos*
Custo exato	$(n+1)a + n{\cdot}b$	$(n+1)a + n{\cdot}b$	não
Profundidade máxima da recursão	$n + 1 = \Theta(n)$	$\lfloor \lg n \rfloor + 2$ no split perfeito; $\Theta(\lg n)$ em qualquer split de fração constante	**sim**
Risco de estouro de pilha	alto para n grande	praticamente nulo	**sim**

Os quadros simultaneamente ativos formam um caminho da raiz até uma chamada vazia. Numa cadeia de n nós são $n + 1$ quadros; numa árvore balanceada de altura $\lfloor \lg n \rfloor$ são $\lfloor \lg n \rfloor + 2$. Conferindo em $n = 1$: cadeia dá $1+1 = 2$, balanceada dá $\lfloor \lg 1 \rfloor + 2 = 2$ — coincidem, como devem, pois com um nó só as duas formas são a mesma árvore.

Para $n = 10^6$ é a diferença entre $10^6 + 1$ quadros (estouro certo na maioria dos ambientes) e $\lfloor \lg 10^6 \rfloor + 2 = 21$ quadros. Não é ganho de complexidade de *tempo*; é ganho de complexidade de *espaço auxiliar*, de $\Theta(n)$ para $\Theta(\lg n)$. Responder "Θ(n) nos dois casos, logo tanto faz" seria errado por omissão.

> **Por que não é como a busca binária.** É tentador esperar ganho logarítmico "porque a árvore está balanceada". A busca binária tem $T(n) = T(n/2) + \Theta(1)$, ou seja $a = 1$, $\beta = 2$, $k = 0$, $\beta^k = 1$; aqui $a = \beta^k$, segundo caso, dando $\Theta(n^k \lg n) = \Theta(\lg n)$. A queda vem de $a = 1$: a busca binária **descarta** uma das metades. `FatorAltura` tem $a = 2$ — desce nas duas e não descarta nada, porque precisa do f.b. de todo mundo. Com combinação de custo constante e as duas partes visitadas ($a = 2$), o equilíbrio não muda o tempo. O que dá à busca binária o ganho logarítmico são *duas coisas juntas*: descartar uma das partes ($a = 1$) *e* ficar, a cada passo, com no máximo uma fração constante $q < 1$ de n — então $T(n) \le T(q{\cdot}n) + \Theta(1)$ e $T(n) = O(\lg n)$. A metade da busca binária é só um caso: ficar com $9/10$ também dá $O(\lg n)$. Descartando um só elemento por vez, $T(n) = T(n-1) + \Theta(1) = \Theta(n)$. E com combinação de custo proporcional ao tamanho, como no contraste da seção 4, a divisão equilibrada ganha mesmo sem descartar nada: $\Theta(n \lg n)$ contra $\Theta(n^2)$.

6. Uma nota sobre a contagem escrita no slide

O **resultado final do slide está exatamente certo**: a expressão "$(n+1)\Theta(1) + n\Theta(1)$" corresponde termo a termo ao que foi provado — $n + 1$ chamadas com árvore vazia e n com árvore não vazia. Não é sequer aproximação: é o custo exato, e vale para qualquer forma, não só para a cadeia.

E o *passo intermediário* é abreviado, não errado — vale desfazer a objeção fácil. O slide escreve $T(n) = T(1) + \sum_{i=2}^{n} \Theta(1)$, e isso de fato liga os extremos: cada $\Theta(1)$ da soma é o incremento de um passo do desenrolar, que já embute o $T(0)$ do filho vazio, de modo que

$$T(1) + (n-1){\cdot}(T(0) + b) = (2a + b) + (n-1)(a + b) = (n+1)a + n{\cdot}b$$

que é exatamente o lado direito que o slide escreve. Objetar que "a soma tem $n-1$ parcelas e o total tem $2n+1$" seria contar parcelas — e $\Theta(1)$ não se compromete com constante nenhuma. O que vale registrar é outra coisa: somar um número *variável* de termos $\Theta(1)$ só é legítimo porque existe uma constante uniforme limitando todos eles — $a + b$, o valor de cada incremento $T(0) + b$ —, e essa uniformidade não está na notação, está no algoritmo. E o caso base declarado é $n = 0$: parar em $T(1)$ é atalho legítimo, desde que se saiba que $T(1) = 2a + b$, e não a.

E o slide diz "o pior caso da recorrência *parece ser* quando um de n_e, n_d é sempre zero". O teorema da seção 3 mostra que, no modelo de contagem de operações, **não há pior caso de forma**: toda árvore com n nós custa exatamente o mesmo. A cadeia é o pior caso para outra grandeza — a profundidade da pilha. O "parece ser" do slide é prudente, e o exercício é justamente o convite a descobrir que a suspeita não se confirma no tempo.

> **Ressalvas declaradas.** A contagem de pilha supõe o modelo usual, um quadro por chamada ativa e sem otimização de chamada em cauda — a chamada da linha 5 não está em posição de cauda (a linha 7 usa `he`), então nenhuma otimização razoável elimina a pilha aqui. E adota-se o modelo de custo uniforme do RAM (CLRS 3ª ed., §2.2): uma subtração, um `max` e uma soma custam $\Theta(1)$ independentemente da magnitude das alturas.

Sobre esta resolução. Fluxo em sete passos, como registrado no `CLAUDE.md` desta pasta: chão determinístico em Python puro antes de qualquer redação; solvers em contexto isolado; limpeza (o raciocínio é descartado, só a prova final segue adiante); verificadores adversariais em contexto limpo, cegos ao raciocínio e uns aos outros, com instrução de *derrubar* a prova e com Bash para conferir as contas; voto assimétrico; correções; e **reverificação da versão corrigida**.
O que foi conferido numericamente. As cinco formas fechadas da Questão 1 por recursão direta, incluindo n = 1, 2, 3; as testemunhas do item (e) em n = 1…200 000. Na Questão 2, as cinco recorrências resolvidas em aritmética inteira exata e comparadas com as formas fechadas; o Lema da expansão, (F1), (F2), a identidade A − B = 1 e os dois Corolários varridos em centenas de triplas (a, b, k) — inclusive o caso a < b^k, que nenhum item exerce; as testemunhas em n = n_0 exatamente, e a refutação explícita de n_0 = 1 nos casos de igualdade (lg 1 = 0). Na Questão 3, enumeração exaustiva de todos os conjuntos rotulados e de todos os pares de subconjuntos, confirmando que a construção falha exatamente em k = 1. No apêndice, o algoritmo foi implementado e medido em cadeia, ziguezague, árvore balanceada e árvores aleatórias.
O que a reverificação pegou. Onze achados na primeira rodada e mais nove na segunda — destes, dois *introduzidos pelas próprias correções* (uma frase sobre reescala de constantes que é falsa para 0 < c < 1, e uma afirmação autocontraditória sobre regras de inferência) e dois que a primeira correção deveria ter removido e não removeu. Todos os patches estão em `geradores/dados/lista04/correcoes-lista04.py`, cada um com o motivo e quem o achou; os scripts de conferência, em `verificacao/lista04-*.py`.
Atualização de 09/09/2026. As Questões 4 e 5 entraram na versão revisada da lista. Textos-fonte verificados: `geradores/dados/lista04/questao4-soma-dos-quadrados-limpo.md` e `questao5-arvores-limpo.md`. Questão 4: 4 de 4 verificadores sem defeito; identidade conferida em n = 0…20 000 e cada linha do passo em aritmética exata. Questão 5: 4 de 4 sem defeito; por sugestão de três verificadores, a frase "o passo vai na direção errada" (que sugeria que construir n → n+1 é sempre inválido) foi reescrita, e a versão nova voltou com 4 de 4. As árvores rotuladas com até 6 vértices (1 296 no último tamanho,

igual a Cayley) foram enumeradas para conferir a contagem de arestas e os Lemas 1 e 2, e o contraexemplo de C(4) foi conferido por força bruta (`verificacao/atualizacao-2026-09-chao-deterministico.py`).

Enunciados. Transcritos literalmente do PDF do professor, com conferência automática contra o original — inclusive onde o original traz erro de digitação ou concordância, marcados com *[sic]* e nunca corrigidos.

Lista de estudo, sem entrega — IFMG Campus Formiga, 2026/2.

Projeto e Análise de Algoritmos — Lista de Exercícios V

Instituto Federal de Minas Gerais, Campus Formiga · Bacharelado em Ciência da Computação · 2026/2 · Lista avaliada (5 pontos), entrega em 24/09/2026
Professor: Mário Luiz Rodrigues Oliveira · Aluna: Luisa Caetano

> **Como ler esta resolução.** Cada questão traz o enunciado literal, a solução em seções e, no fim, um quadro *Resposta*. Toda afirmação de Θ vem com c_1, c_2, n_0 numéricos, conferidos em $n = n_0$ exatamente. Ao citar o Teorema Mestre, "primeira/segunda/terceira linha" segue a ordem impressa no formulário da lista: $a < b^k$, $a = b^k$, $a > b^k$.

Questão 1 — máximo por divisão e conquista

> Enunciado
>
> "Considere o problema de encontrar o valor máximo em um vetor contendo n números reais. Projete um algoritmo usando o paradigma divisão e conquista para esse problema. Considere que n é um número par. Projete duas abordagens, conforme indicado:
> (a) (0.5 ponto) Projete um algoritmo para resolver o problema considerando um subproblema de tamanho n − 1.
> (b) (0.5 ponto) Projete um algoritmo para resolver o problema dividindo-o em dois subproblemas de tamanho n/2.
> (c) (0.5 ponto) Qual abordagem produz um algoritmo assintoticamente mais eficiente no pior caso? Justifique sua resposta. Respostas sem justificativas não serão consideradas."
>
> Formulário fornecido pela lista (Teorema Mestre, forma de Brassard & Bratley): sejam as constantes naturais a ≥ 1, b ≥ 2 e k ≥ 0; f(n) = c·n^k com c > 0 real; T(n) = aT(n/b) + f(n); n potência exata de b; T leva naturais em reais positivos e é assintoticamente não decrescente. Então T(n) ∈ θ(n^k) se a < b^k; θ(n^k·lg n) se a = b^k; θ(n^{log_b a}) se a > b^k.

0. Convenções

Vetor A[1..n] de reais, n ≥ 1. Custo medido em operações primitivas do modelo RAM (CLRS 3ª ed., §2.2): cada linha executada dos pseudocódigos abaixo, fora o tempo gasto dentro das chamadas recursivas, custa uma constante. Contam-se também, à parte, as **comparações entre elementos** do vetor, que é a medida usual para este problema.

Custo de uma chamada, fora as chamadas recursivas que ela faz. Cada algoritmo tem **as suas próprias** constantes — índice a para MaxA, índice b para MaxB; as de um não têm relação com as do outro:

– com tamanho 1 (caso base): uma constante $d_a > 0$ (MaxA) ou $d_b > 0$ (MaxB);
– com tamanho ≥ 2: depende do ramo tomado na combinação (o ramo "senão" executa uma linha a mais), mas fica sempre entre duas constantes, $0 < c'_a \le c''_a$ em MaxA e $0 < c'_b \le c''_b$ em MaxB, onde c″ é o custo quando o ramo "senão" é tomado.

Sobre "n é par": os dois algoritmos são escritos para **todo** n ≥ 1, e precisam ser. Em (a) o subproblema tem tamanho n − 1, que é ímpar; em (b), n par garante apenas que a **primeira** divisão é exatamente n/2 + n/2 — já com n = 6 os subproblemas têm tamanho 3, que se dividem em 2 + 1. Um algoritmo que só funcionasse para n par não se aplicaria aos próprios subproblemas.

(a) Um subproblema de tamanho n − 1

```
MaxA(A, n)                  ▷ devolve max{A[1], …, A[n]},  n ≥ 1
1  se n = 1 então devolva A[1]
2  m ← MaxA(A, n − 1)       ▷ conquista: máximo dos n − 1 primeiros
3  se m ≥ A[n] então devolva m
4  senão devolva A[n]       ▷ combina: compara com o último
```

Chamada inicial: MaxA(A, n).

Correção. P(n): "MaxA(A, n) devolve max{A[1..n]}", para todo n ≥ 1.
Base n = 1: a linha 1 devolve A[1] = max{A[1]}.
H.i.: seja n ≥ 2 e suponha P(n − 1) (note que n − 1 ≥ 1, então P(n − 1) está no domínio). Passo: pela h.i., a linha 2 obtém m = max{A[1..n−1]}; as linhas 3–4 devolvem o maior entre m e A[n], e max{A[1..n]} = max(max{A[1..n−1]}, A[n]). Logo P(n). ∎

Custo. Uma execução de MaxA(A, n) faz n chamadas no total (a inicial e n − 1 recursivas, com tamanhos n, n − 1, …, 1): uma com tamanho 1, de custo d_a, e n − 1 com tamanho ≥ 2, cada uma de custo entre c'_a e c''_a. Somando chamada a chamada, **toda** entrada de tamanho n custa entre $d_a + c'_a(n-1)$ e $d_a + c''_a(n-1)$.
(Com um custo constante c_a por chamada de tamanho ≥ 2 — $c_a = c''_a$ no pior caso —, isso é a recorrência $T_a(1) = d_a$, $T_a(n) = T_a(n-1) + c_a$ para n ≥ 2, cuja forma fechada é $T_a(n) = d_a + c_a(n-1)$: n = 1 dá d_a ✓; se vale para n − 1, $T_a(n) = d_a + c_a(n-2) + c_a = d_a + c_a(n-1)$ ✓.)
Pior caso. O vetor estritamente crescente toma o ramo "senão" (linha 4) em todas as n − 1 combinações, pois ali A[n] é maior que todos os anteriores; seu custo é $d_a + c''_a(n-1)$, que é também a cota superior de qualquer entrada. Portanto o custo de pior caso é exatamente

$$T_a^{\text{pior}}(n) = d_a + c''_a \cdot (n-1).$$

$\Theta(n)$ com $c_1 = \min(c''_a, d_a)$, $c_2 = c''_a + d_a$, $n_0 = 1$:
– cota inferior: $d_a + c''_a(n-1) \ge \min(c''_a,d_a)\cdot 1 + \min(c''_a,d_a)\cdot(n-1) = \min(c''_a,d_a)\cdot n$;
– cota superior: $d_a + c''_a(n-1) \le d_a \cdot n + c''_a \cdot n = (c''_a + d_a)\cdot n$, pois n ≥ 1. Em n = n_0 = 1: $\min(c''_a,d_a) \le d_a \le c''_a + d_a$ ✓. (As mesmas contas com c'_a mostram que até o melhor caso é Θ(n).)

(Este algoritmo não é da forma aT(n/b) + f(n): o Teorema Mestre não se aplica; resolve-se por expansão, como acima.)
Comparações entre elementos: $C_a(1) = 0$ e $C_a(n) = C_a(n-1) + 1$, logo $\mathbf{C_a(n) = n - 1}$ — esta contagem não depende dos valores do vetor.
Chamadas (contando a inicial): n. Profundidade da pilha: n.

(b) Dois subproblemas de tamanho n/2

```
MaxB(A, i, j)                  ▷ devolve max{A[i], …, A[j]},  i ≤ j
1  se i = j então devolva A[i]
2  q ← ⌊(i + j)/2⌋             ▷ divide ao meio
3  esq ← MaxB(A, i, q)         ▷ conquista a metade esquerda
4  dir ← MaxB(A, q + 1, j)     ▷ conquista a metade direita
5  se esq ≥ dir então devolva esq
6  senão devolva dir           ▷ combina
```

Chamada inicial: MaxB(A, 1, n).

Tamanhos. Seja $s = j - i + 1 \geq 2$. A metade esquerda tem $q - i + 1 = \lfloor (j-i)/2 \rfloor + 1 = \lfloor (s-1)/2 \rfloor + 1 = \lceil s/2 \rceil$ elementos; a direita tem $s - \lceil s/2 \rceil = \lfloor s/2 \rfloor$. Para $s \geq 2$ os dois são ≥ 1 e $< s$, então a recursão termina. Com $s = n$ par, os dois têm exatamente $n/2$, como pede o enunciado.

Correção. Indução forte em s. P(s): "MaxB devolve o máximo de qualquer trecho de s elementos", para todo $s \geq 1$. Base $s = 1$: linha 1. H.i.: seja $s \geq 2$ e suponha P(t) para todo $1 \leq t < s$. Passo: os tamanhos $\lceil s/2 \rceil$ e $\lfloor s/2 \rfloor$ estão em $[1, s-1]$, então pela h.i. esq = máximo da metade esquerda e dir = máximo da metade direita; o máximo do trecho é o maior dos dois, que as linhas 5–6 devolvem. Logo P(s). ∎

Custo. Primeiro, um fato sobre a recorrência
$R(1) = u$, $R(s) = R(\lceil s/2 \rceil) + R(\lfloor s/2 \rfloor) + v$ para $s \geq 2$,
com u, v reais quaisquer: sua solução é **exata para todo $s \geq 1$** (não é preciso supor potência de 2):
$R(s) = u \cdot s + v \cdot (s-1)$.
Prova por indução forte: $s = 1$ dá u ✓; para $s \geq 2$, pela h.i.,
$R(s) = u\lceil s/2 \rceil + v(\lceil s/2 \rceil - 1) + u\lfloor s/2 \rfloor + v(\lfloor s/2 \rfloor - 1) + v = u \cdot s + v(s-2) + v = u \cdot s + v(s-1)$ ✓, usando $\lceil s/2 \rceil + \lfloor s/2 \rfloor = s$ (a indução não usa o sinal de u nem de v).
A árvore de chamadas de MaxB depende só de s, não dos valores do vetor. O número de chamadas de tamanho 1 satisfaz R com (u, v) = (1, 0), e o de chamadas de tamanho ≥ 2 satisfaz R com (u, v) = (0, 1): são s e $s - 1$ ($2s - 1$ no total). Somando chamada a chamada, toda entrada de tamanho n custa entre $d_b \cdot n + c'_b(n-1)$ e $d_b \cdot n + c''_b(n-1)$. Com um custo constante c_b por chamada de tamanho ≥ 2 ($c_b = c''_b$ no pior caso), o custo é $T_b(1) = d_b$,
$T_b(s) = T_b(\lceil s/2 \rceil) + T_b(\lfloor s/2 \rfloor) + c_b$, isto é, R com $(u, v) = (d_b, c_b)$:
$T_b(s) = d_b \cdot s + c_b \cdot (s-1)$.
Pior caso. No vetor estritamente crescente, em toda combinação o máximo da metade direita é maior que o da esquerda, e o ramo "senão" (linha 6) é tomado em todas as $n - 1$ combinações; o custo é $d_b \cdot n + c''_b(n-1)$, a cota superior. Portanto
$T_b^{\text{pior}}(n) = d_b \cdot n + c''_b \cdot (n-1)$.
$\Theta(n)$ com $c_1 = d_b$, $c_2 = c''_b + d_b$, no = 1: $d_b \cdot n \leq d_b \cdot n + c''_b(n-1) \leq (c''_b + d_b) \cdot n$ para $n \geq 1$. Em n = 1: $d_b \leq d_b \leq c''_b + d_b$ ✓.

Confirmação pelo Teorema Mestre (n potência exata de 2, onde a recorrência vira $T_b(n) = 2T_b(n/2) + c_b = 2T_b(n/2) + c_b \cdot n^0$; aqui a, b, k são os parâmetros do formulário, e c_b faz o papel do seu c): $a = 2$, $b = 2$, $k = 0$, $c_b > 0$; $b^k = 2^0 = 1 < 2 = a$, isto é, $a > b^k$ — **terceira linha** do formulário: $T_b(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$. As hipóteses sobre T valem: $T_b(s) = (c_b + d_b)s - c_b \geq d_b > 0$ é positiva e crescente em s. Para n par que não é potência de 2 vale a forma fechada exata acima, que dá a mesma classe.
Comparações entre elementos: $C_b(1) = 0$, $C_b(s) = C_b(\lceil s/2 \rceil) + C_b(\lfloor s/2 \rfloor) + 1$, logo $\mathbf{C_b(n) = n - 1}$ (R com (u, v) = (0, 1)) — também independente dos valores.
Chamadas (contando a inicial): $2n - 1$. Profundidade da pilha: $1 + \lceil \lg n \rceil$.

(c) Qual é assintoticamente mais eficiente no pior caso?

Nenhuma das duas. Ambas levam tempo **$\Theta(n)$** no pior caso:
$T_a^{\text{pior}}(n) = d_a + c''_a(n-1) \in \Theta(n)$ e $T_b^{\text{pior}}(n) = d_b \cdot n + c''_b(n-1) \in \Theta(n)$, com as testemunhas exibidas em (a) e (b). Como as duas estão em $\Theta(n)$, vale $T_a^{\text{pior}} \in \Theta(T_b^{\text{pior}})$: nenhuma é assintoticamente melhor. (As constantes de uma e de outra não são comparáveis entre si — cada uma depende das linhas do seu pseudocódigo —, e a pergunta é sobre a classe.) O desvio que depende dos dados (linhas 3–4 em MaxA, 5–6 em MaxB) só altera o custo de cada chamada dentro de uma faixa constante; por isso até o melhor caso de cada uma é $\Theta(n)$ — a classe não depende da entrada, só as constantes.

Mais do que isso: as duas fazem **exatamente o mesmo número de comparações entre elementos, $n - 1$**, qualquer que seja a entrada. A divisão ao meio não reduz trabalho — ela só organiza o mesmo número de comparações numa árvore balanceada em vez de numa cadeia. Em chamadas (contando a inicial), (b) até faz mais: $2n - 1$ contra n — diferença de constante, não de classe.

Por que dividir ao meio não dá lg n aqui. A intuição da busca binária engana: lá há **um** subproblema de tamanho n/2 ($a = 1$, $T(n) = T(n/2) + \Theta(1) \in \Theta(\lg n)$); aqui há **dois** ($a = 2$). Para n potência de 2, o nível i ($0 \leq i < \lg n$) da árvore de recursão de MaxB tem 2^i chamadas internas, cada uma com uma combinação de custo constante, e o nível lg n tem as n folhas, que não combinam: o número de combinações dobra de um nível para o seguinte enquanto o tamanho cai pela metade, e a soma de 2^i para $i = 0, \ldots, \lg n - 1$ dá $n - 1$ (soma vazia se $n = 1$). Para n qualquer, a contagem $n - 1$ vem da recorrência de C_b acima.

Nenhuma abordagem pode ser melhor que $\Theta(n)$. Considere qualquer algoritmo determinístico correto que acessa o vetor só lendo posições, ao custo de ao menos uma operação por leitura. Ele precisa ler as n posições: se numa entrada ele nunca lê A[p], troque A[p] por um valor maior que o valor devolvido e que todos os demais elementos; a execução não muda (A[p] não é lido), a saída não muda, e agora está errada. Logo todo algoritmo gasta ao menos $1 \cdot n$ operações para todo $n \geq 1$, isto é, $\Omega(n)$, e as duas abordagens são ótimas.

A diferença que existe é de espaço, não de tempo: a pilha de recursão tem profundidade n em (a) e $1 + \lceil \lg n \rceil$ em (b). Com A passado por referência e cada quadro de pilha de tamanho constante, isso é $\Theta(n)$ contra $\Theta(\lg n)$ de memória auxiliar; para o segundo,
$\lg n \leq 1 + \lceil \lg n \rceil \leq 2 \lg n$ para todo $n \geq 2$ ($c_1 = 1$, $c_2 = 2$, $n_0 = 2$): a cota inferior vale porque $\lceil \lg n \rceil \geq \lg n$; a superior, para $n \geq 4$, porque $1 + \lceil \lg n \rceil < 2 + \lg n \leq 2 \lg n$, e em $n = 2$ e $n = 3$ por conta direta ($2 \leq 2$; $3 \leq 2 \lg 3 \approx 3{,}17$). $n_0 = 1$ não serve, pois $\lg 1 = 0$ e a profundidade em $n = 1$ é 1.

Resposta

(a) MaxA: resolve A[1..n−1] recursivamente e compara o resultado com A[n].
Recorrência $T_a(1) = d_a$, $T_a(n) = T_a(n-1) + c_a$; pior caso (vetor estritamente crescente, $c_a = c''_a$): $T_a^{\text{pior}}(n) = d_a + c''_a(n-1) \in \Theta(n)$; $n - 1$ comparações em toda entrada.

(b) MaxB: divide em duas metades ($\lceil n/2 \rceil$ e $\lfloor n/2 \rfloor$; exatamente $n/2$ e $n/2$ na primeira divisão, pois n é par), resolve cada uma e devolve o maior dos dois resultados.
$T_b(n) = 2T_b(n/2) + c_b \Rightarrow \Theta(n)$ pelo Teorema Mestre (n potência de 2; $a = 2 > b^k = 1$, terceira linha); para todo n, pior caso exatamente $T_b^{\text{pior}}(n) = d_b \cdot n + c''_b(n-1) \in \Theta(n)$; $n - 1$ comparações em toda entrada.

(c) Nenhuma é assintoticamente mais eficiente: ambas são $\Theta(n)$ no pior caso (e mesmo no melhor), com exatamente $n - 1$ comparações; e $\Theta(n)$ é ótimo, pois todo algoritmo precisa ler as n posições. A única diferença assintótica é a profundidade da pilha: $\Theta(n)$ em (a), $\Theta(\lg n)$ em (b).

Questão 2 — busca num vetor sem fim

> Enunciado
>
> "Resolva o seguinte problema:
> a) (0.5 ponto) Dado um vetor A [.] de tamanho infinito no qual as primeiras m posições contêm números inteiros em ordem crescente e as demais posições são preenchidas com o valor ∞. Não se conhece o valor de m. Descreva um algoritmo que recebe um valor inteiro x como entrada e encontra uma posição no vetor contendo o valor x, se tal posição existir, em tempo o(n). (Se você não se sente confortável com o fato do vetor ter comprimento infinito, assuma em vez disso comprimento de tamanho n, contudo você não conhece esse comprimento. Assuma, também, que a implementação da estrutura de dados vetor em sua linguagem de programação preferida retorna a mensagem de erro ∞ sempre que elementos A[i] com i > n são acessados).
> b) (0.5 ponto) Argumente que seu algoritmo tem a complexidade solicitada."

0. Modelo

Posições 1, 2, 3, …; A[1..m] são inteiros em ordem crescente (o argumento abaixo vale também para ordem não decrescente, com repetições); A[i] = ∞ para i > m; $m \geq 0$ é desconhecido. O valor ∞ é tratado como maior que todo inteiro, então a sequência inteira A[1], A[2], … é não decrescente:

(Monotonia) $i < i' \Rightarrow A[i] \leq A[i']$.

Na versão de comprimento finito n (desconhecido), as posições $m < i \leq n$ contêm ∞ e o acesso a $i > n$ devolve ∞; o algoritmo enxerga exatamente a mesma sequência que na versão infinita, e $m \leq n$.
Custo: cada acesso A[i] e cada operação aritmética ou comparação custa $\Theta(1)$ (modelo RAM); conta-se o número de acessos ao vetor, e o tempo é no máximo uma constante vezes esse número.

Tamanho da entrada. Na versão infinita o único parâmetro é m, e o(n) se lê com $n = m$; na versão finita, n é o comprimento e $m \leq n$. Define-se

$f(m) = 2\lfloor \lg m \rfloor + 3$ para $m \geq 1$, $f(0) = 2$.

f é não decrescente em $m \geq 0$ ($f(0) = 2 < 3 = f(1)$, e $\lfloor \lg m \rfloor$ não decresce), de modo que $f(m) \leq f(n)$ em qualquer das leituras, sempre que $m \leq n$.

(a) O algoritmo: galope seguido de busca binária

```
BuscaIlimitada(A, x)
 1  i ← 1
 2  enquanto A[i] < x faça        ▷ fase 1 (galope): testa as posições 1, 2, 4, 8, …
 3      i ← 2·i
 4  lo ← ⌊i/2⌋ + 1;  hi ← i        ▷ se i = 1, então lo = hi = 1
 5  enquanto lo ≤ hi faça          ▷ fase 2: busca binária em A[lo..hi]
 6      meio ← ⌊(lo + hi)/2⌋;  v ← A[meio]
 7      se v = x então devolva meio
 8      se v < x então lo ← meio + 1
 9      senão hi ← meio − 1
10  devolva "x não está no vetor"
```

A ideia: não se conhece o fim da parte útil, então se procura, dobrando o índice, a primeira potência de 2 cujo conteúdo já é ≥ x. Isso delimita um trecho onde x tem de estar, e nele se faz a busca binária usual. Nenhum passo usa m ou n. Na fase 2 cada volta lê o vetor **uma** vez (linha 6) e guarda o valor em v.

(b) Correção e complexidade

Lema 1 (a fase 1 termina, e quantas voltas dá). Seja J o número de vezes que o corpo da linha 3 executa. Então $i = 2^J$ ao fim da fase 1, e

$J = 0$ se $m = 0$; $J \le \lfloor \lg m \rfloor + 1$ se $m \ge 1$.

Prova. O teste A[i] < x só é verdadeiro se A[i] é finito, isto é, $i \le m$ (pois $\infty < x$ é falso). Logo, cada vez que o corpo executa, o valor corrente $i = 2^t$ satisfaz $2^t \le m$, com t = 0, 1, 2, … em sequência. Se m = 0, A[1] = ∞ e o laço não executa: J = 0. Se m ≥ 1, os valores de t com $2^t \le m$ são 0, 1, …, $\lfloor \lg m \rfloor$ — no máximo $\lfloor \lg m \rfloor + 1$ deles —, e portanto o corpo executa no máximo $\lfloor \lg m \rfloor + 1$ vezes; em particular a fase 1 sempre termina. ∎

Lema 2 (invariante ao fim da fase 1). Ao fim da fase 1, A[i] ≥ x (o laço parou) e, se J ≥ 1, A[i/2] < x (o teste anterior foi verdadeiro, na posição $i/2 = 2^{J-1}$).

Lema 3 (x, se existe, está em A[lo..hi]). Se existe p com A[p] = x, então existe uma tal posição em [lo, hi].

Prova. Se A[i] = x, a própria posição i = hi serve. Se A[i] > x, seja p qualquer posição com A[p] = x. Para p ≥ i, a Monotonia daria A[p] ≥ A[i] > x; logo p < i = hi. Se J ≥ 1, para p ≤ i/2 a Monotonia daria A[p] ≤ A[i/2] < x; logo p > i/2, isto é, $p \ge \lfloor i/2 \rfloor + 1 =$ lo. Se J = 0, lo = 1 ≤ p trivialmente. ∎

Lema 4 (busca binária). Invariante do laço 5–9: "toda posição de [loo, hio] com valor x está em [lo, hi]", onde [loo, hio] é o trecho inicial. Vale no início. Se v = A[meio] < x, toda posição p ≤ meio tem A[p] ≤ A[meio] < x, então descartar [lo, meio] preserva o invariante; se v > x, toda posição p ≥ meio tem A[p] > x, e descartar [meio, hi] o preserva. Logo, se o laço termina sem devolver, [lo, hi] ficou vazio e x não ocorre em [loo, hio] — e, pelo Lema 3, não ocorre no vetor. Reciprocamente, o algoritmo só devolve meio quando A[meio] = x. As posições com ∞ dentro do trecho não atrapalham: ∞ > x, e o trecho continua ordenado.

Número de voltas: se o trecho tem L ≥ 1 posições, após uma volta sem retorno ele tem no máximo $\lfloor L/2 \rfloor$ (a esquerda [lo, meio − 1] tem $\lfloor (L-1)/2 \rfloor \le \lfloor L/2 \rfloor$ posições; a direita [meio + 1, hi] tem $\lceil (L-1)/2 \rceil = \lfloor L/2 \rfloor$). Depois de r voltas, no máximo $\lfloor L/2^r \rfloor$; como uma volta só começa com trecho não vazio, r voltas executadas implicam $L/2^{r-1} \ge 1$, isto é, $r \le \lfloor \lg L \rfloor + 1$. ∎

Correção. Pelos Lemas 3 e 4, o algoritmo devolve uma posição p com A[p] = x quando x ocorre no vetor, e responde que x não está quando não ocorre. ∎

Contagem de acessos — cota superior. Fase 1: J + 1 acessos (J testes verdadeiros e o falso que encerra). Fase 2: o trecho tem L = hi − lo + 1 = i − $\lfloor i/2 \rfloor$ posições, isto é, $L = 2^{J-1}$ se J ≥ 1 e L = 1 se J = 0; pelo Lema 4, no máximo $\lfloor \lg L \rfloor + 1$ voltas, com **uma** leitura por volta (linha 6): no máximo J leituras se J ≥ 1, e 1 leitura se J = 0. Total:

$J = 0$: no máximo $1 + 1 = 2$ acessos;

$J \ge 1$: no máximo $(J + 1) + J = 2J + 1 \le 2(\lfloor \lg m \rfloor + 1) + 1 = 2\lfloor \lg m \rfloor + 3$ (Lema 1). Como $J \ge 1$ exige $m \ge 1$, conclui-se: **para todo m ≥ 0, no máximo f(m) acessos** ($f(m) = 2\lfloor \lg m \rfloor + 3$ para m ≥ 1, e f(0) = 2). O tempo total é no máximo K·f(m) para uma constante K > 0 (cada volta de cada laço faz Θ(1) operações além do acesso), isto é, O(lg m) para m ≥ 2. A cota é atingida quando $m = 2^t - 1$ e x > A[m] (conferido por força bruta para m ≤ 699: atingida em m = 1, 3, 7, 15, …, 511).

Contagem de acessos — cota inferior (pior caso). Seja m ≥ 1 e tome x = A[m] + 1 (inteiro, maior que todo elemento finito). Então $A[2^t] < x$ para toda posição $2^t \le m$, e o teste da linha 2 é verdadeiro exatamente para t = 0, 1, …, $\lfloor \lg m \rfloor$: $J = \lfloor \lg m \rfloor + 1$, e só a fase 1 já faz $J + 1 = \lfloor \lg m \rfloor + 2$ acessos. Logo o pior caso W(m) (máximo de acessos sobre as entradas com esse m) satisfaz

$\lfloor \lg m \rfloor + 2 \le W(m) \le 2\lfloor \lg m \rfloor + 3$ para todo m ≥ 1.

Testemunhas de $W \in \Theta(\lg m)$: $c_1 = 1$, $c_2 = 5$, $m_0 = 2$. A cota inferior vale porque $\lfloor \lg m \rfloor + 2 > \lg m$; a superior, porque $2\lfloor \lg m \rfloor + 3 \le 2 \lg m + 3 \lg m = 5 \lg m$ quando $\lg m \ge 1$, isto é, m ≥ 2. Em $m = m_0 = 2$: lg 2 = 1, e $1 \le W(2) \le 5$ (de fato W(2) = 4). $m_0 = 1$ não serve: lg 1 = 0.

Por que isso é o(n). Pela definição (CLRS 3ª ed., §3.1), $g(n) \in o(n)$ se para **toda** constante c > 0 existe $n_0 > 0$ tal que $0 \le g(n) < c \cdot n$ para todo $n \ge n_0$.

Lema 5. $2^j \ge j^2$ para todo inteiro j ≥ 4.

Base j = 4: 16 ≥ 16. Passo: se $2^j \ge j^2$ com j ≥ 4, então $2^{j+1} = 2 \cdot 2^j \ge 2j^2$, e

$2j^2 - (j+1)^2 = j^2 - 2j - 1 = (j-1)^2 - 2 \ge 9 - 2 > 0$; logo $2^{j+1} \ge (j+1)^2$. ∎

(Falha em j = 3: 8 < 9 — por isso a restrição.)

Consequência: para n ≥ 16, $j = \lfloor \lg n \rfloor \ge 4$ e $n \ge 2^j \ge j^2$, donde $\lfloor \lg n \rfloor \le \sqrt{n}$. Então, para n ≥ 16,

$f(n) = 2\lfloor \lg n \rfloor + 3 \le 2\sqrt{n} + 3\sqrt{n} = 5\sqrt{n}$ (pois $\sqrt{n} \ge 1$).
Dado $c > 0$, tome $n_0 = \max(16, \lfloor 25/c^2 \rfloor + 1)$. Para $n \ge n_0$ vale $n > 25/c^2$, isto é, $\sqrt{n} > 5/c$; multiplicando por $c\cdot\sqrt{n} > 0$: $c\cdot n > 5\sqrt{n} \ge f(n)$. Então $0 \le f(n) < c\cdot n$ para todo $n \ge n_0$, e $f \in o(n)$. Para o tempo $K\cdot f(n)$, repita com c/K no lugar de c: $n_0 = \max(16, \lfloor 25K^2/c^2 \rfloor + 1)$. Exemplo em $n = n_0$: $c = 1$ dá $n_0 = 26$, $f(26) = 11 < 26$; $c = 1/10$ dá $n_0 = 2501$, $f(2501) = 25 < 250{,}1$.
Na versão finita ($n \ge 1$), o tempo é $\le K\cdot f(m) \le K\cdot f(n)$ (f não decrescente, $m \le n$), portanto também $o(n)$.

Por que as alternativas óbvias não servem. A busca linear lê A[1], A[2], … até achar x ou um valor > x; com $x > A[m]$ ela lê $m + 1$ posições — $\Theta(m)$ no pior caso, que **não** é $o(n)$ (na leitura $n = m$, ou na versão finita com $m = n$). A busca binária direta precisa do extremo direito do trecho, que é justamente o que não se conhece. O galope descobre um extremo válido: $h_i = 2^J$, com $h_i \le 2m$ quando $J \ge 1$ (pois $2^{J-1} \le m$) e $h_i = 1$ quando $J = 0$ — em no máximo $\lfloor \lg m \rfloor + 2$ acessos se $m \ge 1$, e 1 acesso se $m = 0$.

Resposta

(a) Galope + busca binária: teste A[1], A[2], A[4], …, $A[2^t]$, … até achar a primeira posição $i = 2^J$ com $A[i] \ge x$ (as posições com ∞ garantem a parada); então faça busca binária em $A[\lfloor i/2 \rfloor + 1 \,..\, i]$, lendo A[meio] uma vez por volta. Devolve uma posição com valor x, ou "não está".

(b) A fase 1 dá $J \le \lfloor \lg m \rfloor + 1$ dobras (só se dobra enquanto $A[i]$ é finito, isto é, $i \le m$), e a fase 2 busca num trecho de 2^{J-1} posições (1 posição se $J = 0$), com no máximo J leituras (1 se $J = 0$). Total: no máximo $2\lfloor \lg m \rfloor + 3$ acessos para $m \ge 1$ (2 para $m = 0$); como x maior que todos força $\lfloor \lg m \rfloor + 2$ acessos, o pior caso é $\Theta(\lg m)$ ($c_1 = 1$, $c_2 = 5$, $m_0 = 2$), com $m \le n$. E $2\lfloor \lg n \rfloor + 3 < c\cdot n$ para $n \ge \max(16, \lfloor 25/c^2 \rfloor + 1)$, qualquer que seja $c > 0$ — logo o tempo é $o(n)$.

Questão 3 — qual dos algoritmos A, B, C

Enunciado

"Suponha que, para um problema com entradas de tamanho n, você tenha que escolher um dentre três algoritmos A, B e C. Considere que trata-se do pior caso de cada algoritmo.
1. O algoritmo A resolve problemas dividindo-os em cinco subproblemas de tamanho n/2, recursivamente resolve cada subproblema e então combina as soluções em tempo linear. 2. O algoritmo B resolve problemas de tamanho n, recursivamente resolvendo dois subproblema [sic] de tamanho $n - 1$ e então combina as soluções em tempo constante.
3. O algoritmo C resolve problemas dividindo-os em nove subproblemas de tamanho n/3, recursivamente resolve cada subproblema e então combina as soluções em tempo $\theta(n^2)$. Indique qual dos algoritmos é assintoticamente mais eficiente no pior caso. Expresse sua resposta em termos da notação θ. Justifique sua resposta. Resposta sem justificativa não será considerada."

Formulário fornecido pela lista (Teorema Mestre, forma de Brassard & Bratley): sejam as constantes naturais $a \ge 1$, $b \ge 2$ e $k \ge 0$; $f(n) = c\cdot n^k$ com $c > 0$ real; $T(n) = aT(n/b) + f(n)$; n potência exata de b; T leva naturais em reais positivos e é assintoticamente não decrescente. Então $T(n) \in \theta(n^k)$ se $a < b^k$; $\theta(n^k \cdot \lg n)$ se $a = b^k$; $\theta(n^{\log_b a})$ se $a > b^k$. A lista também fornece a hierarquia $1 < \log n < \sqrt{n} < n < n\cdot\log n < n^c < c^n < n! < n^n$, "onde c é uma constante arbitrária inteira ≥ 2". Ela não cobre diretamente as comparações desta questão ($\log_2 5$ não é inteiro, e em $n^c < c^n$ a base é o mesmo c do expoente); por isso a §5 as prova diretamente.

0. As três recorrências

Casos base: $T(1) = t > 0$ (qualquer constante positiva). O custo de combinar é positivo:
A: $T_A(n) = 5\cdot T_A(n/2) + f_A(n)$, $f_A(n) \in \Theta(n)$ ("tempo linear")
B: $T_B(n) = 2\cdot T_B(n - 1) + f_B(n)$, $f_B(n) \in \Theta(1)$ ("tempo constante")
C: $T_C(n) = 9\cdot T_C(n/3) + f_C(n)$, $f_C(n) \in \Theta(n^2)$
Primeiro resolve-se cada uma com f exatamente da forma $c\cdot n^k$, $c > 0$ (a que o teorema exige); depois, na §4, mostra-se que trocar "$= c\cdot n^k$" por "$\in \Theta(n^k)$" não muda a classe.

Hipótese de monotonia (a última do formulário, "T assintoticamente não decrescente"), para quaisquer $t, c > 0$. Nas potências de b, onde a recorrência define T: $T(b\cdot n) = a\cdot T(n) + f(b\cdot n) > T(n)$, pois $a \ge 1$, $T(n) > 0$ e $f(b\cdot n) > 0$; e $T(1) = t > 0$, então por indução T é positiva e crescente sobre as potências de b. Para B, do mesmo modo, $T_B(n) = 2T_B(n - 1) + c > T_B(n - 1) > 0$. Que o custo não diminua com n a partir de algum N (isto é, $T(n) \le T(n + 1)$ para todo $n \ge N$) é o que "T assintoticamente não decrescente" significa; o formulário o assume, é natural para tempo de pior caso, e é usado na Observação da §5.

1. Algoritmo A — Teorema Mestre, terceira linha

$T_A(n) = 5T_A(n/2) + c\cdot n$: $a = 5$, $b = 2$, $k = 1$, $c > 0$. $b^k = 2^1 = 2$ e $a = 5 > 2$: **terceira linha** do formulário ($a > b^k$). Logo
$T_A(n) \in \Theta(n^{\log_2 5})$, com $\log_2 5 \approx 2{,}3219$.
Forma fechada ($n = 2^m$, $T(1) = t$): $T_A(n) = (t + 2c/3)\cdot n^{\log_2 5} - (2c/3)\cdot n$.
Checagem com $t = c = 1$: $T_A(n) = (5/3)\cdot n^{\log_2 5} - (2/3)\cdot n$; $T_A(2) = 5\cdot 1 + 2 = 7$ e $(5/3)\cdot 5 - (2/3)\cdot 2 = 25/3 - 4/3 = 7$ ✓.
Testemunhas (t, c quaisquer): $c_1 = t$, $c_2 = t + 2c/3$, $n_0 = 1$. A cota superior é imediata; a inferior, $(t + 2c/3)n^{\log_2 5} - (2c/3)n \ge t\cdot n^{\log_2 5}$, equivale a $(2c/3)(n^{\log_2 5} - n) \ge 0$, verdadeira para $n \ge 1$ porque $\log_2 5 > 1$. Com $t = c = 1$: $c_1 = 1$, $c_2 = 5/3$, e em $n = n_0 = 1$: $1 \le 1 \le 5/3$ ✓.

2. Algoritmo B — fora do Teorema Mestre

A chamada é T(n − 1), não T(n/b): subtrair 1 não é dividir por uma constante b (n − 1 = n/b exigiria b = n/(n − 1), que varia com n). O teorema não se aplica; resolve-se por expansão. Com $T_B(1) = t$ e custo de combinação constante $f_B(n) = c > 0$:

$T_B(n) = 2^{n-1}\cdot t + c\cdot(2^{n-1} - 1) = 2^{n-1}\cdot(t + c) - c$ para $n \geq 1$.

Prova por indução: n = 1 dá t + c − c = t ✓; se vale para n − 1 (n ≥ 2),
$2\cdot[2^{n-2}(t + c) - c] + c = 2^{n-1}(t + c) - c$ ✓.

Testemunhas: $(t/2)\cdot 2^n = 2^{n-1}\cdot t \leq T_B(n) \leq 2^{n-1}(t + c) = ((t + c)/2)\cdot 2^n$ para n ≥ 1, isto é $c_1 = t/2$, $c_2 = (t + c)/2$, $n_o = 1$. Com t = c = 1: $T_B(n) = 2^n - 1$, $c_1 = 1/2$, $c_2 = 1$, e em n = 1: 1 ≤ 1 ≤ 2 ✓. Logo $\mathbf{T_B(n) \in \Theta(2^n)}$.

(Intuição: a árvore de recursão é binária completa de altura n − 1, com $2^n - 1$ nós.)

3. Algoritmo C — Teorema Mestre, segunda linha

$T_C(n) = 9T_C(n/3) + c\cdot n^2$: a = 9, b = 3, k = 2, c > 0. $b^k = 3^2 = 9 = a$: **segunda linha** ($a = b^k$). Logo

$\mathbf{T_C(n) \in \Theta(n^2 \cdot \lg n)}$.

Forma fechada ($n = 3^m$, T(1) = t): $T_C(n) = n^2\cdot(t + c\cdot\log_3 n)$ — cada um dos m níveis da árvore custa $c\cdot 9^i\cdot(3^{m-i})^2 = c\cdot 9^m = c\cdot n^2$, mais as n^2 folhas de custo t.

Checagem com t = c = 1: $T_C(n) = n^2(\log_3 n + 1)$; $T_C(3) = 9 + 9 = 18 = 9\cdot(1 + 1)$ ✓. Testemunhas (t, c quaisquer): $c_1 = c/\lg 3$, $c_2 = (t + c)/\lg 3$, $n_o = 3$. Como $\log_3 n = \lg n / \lg 3$ e, para n ≥ 3, 1 ≤ lg n / lg 3: $(c/\lg 3)\cdot n^2 \lg n \leq n^2(t + c\log_3 n) \leq ((t + c)/\lg 3)\cdot n^2 \lg n$. Com t = c = 1: $c_1 = 1/\lg 3$, $c_2 = 2/\lg 3$; em $n = n_o = 3$, $n^2 \lg n = 9 \lg 3$, logo $c_1\cdot 9 \lg 3 = 9 \leq 18 \leq 18 = c_2\cdot 9 \lg 3$ ✓ (igualdade na cota superior).

$n_o = 1$ não serve: lg 1 = 0 e $T_C(1) = t > 0$.

4. De "$= c\cdot n^k$" para "$\in \Theta(n^k)$"

O enunciado diz "tempo linear", "tempo constante" e "$\theta(n^2)$", não exatamente $c\cdot n^k$. Isso não muda nenhuma classe, pelo seguinte lema de comparação.

Lema. Sejam T e T′ definidas pela mesma recorrência (mesmo a, mesma forma de chamada), com o mesmo valor no caso base e com $f(n) \leq f'(n)$ para todo n. Então $T(n) \leq T'(n)$ para todo n. Prova: indução sobre n (nas potências de b, ou em n ≥ 1 para B), usando a > 0: $T(n) = a\cdot T(\cdot) + f(n) \leq a\cdot T'(\cdot) + f'(n) = T'(n)$. ∎

Como $f(n) \in \Theta(n^k)$ e $f(n) > 0$ para todo n (combinar custa alguma coisa), existem c′, c″ > 0 com $c'\cdot n^k \leq f(n) \leq c''\cdot n^k$ para **todo** n ≥ 1: para $n \geq n_o$ isso é a definição de Θ, e para os finitos $n < n_o$ basta diminuir c′ e aumentar c″ ($f(n)/n^k$ é positivo em cada um deles). Pelo Lema, T fica entre as soluções com $f = c'\cdot n^k$ e com $f = c''\cdot n^k$, e as duas estão na mesma classe Θ (§1–§3, cujas formas fechadas e testemunhas valem para quaisquer t, c > 0). Logo $T_A \in \Theta(n^{\log_2 5})$, $T_B \in \Theta(2^n)$, $T_C \in \Theta(n^2 \lg n)$.

5. Comparação

C é assintoticamente mais eficiente que A. $n^2 \lg n / n^{\log_2 5} = \lg n / n^{\varepsilon}$ com $\varepsilon = \log_2 5 - 2 = \log_2(5/4) \approx 0{,}3219 > 0$. Aplicando L'Hôpital às funções de variável real x > 0 (o limite da sequência é o mesmo),
$\lim \lg x / x^{\varepsilon} = \lim (1/(x \ln 2)) / (\varepsilon\cdot x^{\varepsilon-1}) = \lim 1/(\varepsilon\cdot \ln 2\cdot x^{\varepsilon}) = 0$.
Então $n^2 \lg n \in o(n^{\log_2 5})$: para quaisquer constantes positivas c_2^C e c_1^A, a partir de algum n vale $c_2^C\cdot n^2 \lg n < c_1^A\cdot n^{\log_2 5}$, e portanto — com as duas cotas valendo no mesmo n, pela Observação abaixo — $T_C(n) < T_A(n)$.

A é assintoticamente mais eficiente que B. $n^{\log_2 5} \leq n^3$ para n ≥ 1, e $n^3/2^{n^A} \to 0$ (três aplicações de L'Hôpital, de novo em variável real: $x^3/2^x \to 3x^2/(2^x \ln 2) \to 6x/(2^x \ln^2 2) \to 6/(2^x \ln^3 2) \to 0$, cada seta significando "tem o mesmo limite que"). Logo $n^{\log_2 5} \in o(2^n)$. Ordem: **C ≺ A ≺ B**, isto é, $\Theta(n^2 \lg n) \prec \Theta(n^{\log_2 5}) \prec \Theta(2^n)$.

Observação — comparar A (definido nas potências de 2) com C (potências de 3) no mesmo n. O teorema é enunciado para n potência exata de b; é a hipótese "T assintoticamente não decrescente" que permite estender a conclusão a todo n. Esboço: se T é não decrescente para $n \geq N$ e $c_1 g(b^j) \leq T(b^j) \leq c_2 g(b^j)$ para $b^j \geq n_o$, então para $b^j \leq n < b^{j+1}$ (com $b^j \geq \max(n_o, N, b)$) vale
$T(n) \leq T(b^{j+1}) \leq c_2 g(b^{j+1}) \leq c_2 g(b\cdot n)$ e $T(n) \geq T(b^j) \geq c_1 g(b^j) \geq c_1 g(n/b)$, usando que g é não decrescente para n ≥ 1; para $g(n) = n^e\cdot \lg^p n$ (e > 0, p ≥ 0 — é o caso de $n^{\log_2 5}$ e de $n^2 \lg n$) tem-se $g(b\cdot n) \leq b^e\cdot 2^p\cdot g(n)$ e $g(n/b) \geq g(n)/(b^e\cdot 2^p)$ para $n \geq b^2$, de modo que a classe Θ(g) vale para todo n grande. Com isso, T_A e T_C são comparáveis no mesmo n.

A vantagem de C é assintótica: para as funções de referência (sem as constantes ocultas, que não se conhecem), a razão $\lg n / n^{0{,}3219}$ ainda vale ≈ 1,07 em $n = 2^{10}$ e cai para ≈ 0,23 em $n = 2^{20}$.

Resposta

A: $T(n) = 5T(n/2) + \Theta(n) \Rightarrow a = 5 > b^k = 2$ (terceira linha) $\Rightarrow \Theta(n^{\log_2 5})$, com $\log_2 5 \approx 2{,}32$.

B: $T(n) = 2T(n - 1) + \Theta(1) \Rightarrow$ fora do teorema (n − 1 não é n/b); com T(1) = t e custo de combinação constante c, a expansão dá $T(n) = 2^{n-1}(t + c) - c$, e pelo lema da §4 qualquer custo em Θ(1) fica entre duas dessas $\Rightarrow \Theta(2^n)$.

C: $T(n) = 9T(n/3) + \Theta(n^2) \Rightarrow a = 9 = b^k = 9$ (segunda linha) $\Rightarrow \Theta(n^2 \lg n)$.

O mais eficiente no pior caso é **C, com $\Theta(n^2 \lg n)$**; a ordem é C ≺ A ≺ B, porque $n^2 \lg n \in o(n^{\log_2 5})$ ($\log_2 5 > 2$ e $\lg n \in o(n^{\varepsilon})$ para ε > 0) e $n^{\log_2 5} \in o(2^n)$.

Questão 4 — prova por indução

> Enunciado
> "(1.0 ponto) Prove a assertiva abaixo usando o método da indução matemática.
> a) $1/2 + 1/4 + 1/8 + \cdots + 1/2^n < 1$, $\forall n \geq 1$"

Notação

Para cada inteiro $n \geq 1$, $S(n) = \sum_{i=1}^{n} 1/2^i$. A escrita $1/2 + 1/4 + 1/8 + \ldots + 1/2^n$ do enunciado é a notação usual para essa soma: só entram os termos com $i \leq n$. Para $n \leq 2$ a forma com reticências é apenas ilustrativa — vale a definição por $\sum$: $S(1) = 1/2$ (um único termo) e $S(2) = 1/2 + 1/4 = 3/4$.

Por que a indução ingênua (separando o último termo) não fecha

Tentando provar Q(n): $S(n) < 1$ separando o **último** termo, o passo seria
$S(n+1) = S(n) + 1/2^{n+1} < 1 + 1/2^{n+1}$ — e isso não é < 1. Usando só a informação "$S(n) < 1$", o passo exigiria a implicação, para todo real s, "se $s < 1$ então
$s + 1/2^{n+1} < 1$", que é falsa (contraexemplo da implicação, não da assertiva: $s = 31/32$ e $n = 3$ dão $31/32 + 1/16 = 33/32 \geq 1$). A hipótese "$S(n) < 1$" perde a informação de **quanto** falta para 1. Há duas saídas: fortalecer a hipótese (Prova 1) ou separar o **primeiro** termo em vez do último (Prova 2). As duas são provas completas; basta uma.

Prova 1 — fortalecendo a hipótese

Proposição. Para todo inteiro $n \geq 1$, P(n): $S(n) = 1 - 1/2^n$.

Base ($n = 1$). Lado esquerdo: $S(1) = 1/2$. Lado direito: $1 - 1/2^1 = 1/2$. Iguais: P(1) vale.

Hipótese de indução. Seja $n \geq 1$ um inteiro fixado e suponha P(n): $S(n) = 1 - 1/2^n$.

Passo. Separando o último termo da soma até $n + 1$:

$$\begin{aligned} S(n+1) &= S(n) + 1/2^{n+1} \\ &= 1 - 1/2^n + 1/2^{n+1} \leftarrow \text{h.i. aplicada a } S(n) \\ &= 1 - 2/2^{n+1} + 1/2^{n+1} \\ &= 1 - 1/2^{n+1}, \end{aligned}$$

que é P(n+1). Como P(1) vale e P(n) ⇒ P(n+1) para todo $n \geq 1$, pelo princípio da indução P(n) vale para todo $n \geq 1$.

Conclusão. Como $2^n > 0$, tem-se $1/2^n > 0$ e $S(n) = 1 - 1/2^n < 1$ para todo $n \geq 1$. ∎

Prova 2 — a própria assertiva como hipótese de indução, Q(n), separando o primeiro termo

Proposição. Para todo inteiro $n \geq 1$, Q(n): $S(n) < 1$.

Base ($n = 1$). $S(1) = 1/2 < 1$.

Hipótese de indução. Seja $n \geq 1$ um inteiro fixado e suponha Q(n): $S(n) < 1$.

Passo. Separando o primeiro termo e pondo $1/2$ em evidência no restante:

$$\begin{aligned} S(n+1) &= 1/2 + \sum_{i=2}^{n+1} 1/2^i = 1/2 + \sum_{j=1}^{n} 1/2^{j+1} = 1/2 + (1/2)\cdot\sum_{j=1}^{n} 1/2^j \\ &= 1/2 + S(n)/2 \\ &< 1/2 + 1/2 = 1 \leftarrow \text{h.i.: } S(n) < 1, \text{ logo } S(n)/2 < 1/2 \end{aligned}$$

(divide-se por $2 > 0$) isto é, Q(n+1). Como Q(1) vale e Q(n) ⇒ Q(n+1) para todo $n \geq 1$, pelo princípio da indução Q(n) vale para todo $n \geq 1$. ∎
(A troca de índice é $j = i - 1$, que leva $i = 2, \ldots, n+1$ em $j = 1, \ldots, n$; com $n = 1$ as duas somas têm um único termo, 1/4.)

Conferência

$S(1) = 1/2$, $S(2) = 3/4$, $S(3) = 7/8 = 1 - 1/8$. A identidade $S(n) = 1 - 1/2^n$, a desigualdade $S(n) < 1$ e a relação $S(n+1) = 1/2 + S(n)/2$ foram conferidas em aritmética racional exata para $n = 1, \ldots, 399$.

Resposta

Por indução em n, prova-se a forma mais forte P(n): $1/2 + 1/4 + \ldots + 1/2^n = 1 - 1/2^n$ (base: $1/2 = 1/2$; passo: $1 - 1/2^n + 1/2^{n+1} = 1 - 1/2^{n+1}$); daí a soma é < 1, pois $1/2^n > 0$. A indução direta em "< 1" separando o último termo não fecha; ela fecha separando o primeiro: $S(n+1) = 1/2 + S(n)/2 < 1/2 + 1/2 = 1$.

Questão 5 — Teorema Mestre em dez recorrências

> Enunciado
>
> "(0.5 ponto) Usando o teorema mestre, indique o limite assintótico usando a notação Θ, para cada uma das equações de recorrência abaixo. Considere o caso base T(1) = 1 ou T(n) = c, para alguma constante c e ∀n ≤ n0, para alguma constante n0. Suponha, também, que n uma [sic] potência exata de b. Escolha o valor adequado para b conforme a equação de recorrência a ser resolvida. Justifique todas as suas respostas. Respostas sem justificativas não serão consideradas. Pode acontecer de não ser aplicável a versão do teorema mestre discutida em sala de aula e também apresentada nessa lista de exercícios (veja em Informações que pode [sic] ser úteis). Nesse(s) caso(s) apresente o motivo pelo qual essa versão do teorema mestre não se aplica.
> (a) T(n) = T(n/3) + 1
> (b) T(n) = 2T(n/2) + n ∗ lgn
> (c) T(n) = T(2n/3) + n
> (d) T(n) = 5T(n/4) − n
> (e) T(n) = T(n − 1) + 3
> (f) T(n) = T(n/3) + 0
> (g) T(n) = 5T(n/4) − n²
> (h) T(n) = T(2n/4) + n
> (i) T(n) = 3/2T(n/4) − n²
> (j) T(n) = 3T(n/2) + n"
>
> Teorema mestre (literal, das "Informações que podem ser úteis"): "Sejam as constantes naturais a ≥ 1, b ≥ 2 e k ≥ 0. Considere a função f(n) da forma $c * n^k$, na qual c > 0 e ∈ R, seja a equação de recorrência dada por T(n) = aT(n/b) + f(n) e suponha n uma potência exata de b. Considere ainda que T é uma função que leva números naturais em números reais positivos e que T é assintoticamente não decrescente, Nessas condições, [BRASSARD and BRATLEY, 1996] indicam que a solução dessa recorrência é limitada assintoticamente por T(n) = $\theta(n^k)$ se a < b^k; $\theta(n^k * \lg n)$ se a = b^k; $\theta(n^{\log_b a})$ se a > b^k."

0. O que o teorema exige — a lista de conferência

Para aplicar esta versão é preciso ter, **todas ao mesmo tempo**:

(H1) a forma T(n) = a·T(n/b) + f(n) — uma única chamada "dividida por b", repetida a vezes;
(H2) a natural, a ≥ 1;
(H3) b natural, b ≥ 2;
(H4) f(n) = c·n^k exatamente, com c > 0 real e k ≥ 0 natural;
(H5) T com valores reais positivos e assintoticamente não decrescente (conferida abaixo nas potências de b, que é onde a recorrência define T).

Os casos, **na ordem em que o formulário os imprime**:

1ª linha: a < b^k → Θ(n^k); 2ª linha: a = b^k → Θ(n^k lg n); 3ª linha: a > b^k → Θ($n^{\log_b a}$). Nos itens aplicáveis confere-se o resultado com a forma fechada exata (T(1) = 1, n = b^m) e exibem-se testemunhas c_1, c_2, no verificadas em n = no. Nos itens não aplicáveis dá-se o motivo e, à parte, o que a recorrência realmente faz — o que mostra que a hipótese violada não é detalhe: em (f), (g) e (i), aplicar a fórmula às cegas daria resposta **errada** qualquer que seja o caso base; em (d), daria resposta errada com o caso base T(1) = 1 do enunciado (e com qualquer T(1) ≤ 4).

(a) T(n) = T(n/3) + 1 — APLICA

a = 1, b = 3, f(n) = 1 = 1·n^0 ⇒ c = 1 > 0, k = 0. (H1)–(H4) valem.
b^k = 3^0 = 1 = a: **2ª linha** ⇒ T(n) ∈ Θ(n^0·lg n) = **Θ(lg n)**.
Forma fechada (n = 3^m, T(1) = 1): T(3^m) = m + 1, isto é, T(n) = $\log_3$ n + 1. Positiva e crescente nas potências de 3: (H5) ✓.
Testemunhas: c_1 = 1/lg 3, c_2 = 2/lg 3, no = 3. Com $\log_3$ n = lg n / lg 3 e 1 ≤ lg n / lg 3 para n ≥ 3: (1/lg 3)·lg n ≤ $\log_3$ n + 1 ≤ (2/lg 3)·lg n. Em n = no = 3: 1 ≤ T(3) = 2 ≤ 2 ✓. no = 1 não serve: lg 1 = 0 e T(1) = 1 > c_2·0.

(b) T(n) = 2T(n/2) + n·lg n — NÃO APLICA

a = 2, b = 2 servem, mas **f(n) = n lg n não é da forma c·n^k** (viola H4). Não existem c, k com n lg n = c·n^k para todo n potência de 2: n = 2 daria 2 = c·2^k e n = 4 daria 8 = c·4^k, donde 2^k = 4, k = 2 e c = 1/2; mas em n = 8, c·8^k = 32 ≠ 24 = 8·lg 8. Nem mesmo "f ∈ Θ(n^k)" salvaria: para k ∈ {0, 1}, n lg n / n^k ≥ lg n → ∞; para k ≥ 2, n lg n / n^k ≤ lg n / n → 0. Então f não está em Θ(n^k) para k natural nenhum.
À parte (fora do teorema): por expansão, T(2^m) = 2^m·(1 + m(m + 1)/2), isto é, T(n) = n·(1 + lg n·(lg n + 1)/2) ∈ Θ(n $\lg^2$ n) (c_1 = 1/2, c_2 = 2, no = 2; em n = 2: 1 ≤ 4 ≤ 4).

(c) T(n) = T(2n/3) + n — NÃO APLICA

2n/3 = n/(3/2): o "b" seria **3/2, que não é natural nem ≥ 2** (viola H3). Também não há "n potência exata de b" inteira: $(3/2)^m$ = $3^m/2^m$, com numerador ímpar e denominador par, não é inteiro para m ≥ 1.
À parte: com piso, T(n) = T(⌊2n/3⌋) + n e T(1) = 1, vale n ≤ T(n) ≤ 3n (soma geométrica de razão 2/3), logo Θ(n) — conferido para n = 1…200 000.

(d) T(n) = 5T(n/4) − n — NÃO APLICA

a = 5, b = 4, mas f(n) = −n = (−1)·n^1: **c = −1 < 0** (viola H4). Com o caso base T(1) = 1, (H5) também falha: T(4) = 5 − 4 = 1 e T(16) = 5 − 16 = −11 < 0 — T deixa de ser positiva. À parte: com a recorrência aplicada desde n = 4, T(4^m) = (T(1) − 4)·$n^{\log_4 5}$ +

4n. A resposta dependeria do caso base: $\Theta(n^{\log_4 5})$ se T(1) > 4; exatamente 4n se T(1) = 4; negativa a partir de algum n se T(1) < 4. Com a outra opção do enunciado, T(n) = C para n ≤ n0 (chama-se aqui de C a constante do caso base, que o enunciado chama de c, para não confundir com o coeficiente c = −1 de f), o limiar muda: sendo 4^{m0} a maior potência de 4 que é ≤ n0 (n0 ≥ 1), $T(4^m) = ((C - 4\cdot 4^{m0})/5^{m0})\cdot n^{\log_4 5} + 4n$ para m ≥ m0; o sinal do coeficiente líder é o de $C - 4\cdot 4^{m0}$: positivo dá $\Theta(n^{\log_4 5})$, zero dá exatamente 4n, negativo faz T ficar negativa (por exemplo, n0 = 4 e C = 10 dão $T(4^6) < 0$). Aplicar o teorema às cegas (a = 5 > 4 = b^k, 3ª linha) daria $\Theta(n^{\log_4 5})$ — errado com T(1) = 1, quando T é negativa; só coincidiria com o comportamento real quando T(1) > 4 (ou $C > 4\cdot 4^{m0}$).

(e) T(n) = T(n − 1) + 3 — NÃO APLICA

A chamada é T(n − 1), não T(n/b) (viola H1): n − 1 = n/b exigiria b = n/(n − 1), que varia com n (vale 2 só em n = 2 e fica entre 1 e 2 para n ≥ 3) — não é uma constante. É uma recorrência subtrativa.
À parte: por expansão, T(n) = T(1) + 3(n − 1) = 3n − 2 (com T(1) = 1) ∈ Θ(n); $c_1 = 1$, $c_2 = 3$, $n_0 = 1$ ($n \le 3n - 2 \iff n \ge 1$; em n = 1: 1 ≤ 1 ≤ 3).

(f) T(n) = T(n/3) + 0 — NÃO APLICA

a = 1, b = 3, mas f(n) = 0 **não é da forma $c\cdot n^k$ com c > 0** (viola H4): $0 = c\cdot n^k$ obriga c = 0. A exigência não é formalidade: aplicando a fórmula com "c = 0, k = 0" (a = 1 = b^0, 2ª linha) sairia Θ(lg n), que é **falso**.
À parte: $T(3^m) = T(1)$ para todo m — T é constante, Θ(1) ($c_1 = c_2 = 1$, $n_0 = 1$, com T(1) = 1).

(g) T(n) = 5T(n/4) − n² — NÃO APLICA

a = 5, b = 4, mas f(n) = $-n^2$: **c = −1 < 0** (viola H4). E T não é positiva (viola H5): com T(1) = 1, T(4) = 5 − 16 = −11.
À parte: $T(4^m) = (T(1) + 16/11)\cdot n^{\log_4 5} - (16/11)\cdot n^2$. Como $\log_4 5 \approx 1{,}161 < 2$, o termo $-(16/11)n^2$ domina e T fica negativa a partir de algum n **qualquer que seja o caso base** (com T(1) = 1 já em n = 4; com T(1) = 10^6, em n = 4^{12}); com T(n) = C para n ≤ n0 muda só o coeficiente de $n^{\log_4 5}$, e o de n^2, −16/11, não depende do caso base. Às cegas (a = 5 < 16 = b^k, 1ª linha) sairia $\Theta(n^2)$ — absurdo para uma função que tende a $-\infty$.

(h) T(n) = T(2n/4) + n — APLICA

Simplificando o argumento: 2n/4 = n/2, logo **b = 2** (natural ≥ 2). a = 1, $f(n) = n = 1\cdot n^1 \Rightarrow c = 1 > 0$, k = 1. (H1)–(H4) valem.
$b^k = 2^1 = 2 > 1 = a$, isto é, $a < b^k$: **1ª linha** ⇒ T(n) ∈ **Θ(n)**.
Forma fechada (n = 2^m, T(1) = 1): T(n) = 2n − 1 (T(2) = 1 + 2 = 3 ✓). Positiva e crescente: (H5) ✓.
Testemunhas: $c_1 = 1$, $c_2 = 2$, $n_0 = 1$: $n \le 2n - 1 \iff n \ge 1$, e 2n − 1 ≤ 2n. Em n = 1: 1 ≤ 1 ≤ 2 ✓. A armadilha do item: ler "2n/4" e escolher b = 4 (ou a = 2). O valor adequado de b sai do argumento já simplificado.

(i) T(n) = (3/2)·T(n/4) − n² — NÃO APLICA

Lendo "3/2T(n/4)" como (3/2)·T(n/4): **a = 3/2 não é natural** (viola H2) e **c = −1 < 0** (viola H4) — dois motivos independentes. (Na outra leitura, 3/(2·T(n/4)), nem a forma a·T(n/b) + f(n) existe — H1; e em qualquer leitura f(n) = $-n^2$ tem c < 0. O veredito não depende da leitura.) E T não é positiva (H5): com T(1) = 1, T(4) = 3/2 − 16 = −29/2.
À parte: $T(4^m) = (T(1) + 32/29)\cdot(3/2)^m - (32/29)\cdot n^2$, negativa a partir de algum n para qualquer caso base, pois $(3/2)^m = n^{\log_4(3/2)}$ e $\log_4(3/2) \approx 0{,}29 < 2$ (com T(n) = C para n ≤ n0 muda só o coeficiente de $(3/2)^m$; o de n^2, −32/29, não depende do caso base). Às cegas (a = 3/2 < 16 = b^k, 1ª linha) sairia $\Theta(n^2)$ — absurdo para uma função que tende a $-\infty$.

(j) T(n) = 3T(n/2) + n — APLICA

a = 3, b = 2, $f(n) = n = 1\cdot n^1 \Rightarrow c = 1 > 0$, k = 1. (H1)–(H4) valem.
$b^k = 2 < 3 = a$: **3ª linha** ⇒ T(n) ∈ **$\Theta(n^{\log_2 3})$**, com $\log_2 3 \approx 1{,}585$.
Forma fechada (n = 2^m, T(1) = 1): $T(n) = 3\cdot n^{\log_2 3} - 2n$, isto é, $T(2^m) = 3^{m+1} - 2^{m+1}$ (T(2) = 3 + 2 = 5 = 9 − 4 ✓). Positiva e crescente: $T(2^{m+1}) - T(2^m) = 2\cdot 3^{m+1} - 2^{m+1} > 0$. Testemunhas: $c_1 = 1$, $c_2 = 3$, $n_0 = 1$: a cota inferior $3n^{\log_2 3} - 2n \ge n^{\log_2 3}$ equivale a $n^{\log_2 3 - 1} \ge 1$, verdadeira para n ≥ 1. Em n = 1: 1 ≤ 1 ≤ 3 ✓.

O caso base não muda as classes dos itens aplicáveis

Com T(1) = t > 0 em vez de 1: (a) T(n) = $\log_3 n$ + t; (h) T(n) = 2n − 2 + t; (j) $T(n) = (t + 2)\cdot n^{\log_2 3} - 2n$. Com T(n) = C para n ≤ n0 (a outra opção do enunciado, com a constante do caso base chamada de C como em (d); C > 0, pois o teorema pede T positiva, e n0 ≥ 1), sendo b^{m0} a maior potência de b que é ≤ n0: (a) $T(3^m) = C + (m - m0)$; (h) $T(2^m) = 2n + C - 2^{m0+1}$;
(j) $T(2^m) = ((C + 2\cdot 2^{m0})/3^{m0})\cdot n^{\log_2 3} - 2n$, para m ≥ m0. Em todos os casos o termo dominante tem coeficiente positivo e as classes Θ(lg n), Θ(n) e $\Theta(n^{\log_2 3})$ se mantêm; só as constantes c_1, c_2 (e o no das testemunhas) mudam.

Quadro-resumo

item	a	b	f(n)	hipótese violada	resposta
(a)	1	3	$1 = n^0$	—	$a = b^k = 1$, 2ª linha: $\Theta(\lg n)$
(b)	2	2	$n \lg n$	H4: não é $c \cdot n^k$	não aplica (de fato $\Theta(n \lg^2 n)$)
(c)	1	3/2	n	H3: $b = 3/2$	não aplica (de fato $\Theta(n)$)
(d)	5	4	$-n$	H4: $c < 0$; H5 com $T(1) = 1$	não aplica (T fica negativa com $T(1) = 1$)
(e)	—	—	3	H1: $T(n-1)$	não aplica (de fato $\Theta(n)$; $3n - 2$ com $T(1) = 1$)
(f)	1	3	0	H4: $c = 0$	não aplica (de fato $\Theta(1)$, não $\Theta(\lg n)$)
(g)	5	4	$-n^2$	H4: $c < 0$; H5	não aplica ($T \to -\infty$ para todo caso base)
(h)	1	2	n	—	$a = 1 < b^k = 2$, 1ª linha: $\Theta(n)$
(i)	3/2	4	$-n^2$	H2: $a = 3/2$; H4: $c < 0$; H5	não aplica ($T \to -\infty$ para todo caso base)
(j)	3	2	n	—	$a = 3 > b^k = 2$, 3ª linha: $\Theta(n^{\log_2 3})$

Resposta

(a) $\Theta(\lg n)$ — $a = 1$, $b = 3$, $k = 0$, $a = b^k$.
(b) Não se aplica: $f(n) = n \lg n$ não é da forma $c \cdot n^k$.
(c) Não se aplica: $b = 3/2$ não é natural ≥ 2.
(d) Não se aplica: $c = -1 < 0$ (e, com $T(1) = 1$, T fica negativa: $T(16) = -11$).
(e) Não se aplica: $T(n-1)$ não é da forma $T(n/b)$ (recorrência subtrativa).
(f) Não se aplica: $f(n) = 0$ exige $c = 0$, e o teorema pede $c > 0$ (a fórmula daria $\Theta(\lg n)$; o certo é $\Theta(1)$).
(g) Não se aplica: $c = -1 < 0$ (e T fica negativa: $T(4) = -11$ com $T(1) = 1$).
(h) $\Theta(n)$ — $2n/4 = n/2$, logo $a = 1$, $b = 2$, $k = 1$, $a < b^k$.
(i) Não se aplica: $a = 3/2$ não é natural, e $c = -1 < 0$.
(j) $\Theta(n^{\log_2 3})$ — $a = 3$, $b = 2$, $k = 1$, $a > b^k$.

Informações que podem ser úteis

Anexo da lista (transcrição literal)

Somatórios e Logaritmos:

- $\Sigma_{i=1}^{n} (i) = n(n+1)/2$
- $\Sigma_{i=1}^{n} (a^i) = a(a^n - 1)/(a - 1)$
- $\log (a * b) = \log a + \log b$
- $\log (a/b) = \log a - \log b$
- $\log_b a = \log_c a / \log_c b$
- $\log_b a^c = c * \log_b a$
- $\lg a = \log_2 a$

Hierarquia de funções:

A seguinte hierarquia de funções pode ser definida do ponto de vista assintótico:

$$1 < \log(n) < \sqrt{n} < n < n * \log(n) < n^c < c^n < n! < n^n$$

onde c é uma constante arbitrária inteira ≥ 2.

Teorema mestre:

Sejam as constantes naturais $a \geq 1$, $b \geq 2$ e $k \geq 0$. Considere a função $f(n)$ da forma $c * n^k$, na qual $c > 0$ e $\in R$, seja a equação de recorrência dada por $T(n) = aT(n/b) + f(n)$ e suponha n uma potência exata de b. Considere ainda que T é uma função que leva números naturais em números reais positivos e que T é assintoticamente não decrescente, Nessas condições, [BRASSARD and BRATLEY, 1996] indicam que a solução dessa recorrência é limitada assintoticamente por

$$T(n) = \theta(n^k) \text{ se } a < b^k; \quad \theta(n^k * \lg n) \text{ se } a = b^k; \quad \theta(n^{\log_b a}) \text{ se } a > b^k$$

Referências

- [BRASSARD and BRATLEY, 1996] BRASSARD, G. and BRATLEY, P. (1996). Fundamentals of Algorithms. Prentice-Hall.

O PDF da lista não tem folha de rosto nem bloco de instruções: começa na Questão 1, com numeração de página a partir de 2.

Sobre esta resolução. Fluxo da seção 5 do `CLAUDE.md`: chão determinístico em Python puro antes de qualquer redação (`verificacao/lista05-chao-deterministico.py`); redação; verificadores adversariais em contexto limpo, quatro por questão e por rodada, cegos ao raciocínio e uns aos outros, com

Bash para conferir as contas; voto assimétrico (4 HOLDS confirma, 2 HOLE refuta); correções; e reverificação da versão corrigida, até a confirmação. **Estado da verificação (14/09/2026).** Todas as cinco questões confirmadas por 4 de 4 verificadores independentes na última rodada: Q1 na 4ª rodada, Q2 na 2ª, Q3 na 3ª, Q4 na 2ª, Q5 na 5ª — 64 verificadores ao todo. Nenhuma correção mudou resposta final nem tocou enunciado; caíram justificativas e contagens (por exemplo, a leitura dupla de A[meio] na Q2 e uma frase introduzida pela própria correção da Q1).

Texto-fonte. Este HTML é gerado por `geradores/gerar_lista05.py` a partir dos arquivos `geradores/dados/lista05/q*-limpo.md` — os mesmos que os verificadores leram —, e o gerador confere que nenhuma linha deles caiu ou duplicou. Registro das correções, com o motivo e quem achou: `geradores/dados/lista05/correcoes-lista05.md`.

Enunciados. Transcritos literalmente do PDF do professor, com os erros de digitação marcados com *[sic]* e nunca corrigidos.

Lista avaliada — IFMG Campus Formiga, 2026/2.